Recovering Musical Dynamics for Expressive Automatic Music Transcription

Zhanhong HE

B.Sc. (Hons) in Engineering Science; B.Eng. in Electrical Engineering and Automation



This thesis is presented for the degree of Doctor of Philosophy of The University of Western Australia

School of Engineering
Department of Electrical, Electronic and Computer Engineering

2026

Thesis Declaration

I, Zhanhong He, certify that:

This thesis has been substantially accomplished during enrolment in the degree.

This thesis does not contain material which has been accepted for the award of any other degree or diploma in my name, in any university or other tertiary institution.

No part of this work will, in the future, be used in a submission in my name, for any other degree or diploma in any university or other tertiary institution without the prior approval of The University of Western Australia and, where applicable, any partner institution responsible for the joint award of this degree.

This thesis does not contain any material previously published or written by another person, except where due reference has been made in the text.

The work(s) are not in any way a violation or infringement of any copyright, trademark, patent, or other rights whatsoever of any person.

This thesis contains published work and/or work prepared for publication, some of which has been co-authored.

Signature: _____ Date: _____

Abstract

Automatic music transcription has matured in its treatment of note pitch and timing, but musical dynamics remains less fully addressed. This thesis treats dynamics as a missing expressive label layer that can be recovered from music data already partly specified, without rewriting note content or note timing. Its unifying question is *how musically interpretable dynamics labels can be recovered from incomplete, weakly expressive, or weakly supervised music data*. Two complementary targets within expressive music performance recur throughout: note-level MIDI velocity, the per-onset control variable used in MIDI workflows, and score-level dynamics, the beat-aligned markings such as *p*, *mf*, and *f* written in notation.

The thesis first addresses the symbolic-only setting by reformulating MIDI velocity completion as image colourisation on sparse piano-roll matrices. A U-Net with window attention and a sparsity-aware loss recovers the missing velocity layer, and the results are validated through cross-dataset objective metrics and a MUSHRA-inspired listening test.

It then turns to score-informed velocity estimation in the audio-plus-score setting, using a Transformer correction module that refines AMT-estimated velocity at aligned score onsets. Rather than replacing the AMT backbone, the module uses score structure as a late correction prior, transfers across the HPT, HPPNet, and DynEst backbones, improves cross-dataset behaviour from MAESTRO to SMD, and remains stable under audio–score timing shifts of up to several seconds.

A further study treats the audio-only setting with a score-facing output. It jointly estimates beat-synchronous dynamic markings, change points, beats, and downbeats from performance audio. A Bark-scale specific loudness front-end, a multi-scale encoder, and an MMoE decoder are combined to address the conflicting temporal requirements of dynamic-level estimation, change-point detection, and metrical localisation, producing output that can be written back into a score.

Finally, the thesis reports an exploratory extension beyond piano. To recover note-level velocity for instruments without direct labels, it treats the problem as renderer-conditioned analysis-by-synthesis and compares two differentiable synthesis interfaces. The proposed differentiable SoundFont proxy supervises velocity through note-wise loudness-related descriptors rather than waveforms. On guitar, this proxy interface adapts a piano-pretrained estimator more reliably than a differentiable waveform synthesiser under recording mismatch.

Taken together, these studies form a scenario map rather than an end-to-end pipeline. They show that dynamics recovery becomes tractable when the output layer, temporal coordinate, evidence source, and supervision regime are matched to one another. The thesis therefore extends expressive AMT beyond note detection by making existing music data more expressive, editable, and usable in settings where direct dynamics labels are partial or absent.

Keywords: automatic music transcription; expressive music performance; MIDI velocity; dynamic markings; score-informed velocity estimation; differentiable SoundFont proxy.

Acknowledgements

First, I would like to express my deepest gratitude to my supervisor, **Professor Roberto Togneri**. His coursework unit was the clearest and most inspiring one I took during my undergraduate studies. It showed me the light that led me into this journey. Throughout my PhD, I shared with him many immature ideas and repeated questions, and took far too much of his time, but he never complained once.

The same gratitude goes to my supervisor, **Professor Defeng (David) Huang**. After Roberto's retirement, David continued to support my PhD with great care. Although my research topic was not closely aligned with his own research, he read my writing in depth and gave detailed and thoughtful comments. My supervisors' guidance shaped not only this thesis, but also my daily life. In Chinese, we say that teachers are like fathers, and it is my greatest fortune to have had both of them.

I must thank my **family and relatives**. To my parents, Weihu He and Weili Chi: what you have given me can never be repaid. Your love, sacrifice, and support made everything possible. I have also received much happiness and warmth from my sister, Min Hu, and her family. My nieces, especially Jiahui, often called me to share every happy story with me. I am also so proud of their achievements in school, they have a wonderful future. I share my happiness with my younger brother, Zhanpeng He, who is completing his graduate studies and becoming a professional writer, and with my warmest uncle, Weida He, and his family. I have been overseas for too long, and some family members have left us along the way. I hope they are well in heaven.

Special thanks go to my partner, **Hanyu Meng**. Since we got together at Vivid Sydney, she has brought colour into my otherwise ordinary life and made every day warmer. We share similar topics, similar research interests, and many deep discussions, all of which made my PhD life more joyful and productive. More importantly, we went through many hard times of the PhD together, and we were lucky enough to hold each other through them. I am very lucky to have her, and I will always be grateful for her support, every moment we shared, and her love.

I am also deeply grateful to my **friends at UWA**, who have been an important support system throughout my PhD journey. Zhihui Lai and Kaibu Wang have stood by me since 2018. As the saying goes, "a friend in need is a friend indeed," and I am grateful to have them in my life. My close colleague, Yuliang Zhang, and his family spent many enjoyable days with me in everyday life and in the lab. Clint Yang and many other friends shared meals, hardships, and small moments of happiness with me. Their company made this long journey never lonely. I also thank the people I met during my internship at **Dolby Sydney** and **Huawei Hong**

Kong, my mentor David Cooper and many colleagues there. It was my honour to work with them, and the experience gave me many valuable discussions, perspectives, and memories.

Lastly, my sincere appreciation goes to the **Graduate Research School** and **EECE administration**, especially Leila Easter and Lea Towler, along with many other warm and supportive staff members. Their responses were always prompt, helpful, and kind. Staying at UWA has felt like being part of a family, and I will always remember and miss my time here.

This research was carried out while I was in receipt of a University of Western Australia International Fee Scholarship (UIFS) and a University Postgraduate Award (UPA). I sincerely thank the University for this financial support, which allowed me to pursue research topics that I found interesting without the financial pressure. I am also grateful to UWA High Performance Computing and IT (HPC-IT) for their strong support. The team consistently provided wholehearted assistance to our research, which was essential in completing my PhD.

Authorship Declaration: Co-authored Publications

This thesis contains work that has been published and/or prepared for publication.

Details of the work:
Zhanhong He, David Cooper, Defeng (David) Huang, Roberto Togneri
"Filling MIDI Velocity using U-Net Image Colorizer,"
Proc. 17th Int. Symp. Computer Music Multidisciplinary Research (CMMR),
pp. 949–960, 2025.
Location in thesis: Chapter 3
Student contribution to work: 80%

Details of the work:
Zhanhong He, Roberto Togneri, Defeng (David) Huang,
"Score-Informed Transformer for Refining MIDI Velocity in Automatic Piano Transcription,"
Proc. Sound and Music Computing Conference (SMC), accepted, 2026.
Location in thesis: Chapter 4
Student contribution to work: 80%

Details of the work:
Zhanhong He, Hanyu Meng, Defeng (David) Huang, Roberto Togneri,
"Joint Estimation of Piano Dynamics and Metrical Structure with a Multi-task Multi-Scale Network,"
Proc. IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP),
pp. 14607–14611, 2026.
Location in thesis: Chapter 5
Student contribution to work: 80%

Details of the work:
Zhanhong He, Hanyu Meng, Defeng (David) Huang, Roberto Togneri,
"Beyond Piano: Cross-Instrument MIDI Velocity Estimation via Differentiable SoundFont Proxies,"
Under review (*International Society for Music Information Retrieval (ISMIR)* Conference), 2026.
Location in thesis: Chapter 6
Student contribution to work: 80%

Student signature:

Zhanhong He:

Co-author signatures:

Defeng (David) Huang:

Roberto Togneri:

Hanyu Meng:

David Cooper:

I, Defeng (David) Huang, certify that the student statements
regarding their contribution to each of the works listed above are correct.

Coordinating supervisor signature:

Date:

Contents

Contents vii

List of Figures xiii

List of Tables xvii

List of Abbreviations xxi

1 Introduction 1

1.1 Motivation . . . 2
1.2 Dynamics as a Target in Expressive AMT . . . 3
1.3 Two Complementary Forms of Dynamics . . . 4
1.4 Research Gap and Thesis Stance . . . 5
1.5 Unified Thesis Question and Three Core Scenarios . . . 6
1.6 Aim, Scope, and Assumptions . . . 6
1.7 Main Contributions . . . 7
1.8 Thesis Organisation . . . 8

2 Background and Related Work 9

2.1 Musical Dynamics . . . 9
2.1.1 From physical sound level to symbolic dynamics . . . 9
2.1.2 Dynamics Roles in Music Performance . . . 10
2.2 Dynamics in audio . . . 11
2.2.1 From intensity and sound pressure to perceived loudness . . . 11
2.2.2 Bark-scale and critical bands . . . 13
2.2.3 Bark-scale specific loudness and total loudness . . . 14
2.2.4 Other Audio Representations Related to Loudness . . . 17
2.3 Dynamics in MIDI and Score . . . 18
2.3.1 Dynamic markings as macro-level musical instructions . . . 18
2.3.2 MIDI velocity as micro-level performance control . . . 19
2.3.3 Bridging the gap between dynamics representations . . . 19
2.4 Expressive MIR: Historical Overview and Task Landscape . . . 21
2.4.1 Performance research as a precursor to expressive MIR . . . 22

2.4.2 From note transcription to expressive transcription . . . 22
2.4.3 Three task lines of dynamics-related MIR . . . 23
2.4.4 Cross-cutting validity: temporal units, alignment, and priors . . . 24
2.5 Note-Level MIDI Velocity Recovery . . . 26
2.5.1 Expressive rendering and humanisation . . . 26
2.5.2 Symbolic velocity completion and prediction . . . 27
2.5.3 Audio-driven velocity estimation within AMT . . . 27
2.5.4 Score-informed MIDI velocity estimation and correction . . . 27
2.5.5 Piano bias and the beyond-piano boundary . . . 28
2.6 Beat-Synchronous Score-Dynamics Recovery . . . 28
2.6.1 From performed loudness to score dynamics . . . 28
2.6.2 Change points and dynamic segmentation . . . 29
2.6.3 Beat and downbeat tracking as structural coordinates . . . 29
2.6.4 Multi-scale and multi-task formulations . . . 29
2.7 Audio Rendering and Differentiable Synthesis . . . 30
2.7.1 Traditional renderers and the velocity control point . . . 30
2.7.2 Differentiable synthesis . . . 33
2.7.3 Analysis-by-synthesis and label-scarce velocity recovery . . . 33
2.8 Datasets, Evaluation, and Reproducibility Constraints . . . 34
2.8.1 Dataset modalities and what each chapter can learn . . . 34
2.8.2 Evaluation for MIDI velocity . . . 34
2.8.3 Evaluation for score dynamics . . . 36
2.8.4 Evaluation under proxy supervision . . . 36
2.8.5 Reproducibility, alignment quality, and licensing . . . 36
2.9 Summary of Gaps and Thesis Requirements . . . 37

3 Symbolic MIDI Velocity Completion as Image Colourisation 39
3.1 Chapter Overview and Position in the Thesis . . . 39
3.2 Task-Specific Related Work . . . 39
3.3 Problem Setting . . . 40
3.4 Representation as Image-like MIDI Matrices . . . 41
3.4.1 Matrix Representation . . . 41
3.4.2 MIDI Segmentation . . . 41
3.5 Model Architecture and Loss . . . 42
3.5.1 U-Net with Window Attention . . . 42
3.5.2 Sparsity-aware Loss Function . . . 43
3.6 Experimental Setup . . . 43
3.6.1 Datasets . . . 43
3.6.2 Training Setup . . . 44
3.6.3 Evaluation Metrics . . . 45
3.7 Quantitative Results . . . 45

3.8 Ablation Summary . . . 46
3.9 Perceptual Evaluation . . . 48
3.10 Discussion . . . 50
3.11 Summary . . . 50

4 Score-Informed Transformer Correction for AMT Velocity Estimation 51
4.1 Chapter Overview and Position in the Thesis . . . 51
4.2 Task-Specific Related Work and Motivation . . . 52
4.3 Problem Setting . . . 52
4.4 Modular Correction Principle . . . 53
4.5 Score-Informed Correction Module . . . 54
4.5.1 MIDI score features . . . 54
4.5.2 Architecture . . . 55
4.5.3 Loss function . . . 57
4.5.4 Post-Processing . . . 57
4.6 Experimental Protocol . . . 57
4.6.1 Datasets . . . 57
4.6.2 AMT backbones and training setup . . . 58
4.6.3 Evaluation metrics . . . 58
4.7 Results and Analysis . . . 59
4.7.1 Validation on the score-informed correction . . . 59
4.7.2 Cross-dataset generalisation . . . 60
4.7.3 Comparison with existing systems . . . 62
4.7.4 Robustness to audio–score misalignment . . . 63
4.8 Discussion . . . 64
4.9 Summary . . . 65

5 Audio-to-Score Dynamics via Joint Estimation of Piano Dynamics and Metrical Structure 67
5.1 Chapter Overview and Position in the Thesis . . . 67
5.2 Task-Specific Related Work . . . 67
5.3 Problem Setting and Targets . . . 69
5.4 Perceptually Grounded Features and Multi-Scale Backbone . . . 70
5.4.1 Audio feature representations . . . 70
5.4.2 Shared multi-scale encoder . . . 71
5.5 Joint Estimation with Task-Aware Decoding . . . 71
5.5.1 MMoE decoder . . . 71
5.5.2 Task-specific post-processing . . . 72
5.5.3 Loss function . . . 73
5.6 Experimental Setup . . . 74
5.6.1 Dataset and protocol . . . 74

5.6.2 Baselines . . . 75
5.6.3 Evaluation metrics . . . 76
5.7 Results, Case Study, and Discussion . . . 77
5.7.1 Main results . . . 77
5.7.2 Ablation study . . . 78
5.7.3 Cross-dataset evaluation . . . 80
5.7.4 Discussion: why score dynamics is not a relabelled MIDI velocity problem 81
5.7.5 Limitations . . . 82
5.8 Summary . . . 83

6 Beyond Piano: Cross-Instrument MIDI Velocity Estimation via Differentiable SoundFont Proxies 85
6.1 Chapter Overview and Position in the Thesis . . . 85
6.2 Task-Specific Related Work and Motivation . . . 86
6.3 Problem Setting . . . 88
6.4 Cross-Instrument Velocity Estimation Framework . . . 89
6.4.1 VeloEst: Shared Front-End Estimator . . . 89
6.4.2 Adaptation via Diff-Synth . . . 90
6.4.3 Adaptation via Diff-SFProxy . . . 90
6.4.4 Shared Regularisers and Total Objective . . . 91
6.5 Differentiable SoundFont Proxy . . . 92
6.5.1 Loudness-Related Acoustic Parameters . . . 92
6.5.2 Proxy Architecture . . . 93
6.5.3 MIDI Segment Sampling . . . 94
6.5.4 Teacher Rendering and Training Objective . . . 94
6.6 Experimental Setup . . . 95
6.6.1 Datasets . . . 95
6.6.2 SoundFont Selection . . . 96
6.7 Diff-SFProxy Velocity-Recovery Diagnostic . . . 96
6.7.1 Training Protocol . . . 96
6.7.2 Diagnostic Protocol . . . 96
6.7.3 Results . . . 97
6.8 Cross-Instrument VeloEst Adaptation . . . 97
6.8.1 Training Protocol . . . 98
6.8.2 Evaluation Metrics . . . 98
6.8.3 Results . . . 99
6.8.4 Ablations . . . 100
6.9 Discussion . . . 101
6.9.1 What the Recovered Velocity Actually Represents . . . 101
6.9.2 Why Pearson r on BSSL/BSTL . . . 102
6.9.3 When Diff-Synth Should Still Be Preferred . . . 103

6.9.4 Limitations . . . 103
6.10 Summary . . . 104

7 Conclusion **105**
7.1 Summary of Contributions . . . 106
7.1.1 A unified formulation of dynamics-oriented label recovery . . . 106
7.1.2 Symbolic MIDI velocity completion . . . 107
7.1.3 Score-informed velocity correction . . . 107
7.1.4 Audio-only score-dynamics recovery . . . 107
7.1.5 Beyond-piano velocity recovery through proxy supervision . . . 108
7.2 Cross-Chapter Findings . . . 108
7.2.1 Structure is a condition of recovery, not a convenience . . . 108
7.2.2 Velocity and score dynamics are not interchangeable labels . . . 109
7.2.3 Evaluation must match the claimed representation . . . 109
7.2.4 Weaker supervision changes the claim, not necessarily the usefulness . . . 109
7.3 Limitations and Scope . . . 110
7.4 Future Research Directions . . . 110
7.4.1 Probabilistic bridges between dynamics representations . . . 111
7.4.2 Uncertainty-aware score annotation . . . 111
7.4.3 Instrument-general dynamics coordinates . . . 111
7.4.4 Corpus-scale expressive data curation . . . 112
7.4.5 Towards expressive AMT beyond note detection . . . 112
7.5 Closing Remarks . . . 112

References **113**

List of Figures

1.1 The general architecture of an AMT system, illustrating the transformation of raw acoustic audio into foundational note events and expressive dynamic information. The outputs are MIDI data and musical scores. 2

1.2 Thesis roadmap within the expressive AMT pipeline. The diagram highlights the standalone sub-tasks of dynamics recovery and their corresponding chapters, ranging from symbolic MIDI velocity completion to direct audio-to-score dynamics estimation. 7

2.1 Layered view of dynamics used in this thesis. Physical sound level, perceived loudness, MIDI velocity, and score dynamics are linked but not interchangeable. Each transition is many-to-many or instrument-specific. 10

2.2 Equal-loudness contours comparing the Robinson and Dadson determinations in 1956 (black lines) with the revised ISO 226:2003 standard (red lines). Each contour connects frequency–SPL pairs perceived as equally loud. The label on each contour indicates its loudness level in phons, which corresponds to the SPL of an equally loud 1 kHz pure tone. The dashed lines represent the minimum threshold of hearing [1–3]. 13

2.3 Aligned temporal coordinates across audio, symbolic, and metrical representations 25

2.4 Velocity-to-loudness response of two piano renderers (Salamander Grand Piano and a Yamaha Disklavier Pro SoundFont). Both share the same input control range, but the realised loudness curves differ in slope and shape, which illustrates the renderer dependence of MIDI velocity. 31

2.5 Velocity-to-loudness response for non-piano SoundFonts (Spanish Classical Guitar and Violin LDK). Compared with Figure 2.4, the curves show different dynamic ranges and saturation behaviour, motivating the need for a renderer-aware supervision interface when working beyond piano. 32

3.1 Comparison between MIDI notes with human-performed velocity and music-software default velocity (64 when unspecified). The standard deviation of velocity (SD_{velo}) represents the dispersion of velocities around their mean across pitches. 40

3.2 Proposed U-Net architecture. The model input (F roll) comprises 88 pitch bins and 96 time frames. Attn block denotes windowed scaled dot-product attention. The final velocity roll (V roll) is generated during post-processing by extracting velocity at note positions and assigning each note the velocity at its onset. . . . 42

3.3 MIDI data distributions of the MAESTRO (blue) and SMD (orange) datasets, with density maps highlighting the similarity in their MIDI feature correlations. 44

3.4 Comparison of human-performed velocity with ConvAE and U-Net predictions. The U-Net result is more human-like than ConvAE, but for Chopin Op. 10 No. 4 the standard accuracy metrics do not fully reflect this difference. 47

4.1 Conceptual comparison between the proposed modular correction approach and earlier score-informed feature-fusion approaches. 53

4.2 Proposed score-informed MIDI velocity estimation framework. The AMT backbone provides a preliminary velocity estimate. The score-informed module refines this estimate using features extracted from the aligned MIDI score. 54

4.3 Velocity Estimation Branch in High resolution piano transcription (HPT) system [4]. 55

4.4 Lightweight Conformer-like Transformer encoder used in the score-informed correction module. 56

4.5 Qualitative comparison between HPT and Score-HPT (Transformer applied). The refined estimate is more concentrated around onset locations while remaining fully populated. 60

4.6 Learning curves for HPT and Score-HPT variants using the BiLSTM and Transformer correction module. All models are trained on the MAESTRO training split and evaluated every 10k iterations. 61

4.7 Robustness evaluation on data derived from the SMD dataset, using audio–score misalignments of ±0.1, 0.2, 0.3, 0.4, 0.5, 1, 1.5, 2, and 2.5 s. 64

5.1 Workflow for adding dynamic markings to an AMT-transcribed score from audio. The highlighted region (90–105 s of Chopin Mazurka Op. 6 No. 1) shows a case study. Dynamic markings are aligned with the musical beat grid, and a change point indicates a contextual shift in the dynamic levels. 68

5.2 Architecture of the proposed multi-task framework. A three-branch multi-scale network encodes either log-Mel or BSSL input at temporal resolutions T, T/s, and T/s^2, producing a latent sequence of shape $T \times 8$. An 8-expert MMoE with four gates forms task-specific features, followed by linear heads for the four targets. The scaling factor s is a tunable hyperparameter. 72

6.1 Overview of the cross-instrument velocity estimation framework. VeloEst is trained with ground-truth velocity labels on piano ($\mathcal{L}_{\text{velo}}$) and then fine-tuned for target instruments through one of two adaptation strategies. Diff-Synth provides waveform-domain supervision ($\mathcal{L}_{\text{audio}}$). Diff-SFProxy provides note-wise loudness-parameter supervision ($\mathcal{L}_{\text{params}}$). 89
6.2 An encoder-only Transformer maps padded note sequences to per-note PHE/OSF targets extracted from SoundFont-rendered audio. 93
6.3 Qualitative ablation on a guitar excerpt. From left to right: (a) the reference MIDI notes with ground-truth velocities; (b) the proposed VeloEst + Diff-SFProxy adaptation, whose predicted velocities track the dynamic contour of the reference; (c) the same model trained without the anti-collapse and saturation regularisers ($\mathcal{L}_{\text{anti}} + \mathcal{L}_{\text{satu}}$), where the predicted velocities collapse towards a near-constant median; (d) Diff-SFProxy adaptation without the MAESTRO-pretrained VeloEst initialisation, trained directly on guitar, where the predicted contour drifts away from the reference. The two failure modes correspond to the bottom two rows of Tables 6.2 and 6.3. 101

List of Tables

1.1 Two complementary forms of dynamics that recur throughout this thesis. The two representations differ in their nature, conceptual level, and primary carrier. . 4

1.2 The three core supervision scenarios studied in this thesis. Each scenario specifies the available data, the missing expressive layer, and the target output. 6

2.1 Three dynamics representations that recur throughout this thesis. 11

2.2 Approximate correspondence between phons and sones. 12

2.3 Approximate relation between critical-band rate in Bark and frequency in Hz. The full psychoacoustic scale spans 24 critical bands. 14

2.4 Score-dynamics vocabulary, loudness-related scales, and MIDI velocity scales . . 20

2.5 Representative AMT results across MAPS, MAESTRO, and MusicNet. The numbers show that velocity-aware transcription remains substantially harder than note transcription. 23

2.6 Temporal units that recur across the thesis. 26

2.7 Representative datasets relevant to expressive label recovery in this thesis. Each dataset is summarised by its instrument family, scale, and the expressive modalities (MIDI, velocity, score, score dynamics) it provides. 35

3.1 Selected SMD performances for the subjective listening test. SMD composer statistics and their overlap with the MAESTRO training set are also reported. . 44

3.2 Quantitative results on the MAESTRO test set. Arrows ↑ and ↓ indicate whether higher or lower values are better. Best score is highlighted in bold. 46

3.3 Quantitative results on the SMD dataset. Arrows ↑ and ↓ indicate whether higher or lower values are better. Best score is highlighted in bold. 46

3.4 Ablation study on the MAESTRO test set. *Attn.* denotes component ablation by adjusting the attention window, or removing attention. *Dur.* represents the MIDI segment duration, $< m, w >$ indicates loss with masking and weighting. An asterisk * marks the best configuration used for quantitative evaluation. . . . 48

3.5 Metrics for comparing different PianoTeq configurations against the reference recordings. The best-matching configuration, used to calibrate all listening-test stimuli, is highlighted in bold. 49

3.6 Listening-test results. MOS values with 95% confidence intervals are reported. "Most seen" composers include {Chopin, Bach, Beethoven}; "less seen" composers include {Rachmaninoff, Haydn, Mozart}; and "unseen" composers include {Bartók, Ravel}. Best score is highlighted in bold. 49

4.1 Comparison of HPT and Score-HPT on the MAESTRO test set. Lower values are better for MAE and SD_{ae}, while higher values are better for $\text{Recall}_{10\%}$. Best score is highlighted in bold. 59

4.2 Comparison on the SMD dataset. All models are trained only on the MAESTRO training split. Best score is highlighted in bold. 63

5.1 Beat-level distribution of five-level dynamic labels in the MazurkaBL training set, Vienna4x22, and Batik-plays-Mozart corpora. All three corpora share the same five score-dynamics labels, but their label priors differ substantially. 75

5.2 Performance comparison of the proposed model against all baselines on the MazurkaBL dataset. Per-task F1 scores (%) are reported as mean ± standard deviations over 5-fold cross-validation. PELT is an algorithmic method with no trainable parameters, while ANN did not report this attribute. The single-task parameter count is reported per model instance. The best score is highlighted in bold, and the second-best is underlined. 78

5.3 Ablation study of the multitask multiscale network with BSSL as input. Per-task F1 scores (%, mean over 5 folds) and their arithmetic average are reported. The full model uses MMoE, temporal scaling $s = 5$, 60-second audio segments, and data augmentation. The best score is highlighted in bold, and the second-best is underlined. 79

5.4 Cross-dataset evaluation on Vienna4x22 and Batik-plays-Mozart. Models are trained on MazurkaBL and applied to both external corpora without fine-tuning. Per-task F1 scores (%) report the best transferred MazurkaBL fold under the protocol described above. The best score is highlighted in bold, and the second-best is underlined. 80

6.1 Velocity-recovery diagnostic for Diff-SFProxy. Init and Recovery report per-note velocity MAE in MIDI units $[0, 127]$ before and after optimisation; Gain = (Init − Recovery)/Init × 100%. Within each instrument, the best gain is highlighted in bold. The 5 s configuration is the default used in the cross-instrument adaptation. 97

6.2 Evaluation on piano datasets, where ground-truth velocities are available. Diff-Synth and Diff-SFProxy back-ends process 5 s segments by default. Best score is highlighted in bold, and the second-best is underlined. The bottom two rows are ablations on Diff-SFProxy. 99

6.3 Evaluation on guitar datasets, where ground-truth velocities are not accessible. MAE_{Velo} is therefore omitted. Diff-Synth and Diff-SFProxy back-ends process 5 s segments by default. Best score is highlighted in bold, and the second-best is underlined. 100
6.4 MAESTRO metric diagnostic. Real audio is compared with GT-note resynthesis using GT or flat velocity under the Salamander (default) or YDP Grand Piano. Summary rows show GT-Flat separation and SoundFont drift. Best score is highlighted in bold, and the second-best is underlined. 102

7.1 Scenario-level summary of the thesis. 106

List of Abbreviations

AMT	Automatic music transcription
ANN	Artificial neural network
BCE	Binary cross-entropy
BFCC	Bark-frequency cepstral coefficient
BiLSTM	Bidirectional long short-term memory network
BPM	Beats per minute
BSSL	Bark-scale specific loudness
BSTL	Bark-scale total loudness
BWV	Bach-Werke-Verzeichnis (Bach works catalogue)
CE	Cross-entropy
CNN	Convolutional neural network
ConvAE	Convolutional auto-encoder
CosSim	Cosine similarity
CQT	Constant-Q transform
CRNN	Convolutional recurrent neural network
DAW	Digital audio workstation
dB	Decibel
dBFS	Decibels relative to full scale
DBN	Dynamic Bayesian network
DDSP	Differentiable digital signal processing
Diff-SFProxy	Differentiable SoundFont proxy
Diff-Synth	Differentiable synthesiser
DTW	Dynamic time warping
DynEst	Dynamics estimation network
FFN	Feed-forward network
FFT	Fast Fourier transform

FiLM	Feature-wise linear modulation
FL	François Leduc guitar dataset
GELU	Gaussian error linear unit
GPU	Graphics processing unit
GT	Ground truth
HMM	Hidden Markov model
HPT	High-resolution piano transcription
IOI	Inter-onset interval
ISO	International Organization for Standardization
KV	Köchel-Verzeichnis (Mozart catalogue)
Log-Mel	Logarithmic Mel-scale spectrogram
LSTM	Long short-term memory
MAE	Mean absolute error
MFCC	Mel-frequency cepstral coefficient
MIDI	Musical Instrument Digital Interface
MIR	Music information retrieval
MIREX	Music Information Retrieval Evaluation eXchange
MLP	Multi-layer perceptron
MMoE	Multi-gate mixture-of-experts
MOS	Mean opinion score
MSE	Mean squared error
MTL	Multi-task learning
MUSHRA	MUlti Stimulus test with Hidden Reference and Anchor
MusicXML	Music Extensible Markup Language
OSF	Onset-window spectral flux
PELT	Pruned exact linear time
PHE	Pitch-conditioned harmonic energy
RMS	Root mean square
RNN	Recurrent neural network
SD	Standard deviation
SemiCRF	Semi-Markov conditional random field
SFProxy	SoundFont proxy

SOTA	State-of-the-art
SPL	Sound pressure level
STFT	Short-time Fourier transform
TCN	Temporal convolutional network
U-Net	U-shaped encoder–decoder network
VeloEst	Velocity estimation network

Chapter 1

Introduction

A structurally correct sequence of notes is only the foundation of a musical performance. When performers receive a composed score, they do not merely execute a rigid sequence of instructions. They interpret the piece and imbue it with expressiveness. A central dimension of this interpretation is dynamics. This involves the deliberate shaping of loudness over individual notes, phrases, and larger formal units. Dynamics conveys musical intent and emotional nuance, balances musical texture, and is often the clearest signal that distinguishes a convincing human performance from a flat mechanical rendering.

Music performance is commonly represented in two complementary symbolic forms. The first is MIDI, where note events are paired with note-level expressive parameters such as velocity. MIDI velocity is precise, machine-readable, and often comes from the sensor measurements in musical instruments. The second is the music score, where dynamics is represented by human-readable markings such as *pp*, *p*, *mf*, *f*, *ff*. These markings are relative, context-dependent, and less precise as physical measurements, but they communicate musical intent on a musically meaningful metrical grid.

Automatic Music Transcription (AMT) aims to bridge human performance audio and symbolic music representations, including MIDI and scores. Driven by deep learning and large aligned datasets [5], recent AMT systems can transcribe note events with high accuracy. Yet this progress remains asymmetric [6]. The field is proficient at recovering what notes are played and when they occur. Recovering how loud those notes are played has progressed less, and so have other expressive dimensions, as depicted in Figure 1.1

This thesis posits that dynamics is a foundational component of expressive AMT. While music expression is multifaceted, dynamics represents the most perceptually salient and practically essential dimension. Unlike a standard transcription that yields flat intensities, a dynamics-aware approach enables expressive rendering and high-quality symbolic corpora curation. By focusing on the robust recovery of score dynamics and MIDI velocity, this work provides a critical layer of musical information.

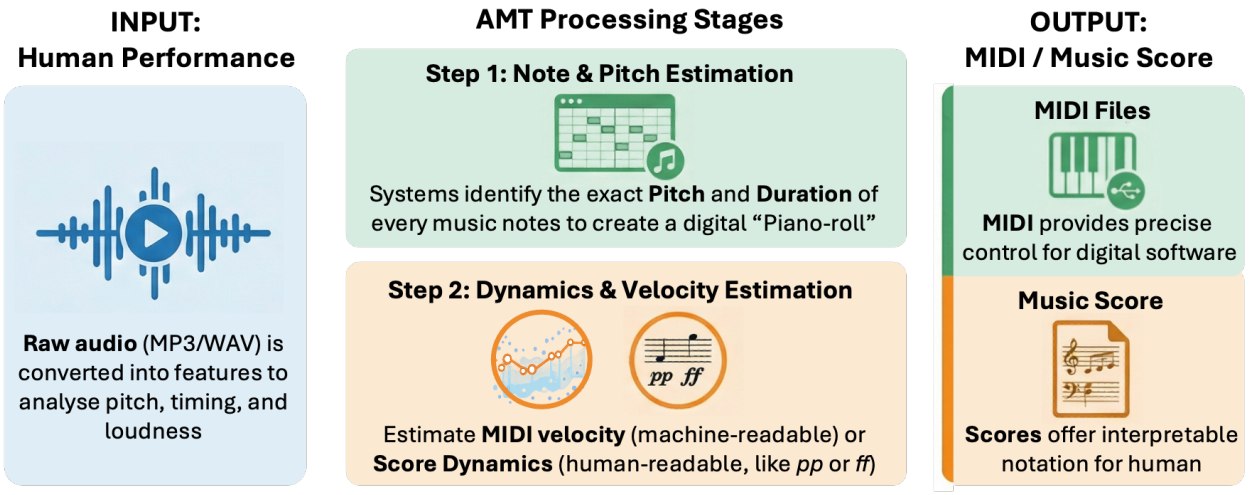


Figure 1.1: The general architecture of an AMT system, illustrating the transformation of raw acoustic audio into foundational note events and expressive dynamic information. The outputs are MIDI data and musical scores.

1.1 Motivation

Transcribing correct pitches and timing is only the foundational step of AMT. Incorporating dynamics into transcription results significantly enhances their capacity to carry expressive intent, taking the form of MIDI velocity or score dynamics. This is not only a technical improvement. Transitioning towards such expressive AMT unlocks significant practical benefits, as outlined below:

MIDI production and expressive rendering. MIDI is the dominant representation in digital music production, supporting efficient editing and flexible rendering. Yet, much software-generated MIDI is created via step input and quantisation, leaving expressive parameters at a lifeless, default constant. Among these, MIDI velocity is one of the most accessible control variables for shaping note intensity and expressive contour [7, 8]. Creators often want to preserve their composed notes and timing. To inject human-like expression, they simply need to adjust velocity values. Still, manual editing is time-consuming and delicate. Developing automated methods for velocity completion becomes an essential step for realistic rendering.

Music education and performance feedback. Score dynamics is also a core pedagogical dimension. In automatic piano performance assessment, dynamics is routinely treated alongside pitch, timing, and rhythm as one of the salient components of performance quality and feedback [9]. More recent work on query-based music coaching likewise emphasises that useful educational feedback needs interpretable information about expressive shaping, not only note correctness [10, 11]. In many educational scenarios, a reference score is already available together with a student performance audio. This makes score-informed dynamics estimation especially attractive, because the symbolic prior can stabilise the prediction while keeping the feedback musically meaningful.

Expressive data curation by enhancing existing archives. A third motivation comes from data curation. Many large-scale music archives provide aligned audio and symbolic content (MIDI sequences or digital scores). However, expressive labels are consistently missing. Velocities are often flattened to a default value, and score dynamic markings are absent or unreliable, limiting the utility of these corpora for downstream applications. Furthermore, a deep semantic gap exists between physical machine measurements (MIDI velocity) and subjective human cognition (score dynamics). Bridging this gap and recovering the missing dimensions is what turns static archives into expressive datasets.

By contrast, recollecting new datasets from scratch is a costly process constrained by audio-to-score synchronisation challenges, required musical expertise, and copyright restrictions [12–14]. Therefore, this thesis treats *label recovery* as a scalable, principled strategy. By retroactively estimating missing dynamics on existing corpora, we entirely bypass acquisition bottlenecks and annotation ambiguities, unlocking a vast, previously untapped resource for data-driven music research.

1.2 Dynamics as a Target in Expressive AMT

Expressive AMT extends beyond note transcription to capture the full aspects of a human performance. A comprehensive expressive transcription can be conceptually divided into three layers [15]:

- **Timing Layer:** Captures the temporal elasticity of a performance, including macro-level tempo variation, beat-level timing, inter-onset intervals (IOIs), and micro-level articulation.
- **Dynamic Layer:** Represents the intensity and energy of the music performance, bridging physical machine measurements (e.g., MIDI velocity) with semantic cognitive notations (e.g., score dynamic markings).
- **Control Layer:** Includes instrument-specific techniques and acoustic modifiers, such as sustain pedalling, vibrato, reverberation, and amplifier configurations.

While all three layers are essential for the computational understanding of expressiveness in performance, this thesis singles out the **Dynamic Layer** as its primary focus. This choice does not diminish the importance of timing or control parameters. Rather, dynamics is chosen to move AMT beyond notes for three reasons:

Perceptual salience. Dynamics is one of the most immediately audible dimensions of expressive performance. Listeners often perceive phrase direction, accent, and balance through intensity change before they perceive subtler aspects such as pedalling or micro-timing [15, 16]. A transcription without dynamics therefore ignores a central part of musical meaning.

Symbolic editability. Unlike complex continuous controls (e.g., guitar overdrive or piano pedalling), dynamics offers an exceptionally tractable and editable symbolic representation. MIDI velocity is a universal loudness variable for all instruments in digital audio workstations (DAWs), while score dynamics provide a standardised notational vocabulary for composition, pedagogy, and performance analysis [15, 17]. This dual status makes dynamics a robust bridge between low-level MIR outputs and higher-level musical practice.

Data scalability. Many expressive targets, particularly in the Control Layer, depend on fine-grained, instrument-specific mechanisms. Although recent datasets have begun to capture control parameters, such parameters remain sparse and weakly standardised [18–22] across different instruments. By contrast, dynamics has already proved tractable across several practical paradigms, including piano and singing-voice dynamics estimation [23, 24]. It thus provides a scalable entry point for expressive AMT, bypassing the bottleneck of instrument-specific data scarcity.

1.3 Two Complementary Forms of Dynamics

Having established dynamics as our primary expressive target, we clarify its two complementary symbolic forms within this thesis, as summarised in Table 1.1.

MIDI velocity. Originating from the physical sensors of electronic keyboards and pianos, MIDI velocity is a machine-centric measurement of physical intensity. To say it operates as a "note-level" parameter means that every music note from a discrete key strike is assigned an independent, static integer representing its initial strike force. This strict per-note measurement dictates that velocity functions as a discrete loudness control parameter. It is highly granular and absolute within its own scale, yet the resulting loudness highly depends on the acoustic environment and the specific instrument being played. Because MIDI velocity is necessary for MIDI-to-audio playback in DAWs and synthesisers, a common default is set to 64 when it is missing.

Score dynamics. Score dynamics represents the subjective cognitive intention of the composer or performer. Instead of attaching to isolated notes, markings like *p* or *crescendo* are conceptually anchored to the underlying metrical structure, acting as shared values across a specific beat

Table 1.1: Two complementary forms of dynamics that recur throughout this thesis. The two representations differ in their nature, conceptual level, and primary carrier.

Representation	Nature	Conceptual Level	Primary Carrier
MIDI velocity	Physical measurement	Note-level: isolated value at each note	DAWs, electronic instruments (synthesisers)
Score dynamics	Subjective intention	Metrical-level: shared value in a beat/phrase	Digital sheet music, human performers

or an entire phrase. This segment-level instruction provides a holistic perception of intensity while leaving room for free human interpretation. Because it is structurally aware and musically meaningful, score dynamics is the ideal target for pedagogical feedback and score analysis. In symbolic environments, the default assumption is often that missing score dynamics should be filled with *mf* (mezzo-forte), which is the neutral middle ground of human perceptual loudness.

1.4 Research Gap and Thesis Stance

This section identifies a narrower and more practical research gap than the general claim that AMT still lacks expressiveness. A recurring paradox in music research is that while structural data (pitch and timing) is increasingly abundant, expressive data remains systematically impoverished. A MIDI file may contain correct note sequences while leaving all velocities at a default constant. A performance corpus may preserve audio and aligned scores but only sparse, unreliable, or absent dynamic markings. The common problem is therefore how to recover a missing expressive label layer from data that are already partly specified.

The ontological gap: dynamics must be inferred across representations. This is not a simple label-conversion problem. Audio loudness is an acoustic and perceptual phenomenon. MIDI velocity is a machine-centric, physical parameter. Score dynamics is a cognitive, context-dependent notation of musical intention. These three forms are related but encode different concepts. A method that translates one to another must bridge the fundamental gap between objective physical measurement and subjective human cognition.

The ambiguity of ground truth: dynamics is more fragile than notes. For note transcription, pitch and onset often remain learnable under moderate changes in instrument, room, or recording conditions. Dynamics ground truth is less stable. Small changes in instruments (timbre), microphone placement, editorial convention, or synthesis setup can alter the apparent relation between sound, MIDI velocity, and score dynamics notation. For non-piano settings, the difficulty becomes sharper because the bottleneck is not ordinary domain shift but missing measurement itself: reliable MIDI velocity labels are rarely available, and in some cases even the ground truth is instrument-dependent.

Fragmented supervision regimes. The MIR community lacks a singular, omnipotent dataset that pairs audio with both cognitive score annotations and physical velocity labels. Supervision is deeply fragmented: some corpora provide aligned scores without performance control, while others provide precise velocity data but no human-readable score. A robust thesis on expressive dynamics cannot rely on an idealised, fully supervised data regime. It must systematically address these disjointed data environments.

Thesis stance: label recovery under structural preservation. For these reasons, this thesis adopts a unified *label-recovery* perspective. Rather than treating the problem as music

generation, full expressive rendering, or style transfer, it asks a pragmatic question: when pitch, onset, and alignment are already established, how can missing dynamics be recovered without rewriting note content or trusted timing? The goal is to make existing music data more expressive and directly usable by providing dynamics, whether as note-level MIDI velocity for production workflows, or as score dynamics for pedagogy and analysis.

1.5 Unified Thesis Question and Three Core Scenarios

The thesis is driven by the following question:

> **How can musically interpretable dynamics labels be recovered from incomplete, weakly expressive, or weakly supervised music data without rewriting note content or note timing?**

This question is studied under three core scenarios, outlined in Table 1.2. These scenarios represent a clear methodological progression. **Scenario S1** isolates the recovery task to a strict *symbolic-only* setting. **Scenario S2** introduces acoustic evidence but maintains a reliable structural prior via an *audio-plus-score* alignment. Finally, **Scenario S3** removes structural support entirely, tackling the fully open, *audio-only* setting to recover a *score-facing* target. By intentionally ordering these environments from the tightly constrained to the ambiguous, they form the core empirical arc of the thesis.

Table 1.2: The three core supervision scenarios studied in this thesis. Each scenario specifies the available data, the missing expressive layer, and the target output.

Scenario	Available Data	Missing Expressive Layer	Target Output
S1	MIDI note sequence	Note-level expressive intensity	MIDI velocity completion
S2	Performance audio and aligned MIDI (score)	Reliable note-level expressive intensity	Score-informed MIDI velocity refinement
S3	Performance audio only	Human-readable score dynamics and metrical coordinates	Beat-synchronous dynamic levels, beats, and downbeats

1.6 Aim, Scope, and Assumptions

Building upon the strict assumption of structural preservation established in Section 1.4, the empirical scope and boundaries of this thesis are defined as follows:

- **Primary empirical focus on piano.** The main experimental focus is piano music. This is a practical constraint, as reliable note-level velocity ground truth currently remains concentrated on pianos (benefited by Yamaha Disklavier systems [5]).

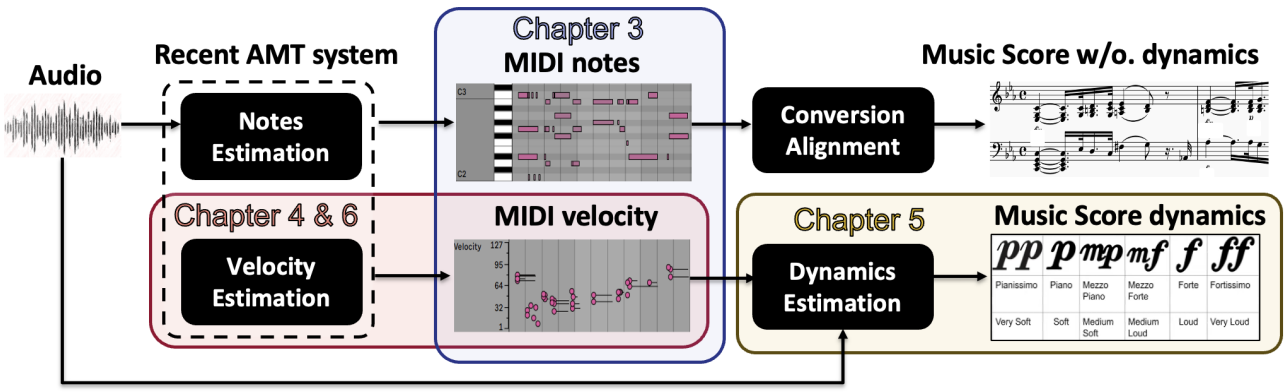


Figure 1.2: Thesis roadmap within the expressive AMT pipeline. The diagram highlights the standalone sub-tasks of dynamics recovery and their corresponding chapters, ranging from symbolic MIDI velocity completion to direct audio-to-score dynamics estimation.

- **Out-of-scope factors.** The thesis does not attempt full computational modelling of articulation, pedal, timbre, or micro-timing, except when these factors provide indirect context for dynamics recovery.
- **Exploration beyond the piano.** Extending velocity estimation beyond the piano is recognised as an important direction to counter the dataset bias in expressive AMT. However, large-scale non-piano score-dynamics transcription with human labels remains outside the scope of the core scenarios, and will only be investigated as an exploratory extension.

1.7 Main Contributions

The main contributions of this thesis are as follows.

- **A thesis-level formulation of dynamics-oriented label recovery.** The dissertation argues that dynamics should be treated as a primary target in expressive AMT and evaluates the robustness across diverse data distributions rather than relying on a single, idealised dataset.
- **A symbolic method for note-level velocity completion (Addressing S1).** It reformulates MIDI velocity completion as an image-colourisation problem on sparse piano-roll representations, introducing a U-Net architecture with window attention and a sparsity-aware loss [25].
- **A modular score-informed correction strategy (Addressing S2).** It introduces a lightweight Transformer correction module that refines AMT-estimated velocity using an aligned score. The design prioritises modularity and compatibility with existing AMT systems over a fully bespoke end-to-end architecture.
- **An audio-only framework for score-dynamics recovery (Addressing S3).** It proposes a compact multi-task multi-scale network that jointly estimates dynamic levels,

change points, beats, and downbeats. It innovatively combines a perceptually motivated Bark-scale specific loudness front-end with task-aware decoding through MMoE [26, 27].

- **An exploratory cross-instrument methodology.** It confronts the piano bias by investigating cross-instrument MIDI velocity estimation when true labels are scarce, employing a differentiable proxy for black-box synthesisers as part of the supervision signal.

1.8 Thesis Organisation

Referring to Figure 1.2, the core methodological chapters (Chapters 3–5) preserve the technical contributions of their respective peer-reviewed publications, adapted to form a unified narrative.

Chapter 2 establishes the shared conceptual system, defining dynamics across audio, MIDI, and score, while reviewing baseline AMT models, datasets, and evaluation strategies.

Chapter 3 addresses Scenario S1. Supported by listening tests, it demonstrates that recovering a decoupled expressive layer on fixed symbolic notes is enough to transform a mechanical MIDI sequence into an expressive performance.

Chapter 4 addresses Scenario S2. It details the Transformer correction module, demonstrating superior generalisation under dataset shift and robustness to audio-to-score misalignment.

Chapter 5 addresses Scenario S3. It presents the multi-task framework to extract human-readable score dynamics alongside their metrical anchors directly from audio.

Chapter 6 acts as an exploratory extension. It probes the frontier where direct velocity labels disappear, testing a route for cross-instrument MIDI velocity estimation via proxy supervision.

Chapter 7 synthesises the findings across the four studies, states the limitations clearly, and outlines future directions for expressive AMT.

Chapter 2

Background and Related Work

This chapter provides the shared foundations for the dissertation. It serves two purposes. The first three sections are introductory. They operationalise the objects and coordinates that recur across the later chapters: musical dynamics, perceived loudness in audio, and dynamics in MIDI and score. The remaining sections form a literature review aligned with the three thesis contribution lines and the data conditions under which they operate.

Section 2.1 operationalises musical dynamics across physical sound level, perceived loudness, MIDI velocity, and score-level symbols. Section 2.2 introduces loudness-oriented audio representations. Section 2.3 covers symbolic dynamics in MIDI and score, including alignment and representation choices that determine what supervision actually means. Section 2.4 then turns to the expressive-MIR landscape: it sketches the historical trajectory from performance research to deep-learning AMT and identifies the three task lines that anchor the thesis. Sections 2.5 and 2.6 review the first two of these literatures in depth: note-level MIDI velocity recovery and beat-synchronous score-dynamics recovery. Section 2.7 reviews the third: traditional and differentiable audio synthesis as it relates to dynamics, including the analysis-by-synthesis route to label-scarce velocity recovery. Section 2.8 surveys the datasets and evaluation conventions that constrain what each chapter can claim. The chapter ends by converting the literature gaps into explicit thesis requirements.

2.1 Musical Dynamics

The word *dynamics* is used in several related but non-identical ways across music theory, performance research, and MIR. That distinction is not a terminological luxury. It determines what target is being estimated, what evidence is relevant, and what evaluation is legitimate. Figure 2.1 provides the layered view of dynamics used throughout this thesis.

2.1.1 From physical sound level to symbolic dynamics

A useful starting distinction is between *physical intensity*, *perceived loudness*, *performance control*, and *score notation*. Physical measurements such as sound-pressure level in decibels describe the signal. Human listeners do not hear decibels directly. They hear loudness, which

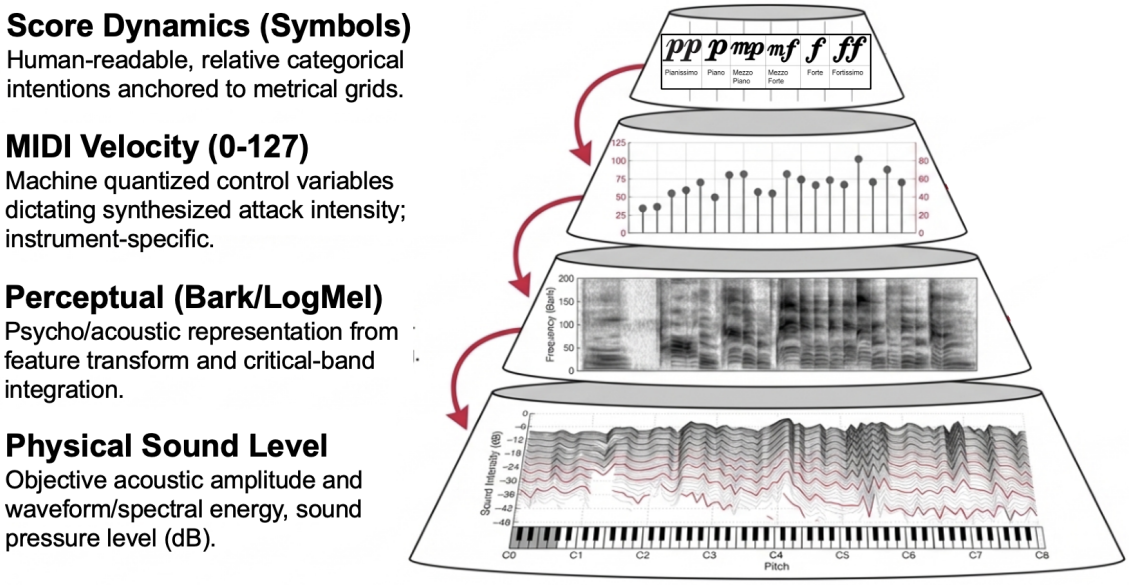


Figure 2.1: Layered view of dynamics used in this thesis. Physical sound level, perceived loudness, MIDI velocity, and score dynamics are linked but not interchangeable. Each transition is many-to-many or instrument-specific.

depends on frequency content, masking, and temporal integration [28, 29]. MIDI velocity describes a control quantity at note onset. In acoustic piano settings it is best interpreted as a proxy for performer action rather than as a universal loudness unit [7, 16]. Score dynamics such as *pp*, *mf*, *f*, or a hairpin do something else again. They encode relative musical intention in context [17, 30].

This layered view has an important methodological consequence. There is no single exact lookup table that converts score dynamics, loudness, and MIDI velocity into one another. Nominal mappings are convenient for notation software and rendering systems, but they remain conventions. They should not be mistaken for measurement standards. This point is central to the whole dissertation because each chapter targets a different layer of the same broader expressive problem. Table 2.1 summarises the three dynamics representations that recur throughout this thesis, together with their temporal anchors, strengths, and limitations.

2.1.2 Dynamics Roles in Music Performance

Dynamics is not only a change in intensity. In performance and analysis, it shapes phrase direction, marks local accent, balances melody against accompaniment, projects cadential arrival, and articulates formal contrast. A dynamic marking in a score therefore functions less like a fixed physical instruction and more like an underdetermined musical cue. Its realisation depends on texture, register, instrumentation, style, performer choice, and room response. A system may estimate intensity-related evidence correctly while still failing to produce a musically plausible symbolic label.

Performance analysis research further shows that dynamics does not act alone [31, 32]. Louder attacks can change timbral brightness. Local accent can be projected by both intensity

Table 2.1: Three dynamics representations that recur throughout this thesis.

Representation	Temporal anchor	Main strength	Main limitation
Audio loudness	frame-wise / continuous	Closest to what is directly observable from audio about perceived intensity	Entangled with timbre, pitch, room acoustics, the recording chain, and instrument-specific response
MIDI velocity	note onset	Precise, machine-readable, and editable in symbolic performance workflows	Acts as a control proxy, not a universal loudness scale; not directly comparable across instruments or renderers
Score dynamics	beat / phrase level	Human-readable, interpretable, and naturally attached to score structure	Relative, sparse, context-dependent, and expensive to verify at scale

and timing. Phrase endings often coordinate dynamics and tempo together. The present thesis does not model all expressive dimensions jointly. Nevertheless, it treats the dynamics targets studied later as proxies for a richer interpretive process rather than as self-sufficient physical quantities.

2.2 Dynamics in audio

In audio, musical dynamics are most naturally discussed through loudness, a psychoacoustic quantity. It is shaped not only by physical intensity or sound-pressure level but also by frequency-dependent hearing sensitivity, critical-band interaction, masking, and temporal integration [28]. This distinction matters throughout the thesis because the audio layer must later be related to MIDI and score. A frame with greater broadband energy is not necessarily perceived as dynamically stronger if the change is mainly timbral or spectrally redistributed.

For this reason, dynamics analysis in audio benefits from perceptually informed input features. Loudness-inspired features aim to approximate human intensity perception more directly than generic spectral magnitudes or raw energy measures. Recent singing-voice studies show that loudness-oriented and Bark-based representations are competitive with, and in some settings superior to, conventional log-Mel features for dynamics-related estimation tasks [24, 33]. This motivates the Bark-scale representation used later in the audio-only score dynamics component of this thesis.

2.2.1 From intensity and sound pressure to perceived loudness

The physical starting point is sound intensity, which measures the acoustic power per unit area. Let I denote the intensity measured in $\mathrm{W/m^2}$. Using a reference intensity I_0 near the threshold of hearing at 1 kHz (commonly $I_0 = 10^{-12}\,\mathrm{W/m^2}$), the corresponding intensity level

L_I is expressed in decibels (dB) as

$$L_I = 10 \log_{10}\left(\frac{I}{I_0}\right). \tag{2.1}$$

For example, if $I = 10^{-8}\,\mathrm{W/m^2}$, then $I/I_0 = 10^4$ and $L_I = 40\,\mathrm{dB}$. In practice, audio measurements more frequently use sound-pressure level (SPL) instead of intensity level, since pressure variations are what microphones actually capture. Let p denote the root-mean-square (RMS) sound pressure measured in pascals ($\mathrm{Pa} = \mathrm{N/m^2}$) and $p_0 = 20\,\mu\mathrm{Pa}$ be the reference pressure in air; the corresponding SPL L_p is similarly expressed in decibels:

$$L_p = 20 \log_{10}\left(\frac{p}{p_0}\right). \tag{2.2}$$

The logarithmic nature of the decibel scale is highly practical because acoustic energy spans many orders of magnitude, and human loudness perception is broadly compressive rather than linear. However, decibel levels do not represent an actual perceptual scale. For instance, a 60 dB tone at 100 Hz does not sound as loud as a 60 dB tone at 1 kHz, because human hearing sensitivity varies dramatically across frequencies. To address this discrepancy, equal-loudness contours were established through pioneering psychoacoustic experiments by Fletcher and Munson [34], refined by Robinson and Dadson [1], and now standardised in ISO 226:2003 [2], as illustrated in Figure 2.2. These contours map the sound pressure level (SPL) required across various frequencies to maintain a constant perceived loudness relative to a 1 kHz reference tone. From these contours, loudness level can be quantified in a unit called the *phon*.

In loudness-related analysis, the *phon* is still not the most convenient end point because it does not provide a linear scale of subjective loudness. A sound of 80 phons is not perceived as twice as loud as one of 40 phons. A more useful perceptual unit is the *sone*, which is designed such that its value is linearly proportional to perceived loudness. A sound of 2 sones is perceived as twice as loud as a 1-sone sound. In practice, there are two common routes to obtain sone values.

Table 2.2: Approximate correspondence between phons and sones.

Phons	40	50	60	70	80	90	100	110	120
Sones	1	2	4	8	16	32	64	128	256

The first is to convert the loudness level in phons into sones. Based on the loudness relations summarised by Backus [35], Table 2.2 gives the anchor pairs, while intermediate values are obtained by the standard approximation (where the unit of L_N is phons):

$$S = \begin{cases} 2^{(L_N - 40)/10}, & L_N \geq 40, \\ (L_N/40)^{2.642}, & L_N < 40. \end{cases} \tag{2.3}$$

The second route is to approximate perceived loudness directly from physical intensity via

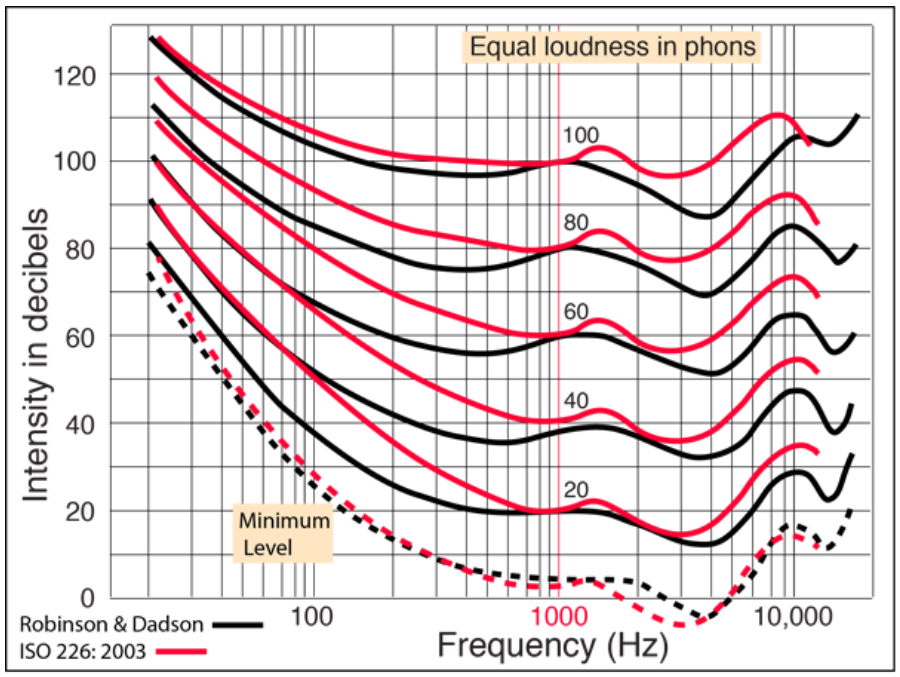


Figure 2.2: Equal-loudness contours comparing the Robinson and Dadson determinations in 1956 (black lines) with the revised ISO 226:2003 standard (red lines). Each contour connects frequency–SPL pairs perceived as equally loud. The label on each contour indicates its loudness level in phons, which corresponds to the SPL of an equally loud 1 kHz pure tone. The dashed lines represent the minimum threshold of hearing [1–3].

Stevens' power law, bypassing the need to read L_N from equal-loudness contours:

$$S = \frac{1}{15.849} \left(\frac{I}{I_0} \right)^{0.3}, \tag{2.4}$$

where S denotes loudness in sones [35]. In MIR practice, the first route is usually preferable because frequency dependence has already been made explicit through equal-loudness contours and can later be refined further by critical-band modelling.

2.2.2 Bark-scale and critical bands

The pure-tone framework above explains how physical level relates to perceived loudness for isolated tones. For musical audio, however, it is not sufficient to treat each FFT bin as an independent loudness event. Singing, piano, and polyphonic textures distribute energy across harmonics, formants, accompaniment, and transients, and the auditory system integrates nearby components within auditory filters whose bandwidth varies with frequency. This is the psychoacoustic motivation for *critical bands* [29].

The audible frequency range is conventionally organised into 24 critical bands indexed by the Bark scale, a psychoacoustic concept originally proposed by Zwicker in 1960 [36]. Table 2.3 shows the approximate relation between Bark index, frequency, and bandwidth. The important point is that Bark is a perceptual frequency coordinate rather than a loudness value: low-frequency bands are relatively narrow in Hz, whereas higher-frequency bands become progressively broader.

For sampled audio, the usable part of this scale is bounded by the Nyquist frequency. The table therefore provides the practical basis for deciding which Bark bands are retained at a given sampling rate, with the highest band treated according to the implementation when its frequency range is only partially available.

Table 2.3: Approximate relation between critical-band rate in Bark and frequency in Hz. The full psychoacoustic scale spans 24 critical bands.

Critical band rate z **[Bark]**	**Frequency** f **[Hz]**	Δf **[Hz]**	**Critical band rate** z **[Bark]**	**Frequency** f **[Hz]**	Δf **[Hz]**	**Critical band rate** z **[Bark]**	**Frequency** f **[Hz]**	Δf **[Hz]**
0	0	100	9	1080	190	18	4400	900
1	100	100	10	1270	210	19	5300	1100
2	200	100	11	1480	240	20	6400	1300
3	300	100	12	1720	280	21	7700	1800
4	400	110	13	2000	320	22	9500	2500
5	510	120	14	2320	380	23	12000	3500
6	630	140	15	2700	450	24	15500	–
7	770	150	16	3150	550			
8	920	160	17	3700	700			

A common analytic approximation from physical frequency f in kHz to Bark rate z is

$$z(f_{\mathrm{kHz}}) = 13 \arctan(0.76 f_{\mathrm{kHz}}) + 3.5 \arctan\left(\frac{f_{\mathrm{kHz}}^2}{7.5^2}\right), \tag{2.5}$$

where f_{kHz} denotes frequency in kHz [29, 37, 38]. This equation defines the Bark coordinate itself. It does not yet compute loudness.

Once this mapping is available, a linear-frequency spectrogram can be regrouped into Bark bands. If $P(k,t)$ denotes the power at frequency bin k and frame t, and $\mathcal{B}_b$ is the set of bins whose centre frequencies fall into Bark band b, then the Bark spectrogram for the retained Bark bands is

$$B(b,t) = \sum_{k \in \mathcal{B}_b} P(k,t), \qquad \mathcal{B}_b = \{k \mid z_{b-1} \le z(f_k) < z_b\}. \tag{2.6}$$

This step changes the spectral coordinate system from Hz to critical-band rate. The actual loudness model is introduced in the next subsection, where auditory weighting, spectral spreading, and nonlinear conversion are applied to obtain specific loudness.

2.2.3 Bark-scale specific loudness and total loudness

Among psychoacoustic-informed audio representations, Bark-scale specific loudness (BSSL) is designed to represent how perceived loudness is distributed across auditory critical bands, as human perception of loudness is more sensitive to certain frequency bands [28, 29]. Because it preserves both spectral location and perceptual magnitude, BSSL is well suited to dynamics-related MIR tasks [24, 26]. While several BSSL extraction methods exist, such as DIN 45631/A1:

2010 [39] and ISO 532-1:2017 [40], they are designed for industrial noise and sound quality assessment, assuming specific sound profiles. In MIR, however, music signals are typically highly polyphonic with negligible background noise. Therefore, the implementation proposed by Pampalk et al. [37, 38] is widely adopted. The extraction pipeline is defined as follows.

First, the audio is converted to mono, peak-normalised to $-1\,\mathrm{dBFS}$, and resampled to a chosen sampling rate f_s that preserves the spectral range required by the downstream MIR task. A short-time Fourier transform (STFT) then yields the complex spectrogram $X(k,t)$, where t denotes the frame index and k the frequency-bin index. The power spectrogram $P(k,t)$ is then formed using window-energy normalisation:

$$P(k,t) = \frac{|X(k,t)|^2}{(\sum_{n=0}^{N-1} w[n])^2}. \tag{2.7}$$

To approximate the frequency response of the human outer and middle ear, Terhardt's weighting function $W_{\mathrm{dB}}(f_k)$ [41] is applied. This weighting is specified in decibels, where f_k denotes the centre frequency (in kHz) of the k-th bin:

$$W_{\mathrm{dB}}(f_k) = -3.64\, f_k^{-0.8} + 6.5\, e^{-0.6\,(f_k-3.3)^2} - 10^{-3}\, f_k^4. \tag{2.8}$$

Since $P(k,t)$ is a linear-scale power quantity, this weighting must be converted to a linear power gain before being applied:

$$W(f_k) = 10^{\,W_{\mathrm{dB}}(f_k)/10}, \qquad P_W(k,t) = P(k,t) \cdot W(f_k). \tag{2.9}$$

The weighted power spectrogram $P_W(k,t)$ is then mapped to the Bark scale by the same pooling operation defined in Equation (2.6), yielding the weighted Bark spectrogram $B_W(b,t)$. In sampled audio, the number of usable Bark-band channels is constrained by the Nyquist frequency. Let B_{ret} denote the number of Bark channels retained after applying this sampling-rate and implementation constraint; the retained bands are therefore $b \in \{1, \ldots, B_{\mathrm{ret}}\}$. The notation below uses B_{ret} rather than fixing BSSL to a nominal 22- or 24-band representation.

Taking into account spectral masking over neighbouring Bark bands, where a strong component can make weaker nearby components harder to perceive, the spreading function proposed by Schroeder et al. [42] is applied:

$$G_{\mathrm{dB}}(\Delta) = 15.81 + 7.5(\Delta + 0.474) - 17.5\sqrt{1 + (\Delta + 0.474)^2}, \tag{2.10}$$

where $\Delta = b - j$ denotes the distance from source band j to target band b. Since G_{dB} is specified in decibels and $B_W(b,t)$ is represented on a linear power scale, the spreading function is converted to a linear power gain:

$$G(\Delta) = 10^{G_{\mathrm{dB}}(\Delta)/10}. \tag{2.11}$$

The spread Bark-scale excitation is then computed as

$$\widetilde{B}_W(b,t) = \sum_{j=1}^{B_{\text{ret}}} B_W(j,t)\, G(b-j), \qquad b = 1, \ldots, B_{\text{ret}}. \tag{2.12}$$

This step reflects a fundamental psychoacoustic principle: loudness perception is not a naive sum of independent narrowband energies, but involves auditory interaction across neighbouring critical bands.

Traditionally, converting a sound representation expressed in sound-pressure level to the perceptual *phon* scale requires mapping through equal-loudness contours. For the spread Bark excitation $\widetilde{B}_W(b,t)$, however, the previously applied outer/middle-ear weighting, Bark-band pooling, and spectral spreading have already introduced frequency-dependent auditory adjustments. Therefore, this implementation does not apply an additional equal-loudness contour lookup at this stage. Instead, the logarithmic level of $\widetilde{B}_W(b,t)$ is treated as a phon-like loudness-level input for the subsequent sone conversion. Following Zwicker's 1999 pipeline [29, 37, 38], this logarithmic conversion is defined as:

$$L_B(b,t) = \max\Big(0,\ 10\log_{10}\Big(\widetilde{B}_W(b,t) + \epsilon\Big)\Big), \tag{2.13}$$

where ϵ is a small constant used to avoid taking the logarithm of zero. The level $L_B(b,t)$ then serves as the input for the phon-to-sone conversion in Equation (2.3). The resulting Bark-scale specific loudness (BSSL), denoted as $N'(b,t)$, represents the specific loudness for Bark band b at frame t:

$$N'(b,t) = \begin{cases} 2^{(L_B(b,t)-40)/10}, & L_B(b,t) \geq 40, \\ (L_B(b,t)/40)^{2.642}, & L_B(b,t) < 40. \end{cases} \tag{2.14}$$

The resulting BSSL is a B_{ret}-dimensional time–frequency representation of perceived loudness, with $N'(b,t)$ measured in sone/Bark. The corresponding Bark-scale total loudness (BSTL), denoted as $N_{\text{tot}}(t)$, collapses this frequency-resolved representation into a single time-varying scalar by integrating over the retained Bark axis:

$$N_{\text{tot}}(t) = \int_0^{B_{\text{ret}}} N'(z,t)\, dz. \tag{2.15}$$

In the discrete retained-band implementation, this is approximated as:

$$N_{\text{tot}}(t) \approx \sum_{b=1}^{B_{\text{ret}}} N'(b,t)\Delta z, \qquad \Delta z = 1. \tag{2.16}$$

In other words, BSSL preserves where loudness is located in the auditory spectrum, whereas BSTL retains only the overall perceptual magnitude at each time frame. The two representations are therefore complementary: BSTL is appropriate when a compact loudness curve is sufficient, while BSSL is preferable when the spectral distribution of loudness is expected to carry musically

relevant information. This distinction is visualised later in Figure 2.3(b)–(c), where BSSL retains the Bark-wise loudness distribution while BSTL reduces it to a single trajectory.

2.2.4 Other Audio Representations Related to Loudness

Besides explicit loudness measures, MIR also relies on time–frequency representations that retain amplitude information with different degrees of perceptual relevance. At the simplest level, short-time RMS energy provides a frame-level estimate of signal power and is widely used as a baseline descriptor of local intensity. However, RMS remains only a coarse proxy for perceived loudness: it reduces each analysis frame to a single scalar and therefore cannot represent how dynamic impression also depends on the spectral distribution of energy, rather than on overall level alone. This limitation is important because perceived dynamics are not determined by level alone. The pitch, loudness, and timbre can jointly affect perceived instrument dynamics, and broader MIR work on musical affect and tension also treats loudness as only one cue among several acoustic factors [43–45].

For this reason, sub-band spectral representations are often more informative than scalar energy features for dynamics-related MIR tasks, especially the Mel-based representations [43, 46–48]. A Mel spectrogram aggregates spectral energy into perceptually motivated frequency bands, preserving not only how much energy is present but also where that energy lies in the spectrum [49]. A log-Mel spectrogram further applies logarithmic compression, reducing the dominance of very large amplitudes, stabilising the representation across a wide dynamic range, and making it more consistent with the nonlinear character of loudness perception. Nevertheless, log-Mel does not explicitly model human loudness perception. Consequently, recent works in loudness estimation typically employ log-Mel as a standard baseline against which the superior loudness modelling capabilities of BSSL are compared [24, 26].

Compact cepstral features such as MFCCs and BFCCs provide lower-dimensional summaries of filterbank energies on perceptual frequency scales. MFCCs are derived from Mel-scale filterbank energies and have long been used in speech and music modelling [50, 51], while BFCCs apply the same general cepstral idea on the Bark scale [52]. These features are attractive when compactness, decorrelation, and robust timbral description are important, and they remain common in MIR applications such as music classification and instrument recognition [43, 46, 53]. However, this compression is also their main limitation for fine-grained dynamics analysis: by transforming sub-band energies into a compact cepstral representation, they remove part of the band-wise structure that may carry local loudness, resonance, and spectral-balance cues. BFCCs inherit the same trade-off on the Bark scale. For dynamics-focused modelling, directly retaining Mel/Bark band energies or using specific-loudness representations is therefore usually easier to justify than relying on cepstral summaries alone [24, 26, 48, 54].

By contrast, the Constant-Q Transform (CQT) is designed around logarithmically spaced frequency bins whose spacing aligns closely with musical pitch [55]. This makes CQT well suited to pitch-, harmony-, and transcription-oriented tasks, where the representation benefits from a frequency axis organised according to musical scale structure [48, 56]. CQT can still

retain amplitude variation implicitly, but its frequency organisation is not based on critical-band loudness or specific-loudness modelling. For that reason, CQT is better understood as a pitch- and harmony-oriented representation than as a primary representation for loudness or musical dynamics.

2.3 Dynamics in MIDI and Score

Audio captures the sounding result of dynamics: pressure variation, spectral energy, and perceived loudness. Symbolic formats describe dynamics through a different kind of abstraction. In a score, dynamics are written as *dynamic markings*: sparse, human-readable instructions such as *p*, *f*, and *crescendo*. In MIDI, dynamics are represented primarily by *velocity*: a dense, machine-readable control value attached to individual note onsets. These two representations are connected through performance, but they are not interchangeable measurements. Dynamic markings describe an expressive intention over musical context; MIDI velocity records or controls local note attack. The gap between them is therefore a central modelling problem rather than a formatting problem.

2.3.1 Dynamic markings as macro-level musical instructions

Dynamic markings are the conventional score vocabulary for notated loudness and dynamic change. They include static markings such as *pp*, *p*, *mp*, *mf*, *f*, and *ff*, as well as directional indications such as *crescendo*, *diminuendo*, and hairpins. Although a marking is printed at a specific visual position, its musical scope is usually broader than that position. A single *p* may govern a phrase, a texture, or a formal region until a later marking or musical event changes the dynamic state. A crescendo indicates a tendency over a span, not an isolated value at one note.

The crucial point is that dynamic markings are not calibrated volume labels. They are relational instructions. A *forte* passage is not defined by a fixed sound-pressure level, nor does a *piano* passage prescribe a fixed physical intensity. The intended realisation depends on register, texture, melodic role, harmonic tension, instrumentation, acoustic setting, historical style, and the performer's interpretive priorities. A melody marked *p* may require more acoustic energy than an accompaniment marked *mf*; an inner voice may remain physically soft while sounding perceptually salient because of register or timbral contrast. This context dependence is central to empirical work on the relation between dynamic markings and performed loudness [17, 30].

Dynamic markings are therefore macro-level representations. They compress many local performance decisions into a small number of expressive symbols. Their value lies precisely in this compression: they allow a composer or editor to specify the character of a passage without prescribing every note attack. Expressive performance research treats dynamics as one dimension of a broader transformation from score structure into timing, intensity, articulation, and timbral choices [15, 57]. Recent modelling work on piano performance also shows that performers produce systematic dynamic profiles from phrase structure, metrical position, register, and style, even when explicit dynamic markings are removed [58]. Dynamic markings should

therefore be understood as musically precise but physically under-specified instructions.

For MIR, this has a direct consequence. A dynamic marking is a weak but meaningful label. It is weak because it does not determine exact amplitude, MIDI velocity, or loudness. It is meaningful because it identifies the expressive region that a listener, performer, or teacher can recognise in the score. In this thesis, the term *dynamic markings* is used for this score-facing representation throughout, and its practical coordinate is the beat, bar, or phrase rather than the audio frame.

2.3.2 MIDI velocity as micro-level performance control

MIDI velocity is a different kind of representation. It is attached to a note onset rather than to a phrase or region. In conventional MIDI performance data, a sounding note onset carries a discrete velocity value on a finite integer scale. In a chord, each note may have a different velocity. In a rapid passage, every note can carry its own local attack value. MIDI velocity is therefore a micro-level representation: it serialises note-level actions rather than summarising the expressive state of a passage.

The word *velocity* is often described informally as key-strike force. For keyboard instruments this is useful but imprecise. In sensor-equipped acoustic pianos, the recorded value is more accurately treated as a discretised proxy for key or hammer motion near the note attack. Studies of computer-controlled pianos distinguish the player's physical action, the hammer–string interaction, the measured control value, and the resulting acoustic signal [16, 59]. Large piano datasets such as MAESTRO use this property to provide aligned audio and MIDI data with key-strike velocity labels, making note-level velocity a practical target for MIR systems [5].

MIDI velocity is exact as symbolic data. A note with velocity 80 remains velocity 80 when the MIDI file is copied, edited, or passed to a model. It is also mechanical in the sense that it refers to a local performance control: how strongly a note attack is executed or how strongly a synthesiser should respond. This objectivity is one reason why velocity is attractive for automatic transcription, expressive rendering, and symbolic performance modelling [23, 60, 61].

However, exactness in the data structure does not imply universality in sound. MIDI velocity is not loudness, not amplitude, and not a dynamic marking. It is a control parameter whose acoustic effect depends on the instrument, renderer, register, sample layer, synthesis design, and recording chain. Dannenberg's analysis of MIDI velocity shows that different synthesisers and patches implement substantially different velocity-response curves [7]. Thus, a MIDI value can be objective without being perceptually absolute. It is exact as a note-level control value, but relative as an acoustic and musical quantity.

2.3.3 Bridging the gap between dynamics representations

A tempting solution is to convert between MIDI velocity and dynamic markings with a fixed lookup table. Such a table might assign low velocities to *p*, middle velocities to *mf*, and high velocities to *f*; the inverse table would assign each printed marking a nominal playback velocity.

Table 2.4 illustrates how representative notation software (Logic Pro, MuseScore, Sibelius, Finale, Dorico) implements such mappings between score-dynamics symbols, loudness-related scales, and MIDI velocity values. This is useful for deterministic playback in notation software and digital audio workstations. It is not a reliable model of musical dynamics. Formally, the naive assumption is that a single scalar function can map a note-level value v to a score-level marking d:

$$d = g(v), \qquad d \in \{pp, p, mp, mf, f, ff\}. \tag{2.17}$$

Table 2.4: Illustrative correspondences between score-dynamics vocabulary, loudness-related scales, and MIDI velocity values in representative music software.

Loudness	**Score Dynamics**	**MIDI Velocity**				
sones	**Symbol & Meaning**	**Logic Pro** [62]	**MuseScore** [63]	**Sibelius** [64]	**Finale** [8]	**Dorico** [8]
128	*ffff*: threshold of pain	–	127	–	127	123
64	*fff*: yelling	127	126	127	114	119
32	*ff*: very loud	112	112	113	101	101
16	*f*: loud	96	96	98	88	89
8	*mf*: normal speaking	80	80	84	75	77
8	*mp*: normal speaking	64	64	71	62	61
4	*p*: soft	48	49	61	49	46
2	*pp*: almost a whisper	32	33	39	36	25
1	*ppp*: whispering	16	16	20	23	14
0	*pppp*: threshold of hearing	–	10	–	10	5

Notes. The table is illustrative rather than normative. The listed values reflect representative software mappings and do not define a universal conversion between score dynamics and performed loudness. Logic Pro entries refer to Apple's documented step-input velocity buttons rather than score-playback dynamics [62, 65]. Finale and Dorico entries follow the comparative values reported in [8].

The failure of this assumption has two main causes. Both follow from the same representational problem: MIDI velocity, perceived loudness, and score dynamics live on different coordinates. A velocity value is attached to a note onset, a loudness impression is distributed over time and frequency, and a score marking describes a contextual musical state. Any direct lookup table therefore discards the intermediate stages that make dynamics musically meaningful.

Instrument dependence. MIDI velocity is physically arbitrary across instruments. The same velocity value may trigger different amplitudes, spectra, sample layers, envelopes, or synthesis behaviours depending on the instrument. Velocity 80 on a sampled grand piano may produce a bright attack and strong hammer noise; velocity 80 on a soft synthesiser patch may mainly open a filter or change an envelope; velocity 80 on another renderer may produce only a small loudness change. Even within piano, the relationship between velocity and acoustic output depends on pitch, register, hammer mechanics, resonance, microphone placement, and the reproduction system [7, 16]. A fixed velocity threshold therefore cannot define an instrument-independent boundary between *mp*, *mf*, and *f*.

This problem also affects the reverse direction. Assigning *f* to a single nominal velocity may be adequate for rough playback, but it ignores the instrument-specific response needed to make

a passage sound forte-like. A convincing *forte* on one instrument may require a different velocity range, spectral balance, or articulation strategy on another. Rule-based approaches to musical dynamics can impose useful rendering conventions, but they cannot remove the instrument dependence of the underlying control-to-sound mapping [66].

Perceptual loudness and performer style. The second obstacle is perceptual and musical. Human listeners do not hear MIDI velocity values. They hear loudness, salience, balance, and expressive shape. Loudness perception is nonlinear and depends on frequency-dependent sensitivity, critical-band interaction, masking, and temporal integration [28, 29]. A dense chord played with moderate note velocities can be perceived as louder than a single note with a higher velocity because total loudness depends on how energy is distributed across frequency bands and time. Conversely, a high-velocity note may be masked by surrounding texture or may fail to dominate if it occupies a less perceptually prominent register.

Performer style adds another layer. A pianist's realisation of *forte* is shaped by phrase direction, metrical hierarchy, historical idiom, local texture, and personal expressive strategy. Empirical studies of dynamic markings and performed loudness show that the mapping between notated symbols and realised loudness is learnable within constrained contexts but unstable across pieces, performances, and performers [30]. Related modelling work shows that different pianists can agree on broad dynamic tendencies while still differing in the local strategies used to produce them [58]. A dynamic marking is therefore not a target number; it is an interpretive category.

The bridge between MIDI velocity and dynamic markings must consequently model context. It must account for the instrument that turns velocity into sound, the auditory system that turns sound into perceived loudness, and the performer or annotator who turns musical context into an expressive label. A lookup table collapses these stages into one scalar threshold. The later chapters of this thesis instead treat symbolic dynamics recovery as a structured MIR problem: note-level velocity estimation, score-informed correction, and beat-synchronous dynamic-marking recovery address different parts of the same many-to-many relationship between machine-readable performance control and human-readable musical notation.

2.4 Expressive MIR: Historical Overview and Task Landscape

The previous three sections defined what *dynamics* means at each layer of representation. They did not yet describe how the MIR community has tried to recover those representations from data. The present section serves that purpose. It is the first literature-review section of the chapter, and it sets up the three task-specific reviews that follow (Sections 2.5, 2.6, and 2.7).

2.4.1 Performance research as a precursor to expressive MIR

Music performance research is the historical reason why dynamics cannot be treated as a small extension to note transcription. Earlier studies of expressive performance showed that timing and dynamics are shaped by phrase structure, metre, accent, local tension, and instrumental constraints [15, 16]. These findings matter for MIR because they make dynamics different from pitch or onset. A pitch label can often be judged against a relatively stable musical event, whereas a dynamics label depends on how a performer interprets that event in context.

This does not mean that expressive MIR should abandon discrete labels. It means that the label must be tied to the right musical coordinate. A note-level velocity is meaningful when the task is to control the rendering of individual note events. A score-level dynamic marking is meaningful when the task is to annotate a metrical or phrase-level change in loudness. A frame-level loudness contour is useful as acoustic evidence, but it is not itself a score annotation. Performance research therefore supplies more than motivation: it explains why representation choice is a validity condition. A system can be accurate with respect to one dynamics representation while being musically inappropriate for another.

2.4.2 From note transcription to expressive transcription

Classical AMT focused on recovering pitch content and note boundaries. Recent deep-learning systems have made this problem increasingly reliable for piano, especially on datasets such as MAESTRO and MAPS [4, 6, 67–71]. Table 2.5 should therefore be read not only as a benchmark summary, but as evidence of a shift in the bottleneck. For piano, note and note-plus-offset scores are now strong enough that the remaining expressive layer becomes visible: velocity-aware performance remains weaker, and the gap is not simply a matter of detecting more notes. It reflects the difficulty of estimating how strongly each known note should be rendered.

The multi-instrument case exposes a second bottleneck. On MusicNet and related corpora, the transcription problem is harder, and velocity-aware evaluation is often absent because reliable note-level velocity supervision is not available outside sensor-equipped piano datasets. This creates an asymmetry that runs through the thesis: piano provides the cleanest supervised setting for velocity recovery, while other instruments often provide audio and aligned notes but no trustworthy velocity labels. Expressive transcription is therefore not a detached field from AMT. It is the next layer of symbolic recovery once note content and timing are sufficiently constrained, and it requires different assumptions about supervision and evaluation.

Expressive AMT is therefore not a separate field detached from AMT. It is the next layer of symbolic recovery once note content and timing become reasonably stable. The thesis positions its contributions as extensions of AMT towards expressive and editable symbolic outputs rather than as alternatives to note transcription.

Table 2.5: Representative AMT results across MAPS, MAESTRO, and MusicNet. The numbers show that velocity-aware transcription remains substantially harder than note transcription.

Method	Year	Model	F1 frame	F1 note	F1 note&off	F1 note&off&velo
MAPS (piano)						
OaF [67]	2018	CRNN	78.30	82.29	50.22	35.39
Wu et al. [72]	2019	CNN	86.73	–	–	–
ADSR [73]	2019	CNN + HMM	77.16	81.38	56.08	–
hFT-Trans [69]	2023	Transformer	82.89	85.14	66.34	48.20
MAESTRO (piano)						
OaF† [67]	2018	CRNN	87.82	94.93	79.65	76.05
HPT† [4]	2021	CRNN	90.15	96.77	84.45	83.02
SemiCRFs† [70]	2021	CRNN + SemiCRF	91.11	96.51	88.72	87.75
Seq2Seq [68]	2021	Transformer	–	96.01	83.94	82.75
HPT-T [4]	2021	CRNN	90.09	96.77	83.20	81.90
HPPNet-sp [74]	2022	CRNN	93.15	97.18	83.80	82.24
hFT-Trans [69]	2023	Transformer	93.24	97.44	90.53	89.48
Transkun v2 [71]	2024	Trans. + SemiCRF	95.35	98.32	93.48	92.94
MusicNet (multi-instrument)						
Wu et al. [72]	2019	CNN	73.70	–	–	–
ReconVAT [75, 76]	2021	U-Net	48.0	29.0	11.0	–
MT3 [76]	2022	Transformer	68.0	50.0	33.0	–
PerceiverTF [77]	2023	Trans. + Perceiver	63.7	46.1	29.0	–
PF2N§ [78]	2025	PF2N + Perceiver	64.5	47.5	30.7	–

† Re-evaluation on MAESTRO v3 by SemiCRF paper.

2.4.3 Three task lines of dynamics-related MIR

The literature separates most clearly not by model family, but by what musical structure is assumed to be trustworthy. This is the organising principle used in the rest of the chapter.

Note-level MIDI velocity recovery. The first task line assumes that the note events are already known and asks only for the missing per-note velocity values. This setting is attractive because it preserves the composition exactly: pitch, onset, and duration remain fixed, while the model restores an editable control layer. Its limitation is equally clear. MIDI velocity is a renderer-facing parameter, not a direct measurement of perceived loudness or score dynamics. It is therefore well suited for expressive rendering and MIDI data curation, but it should not be treated as a universal representation of musical dynamics.

Beat-synchronous score-dynamics recovery. The second task line aims at dynamic markings such as *p*, *mf*, and *f*. Here the output is not a velocity value attached to every note, but a sparse symbolic description aligned with the metrical structure of the score. This changes both the temporal unit and the semantics of the target. A score marking is intentionally coarse and contextual; it describes a region of musical interpretation rather than the intensity of a

single event. Methods in this line must therefore model loudness evidence together with beats, downbeats, and change points, rather than treating dynamics as frame-wise audio classification alone.

Renderer-based and proxy-based recovery. The third task line appears when the target-domain velocity label is missing. In this case, supervised learning from ground-truth velocities is not available, even if audio and aligned notes exist. A renderer or differentiable proxy can then provide an indirect training signal: predicted velocities are judged by whether their rendered dynamics match the performance audio. This formulation is weaker than direct supervision, but it is not merely a workaround. It changes the claim being made. The recovered value is a usable dynamics coordinate for rendering or analysis under a specified synthesis condition, rather than a canonical physical measurement of the performer's gesture.

2.4.4 Cross-cutting validity: temporal units, alignment, and priors

Independently of the specific task line, three cross-cutting concerns determine whether a dynamics-related MIR system is even well-posed: which temporal unit indexes the prediction, whether the supervision is aligned, and what structural prior makes an underdetermined target tractable.

Temporal units. A dynamics-related event can be represented at several temporal resolutions, and each resolution carries a different modelling assumption. At the densest level, log-Mel and BSSL provide frame-level time–frequency evidence, while BSTL collapses the Bark-wise loudness distribution into a frame-level loudness trajectory. At the symbolic level, note onsets and note events index the local control values used for MIDI velocity. At the score-facing level, beats, downbeats, and phrase boundaries provide the coordinates on which dynamic markings, change points, and longer expressive spans become musically interpretable. Figure 2.3 illustrates these representations on the same performance excerpt, and Table 2.6 summarises their roles across the thesis. Choosing the wrong coordinate is not a presentation issue. It changes what the model can learn and how the prediction should be evaluated.

Alignment. Alignment is not a preprocessing detail. It determines what the supervision actually means. For note-level velocity, note onsets must be aligned with either the audio or the reference score; instrumented piano corpora supply this directly through synchronised audio–MIDI capture [5]. For score dynamics, the critical coordinate is the beat grid: dynamic symbols are rarely meaningful as an unconstrained frame-wise label stream but become interpretable once mapped to beats, bars, and phrase-level change points. Figure 2.3 makes this distinction visible: the same audio frames support several possible symbolic coordinates, but only some of those coordinates match the intended target. For label-scarce settings, alignment also determines whether a proxy supervision signal (e.g. rendered audio reconstruction) can be compared to the observed performance at all.

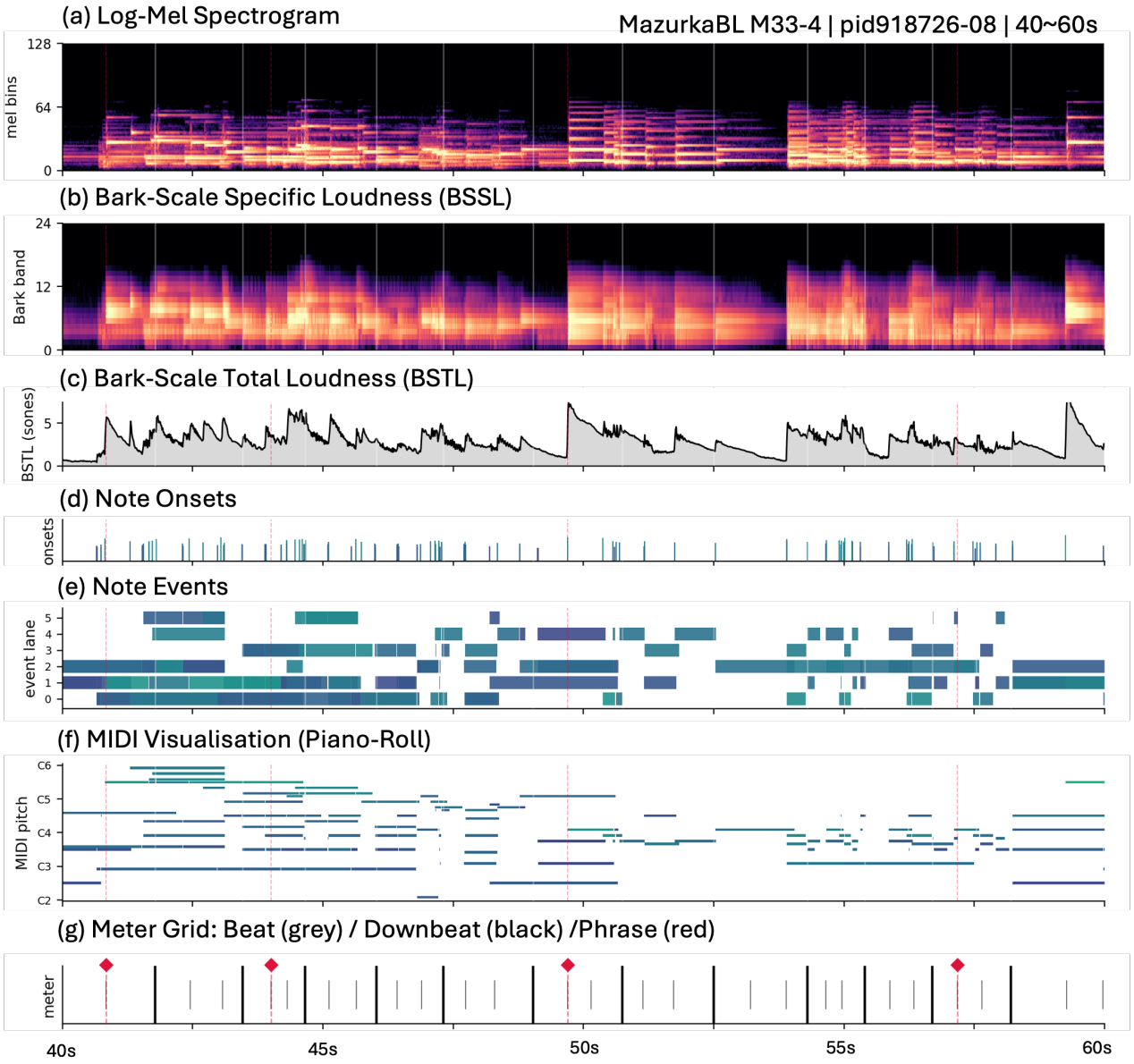


Figure 2.3: Aligned temporal representations for a 20 s excerpt from MazurkaBL M33-4 (pid918726-08, 40–60 s). Panels (a)–(c) show frame-level acoustic evidence: the log-Mel spectrogram, Bark-scale specific loudness (BSSL), and Bark-scale total loudness (BSTL). Panels (d)–(f) show note-level symbolic coordinates: note onsets, note-event spans, and a MIDI piano-roll visualisation. Panel (g) shows the metrical and phrase grid, with beats in grey, downbeats in black, and phrase boundaries in red. The figure illustrates how the same performance segment can be re-indexed from dense audio frames to note events and then to beat-, bar-, and phrase-level coordinates for dynamics recovery.

Table 2.6: Temporal units that recur across the thesis.

Unit	Where it appears	Why it matters
Frame	log-Mel, BSSL, BSTL, and acoustic features	Captures fine temporal evidence but is too dense for score-level dynamics
Note onset	MIDI velocity and AMT outputs	Natural coordinate for local attack strength and note-level intensity control
Note event	symbolic completion and score-informed correction	Supports note-wise masking, correction, and editable symbolic output
Beat	score dynamics and metrical tracking	Main coordinate for human-readable dynamic labels and beat-synchronous estimation
Downbeat	score dynamics and bar-level structure	Anchors metrical organisation and change-point interpretation
Phrase / segment	expressive interpretation and dynamic-change context	Explains longer-range shaping that cannot be reduced to isolated notes

Structural priors. Each task line uses a different prior to make an otherwise underdetermined dynamics target tractable. Score-informed velocity correction uses an aligned score as a structural prior over note identity (Chapter 4). Beat-synchronous score dynamics uses the metrical grid as a coordinate prior on which the label sequence lives (Chapter 5). Cross-instrument velocity recovery uses a differentiable renderer as a generative prior linking predicted control values to a measurable property of the recording (Chapter 6). The mechanisms differ, but they share a function: they match the output layer to an evidence source and inject structure where direct supervision is weak or absent.

2.5 Note-Level MIDI Velocity Recovery

This section reviews the first of the three task lines: recovering note-level MIDI velocity from symbolic notes, audio, or both. It tracks the literature from rule-based humanisation through symbolic completion, audio-driven AMT, and score-informed correction, and ends at the boundary where reliable velocity supervision disappears outside piano. Chapters 3 and 4 contribute to this line; the label-scarce extension is deferred to Section 2.7.

2.5.1 Expressive rendering and humanisation

Expressive performance rendering aims to generate expressive control signals from symbolic input. Many systems predict timing, dynamics, and sometimes articulation jointly [79, 80]. These models are important because they show that expressive performance can be learned from symbolic structure. At the same time, they reveal a practical limitation for the present thesis. Joint expressive rendering may alter more than the user wants. In production or corpus-upgrading settings, the desired operation is often narrower: preserve the notes and timing, and recover only the missing dynamics layer. This narrower, controllable objective is the entry point

for Chapter 3.

2.5.2 Symbolic velocity completion and prediction

Symbolic velocity completion assumes that the note sequence is already known and that the missing variable is note-level velocity. Early work used rule-based or statistical approaches [66]. More recent work formulates the task with deep learning, including convolutional autoencoders and sequence-to-sequence models with attention [81, 82]. Diffusion-based note-intensity estimation has also been explored in related score-informed settings [83].

A central challenge in this literature is the sparsity of symbolic piano-roll data. Dense frame grids contain large silent regions, and a model that treats all cells equally can waste much of its capacity. Two architectural patterns recur in response. The first rasterises MIDI as piano-roll-like matrices indexed by pitch and time [25], then processes them with U-Net-style encoders that preserve fine local detail through skip connections while aggregating larger context through the encoder–decoder path. The second couples this locality with windowed or linear attention to extend the receptive field without the full cost of global attention [84, 85]. Together they motivate the image-like, sparse-2D formulation adopted in Chapter 3.

2.5.3 Audio-driven velocity estimation within AMT

Velocity recovery from audio has become increasingly feasible for piano because large aligned datasets exist with sensor-recorded note velocities [5]. Representative velocity-enabled AMT systems include OaF, HPT, hFT-Trans, and event-based models [4, 67–71]. Related work also studies robustness, semi-supervised learning, and parameter-efficient piano transcription [74, 86–89].

Despite strong progress, note-level velocity remains more fragile than note pitch and onset estimation. The trend is already visible in Table 2.5: as the evaluation tightens from frame F1 to note+offset+velocity F1, the gap between leading systems widens. Domain shift in piano timbre, microphone placement, room acoustics, and recording chain degrades the mapping from acoustic evidence to velocity more severely than the mapping from acoustic evidence to pitch. This fragility is one of the main motivations for the score-informed correction strategy of Chapter 4.

2.5.4 Score-informed MIDI velocity estimation and correction

Score-informed velocity estimation assumes that an aligned score is available together with performance audio. Historically, this line of work developed partly because early AMT systems focused on notes and offered limited support for reliable velocity estimation [90]. Early systems used hand-designed measurements or parametric estimation strategies [7, 16, 60, 91]. Later work introduced restricted Boltzmann machines, score-informed non-negative matrix factorisation, diffusion-based note-level estimation, and FiLM-conditioned deep models [23, 61, 83, 92–95].

A key pattern in this literature is *feature fusion*. Score information is injected into the acoustic model during feature extraction. This can improve accuracy, but it also ties the score-informed method tightly to a particular backbone. The present thesis instead emphasises *late correction*. A correction module can attach to an existing AMT system and refine its velocity estimate while leaving the backbone largely unchanged. The natural interface for this stance is a note-token representation rather than a dense frame grid: when note events are already known, focusing modelling capacity on the events that actually require correction is more efficient than re-running a full transcription model end to end. This is the design stance developed in Chapter 4, where a score-informed Transformer over note tokens corrects a coarse velocity estimate from a frozen AMT front-end.

2.5.5 Piano bias and the beyond-piano boundary

Reliable velocity supervision is still strongly piano-centric. Instrumented pianos make note-level velocity available at scale, but comparable datasets for other instruments are scarce. This creates both a training bias and an evaluation bias. Transfer beyond piano must therefore confront two bottlenecks: missing labels and instrument-dependent mappings between performer action, timbre, and loudness.

The important point is that beyond-piano failure is not only ordinary domain shift. It is a measurement problem. Outside piano, even defining what a "ground-truth" note-level control signal should be is non-trivial: there is no sensor analogue of the Disklavier hammer trace for guitar, violin, or wind instruments. This distinction is essential for the honest positioning of Chapter 6, and it motivates the analysis-by-synthesis route reviewed in Section 2.7.

2.6 Beat-Synchronous Score-Dynamics Recovery

This section reviews the second task line: recovering beat-anchored dynamic markings such as *p*, *f*, and crescendo from performance audio, together with the metrical structure on which those markings live. The output is a sparse, score-facing sequence rather than dense note-level control, and the target representation is therefore quite different from the velocity recovery reviewed in Section 2.5. Chapter 5 contributes to this task line.

2.6.1 From performed loudness to score dynamics

Computational modelling of dynamics has a long history in expressive performance research [15]. Rather than focusing only on note-level control, this line of work studies how performed loudness relates to symbolic dynamic markings and phrase structure. Kosta and collaborators showed both the usefulness and the ambiguity of this relationship [17, 30, 96]. Other work has further analysed dynamic principles in piano performance [58].

From a machine-learning perspective, score-dynamics recovery can be framed as classification or structured sequence labelling. The input is audio. The output is a sequence of dynamic

symbols or dynamic regimes. The choice of audio representation is therefore not neutral. In the MazurkaBL line of work, the performed-loudness side of this mapping is extracted through sone-based psychoacoustic processing rather than raw amplitude, precisely because the target concerns perceived intensity rather than unweighted energy [97]. More generally, perceptual front-ends such as Bark-scale specific loudness preserve both a scalar loudness trajectory and the Bark-wise distribution of loudness across the spectrum, which gives a better inductive bias for loudness-oriented tasks than log-Mel features [24, 26]. Figure 2.3(a)–(c) shows this contrast on the same excerpt. The practical consequence is also clear: a Nyquist-constrained Bark front-end is substantially lower-dimensional than a 128-band log-Mel representation, which enables longer audio segments and the phrase-level context that score dynamics actually requires. The challenge is that the labels themselves are sparse, relative, and musically contextual rather than physically absolute. This is one reason why a beat-synchronous formulation is more natural than direct frame-wise regression of continuous loudness.

2.6.2 Change points and dynamic segmentation

Dynamic markings rarely behave as independent frame labels. They form local regimes that persist over spans of music, and musical meaning often lies in the transitions between those regimes. Change-point detection therefore plays an important role. It supports segmentation, interpretation, and the recovery of phrase-level dynamic structure [96, 97]. The phrase markers in Figure 2.3(g) show the kind of longer span that cannot be recovered from isolated note or frame evidence alone. In Chapter 5, change points are estimated jointly with dynamic levels so that a score-facing output includes both local states and their transitions.

2.6.3 Beat and downbeat tracking as structural coordinates

Beat and downbeat tracking has a mature literature. Early deep-learning approaches treated beats and downbeats as jointly modelled targets using recurrent neural networks combined with probabilistic post-processing such as dynamic Bayesian networks [98], an RNN–DBN paradigm that dominated the field for several years. Subsequent work refined this combination with stronger convolutional encoders and tighter post-processing [99, 100], while more recent Transformer-based systems further reduce reliance on heavy post-processing [101]. For this thesis, beat and downbeat tracking is not only a useful auxiliary task. It is the coordinate system that makes score dynamics interpretable, comparable, and writable back to a score, as reflected in the bottom grid of Figure 2.3.

2.6.4 Multi-scale and multi-task formulations

Dynamics estimation and metrical estimation are closely related but not identical, and they live at different temporal scales. Beat and downbeat tracking needs fine temporal resolution to localise peaks; phrase-level dynamics needs much longer context to read shape, contrast,

and change-point structure. Multi-scale architectures address this scale mismatch by providing parallel or hierarchical pathways with different effective receptive fields [102].

The remaining issue is that even a shared multi-scale encoder can suffer from negative transfer when several tasks compete for the same representation. Multi-task learning is attractive when the outputs share information, but it is dangerous when the tasks prefer different acoustic cues. Mixture-of-experts methods provide a practical compromise: they combine shared representation learning with task-specific routing so that each output draws differently from a shared expert pool [27]. The MMoE-style decoder used in Chapter 5 follows precisely this logic, and the combination of a perceptually grounded multi-scale encoder with task-aware routing is the architectural shape that this task line ultimately motivates.

2.7 Audio Rendering and Differentiable Synthesis

This section reviews the third task line: mapping symbolic control values back to audio through a renderer, and using that mapping as supervision when direct labels are missing. The review proceeds in three steps. Section 2.7.1 surveys traditional renderers and the role velocity plays inside them. Section 2.7.2 reviews differentiable synthesis and the design space it opens. Section 2.7.3 discusses the analysis-by-synthesis paradigm and its application to label-scarce velocity recovery in Chapter 6.

2.7.1 Traditional renderers and the velocity control point

Traditional MIDI-to-audio rendering covers three broad families. *Sample-based* synthesisers such as SoundFont-driven engines (e.g. FluidSynth [103]) produce sound by selecting prerecorded samples and post-processing them with envelopes and filters; commercial samplers and free SoundFonts such as Salamander Grand Piano [104] share this design. *Physical models* such as Pianoteq [105] replace samples with mechanical simulations of the instrument. *Wavetable, FM, and additive engines* synthesise the signal from compact spectral or oscillator parameters. The three families differ in the underlying model, but they all expose MIDI velocity as a control input that drives intensity, timbral brightness, attack character, and sometimes sample-layer selection.

The implication for dynamics recovery is twofold. First, velocity is not a renderer-independent loudness unit: the same MIDI velocity can produce substantially different acoustic outputs depending on which sample layers, envelopes, or physical parameters the renderer uses, as already discussed in Section 2.3. Figures 2.4 and 2.5 make this concrete by showing how the velocity-to-loudness response curves of representative SoundFonts differ within the piano family and across instruments. Second, traditional renderers are not differentiable: they expose discrete sample selection, table lookups, and non-smooth envelopes, so gradients cannot flow from rendered audio back to the velocity input. Any system that wants to use rendering as a learning signal must therefore replace, approximate, or wrap the renderer with a differentiable proxy.

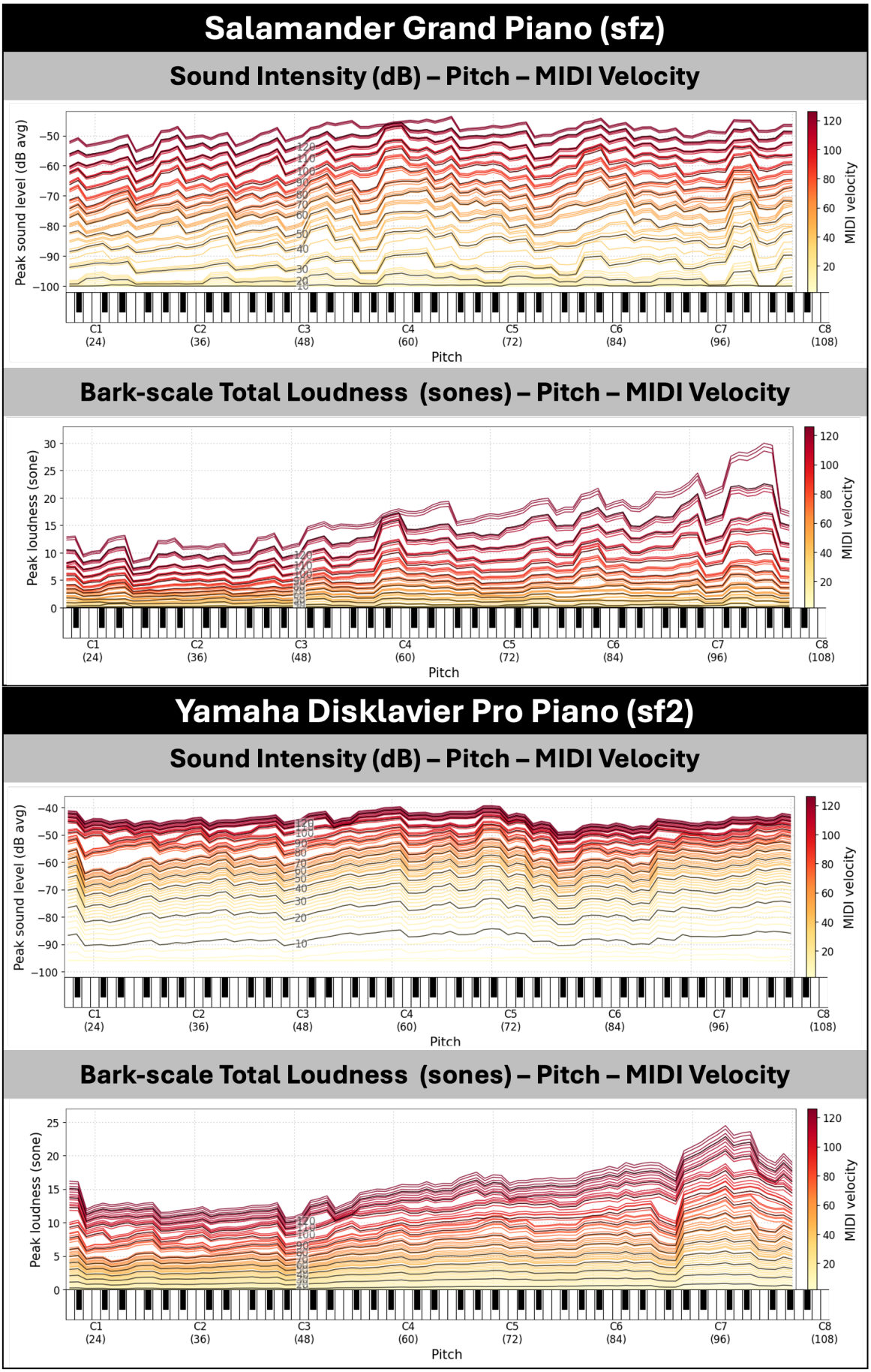


Figure 2.4: Velocity-to-loudness response of two piano renderers (Salamander Grand Piano and a Yamaha Disklavier Pro SoundFont). Both share the same input control range, but the realised loudness curves differ in slope and shape, which illustrates the renderer dependence of MIDI velocity.

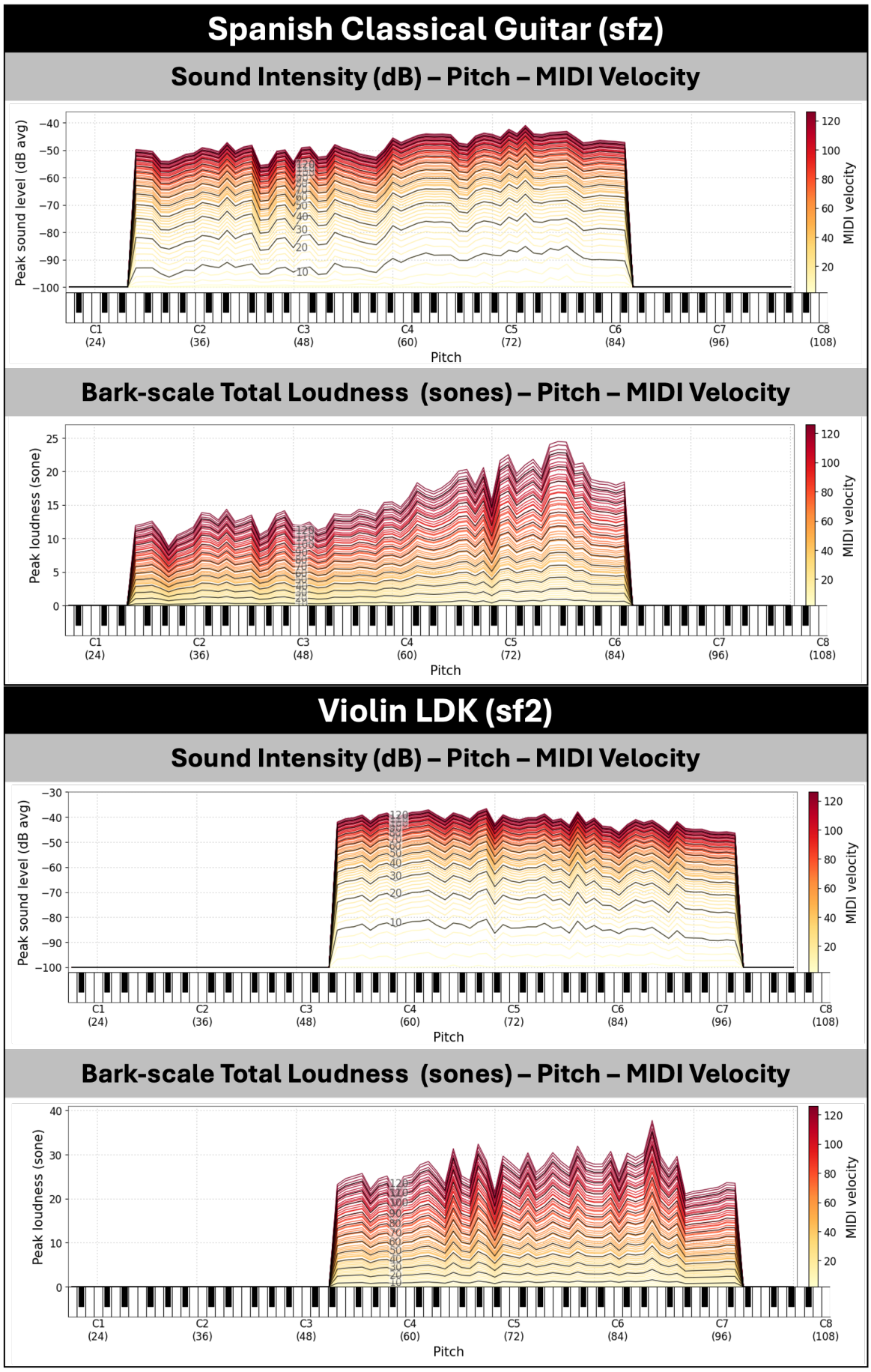


Figure 2.5: Velocity-to-loudness response for non-piano SoundFonts (Spanish Classical Guitar and Violin LDK). Compared with Figure 2.4, the curves show different dynamic ranges and saturation behaviour, motivating the need for a renderer-aware supervision interface when working beyond piano.

2.7.2 Differentiable synthesis

Differentiable synthesis was introduced to make audio rendering compatible with gradient-based learning. The DDSP framework expresses a renderer as a small differentiable graph (harmonic-plus-noise oscillators, time-varying filters) whose control signals are predicted by a neural network [106, 107]. Loss functions are then applied in the signal or spectral domain, and gradients flow back through the renderer to the network. The same idea has been adapted to specific instruments and rendering goals: MIDI-DDSP couples DDSP with explicit note-level control for expressive performance synthesis [108], DDSP-Piano specialises the framework to piano timbre with mechanically motivated components [109], and follow-up work extends it to acoustic guitar and other plucked-string instruments [110, 111]. Beyond DDSP, differentiable approximations to traditional samplers and effects processors have been studied as well [112–114], and recent work has explored normalising-flow and transformer-based generative renderers in similar settings [115, 116].

For dynamics recovery, the appeal is direct. A differentiable renderer turns the velocity-to-audio mapping into a learnable layer. A model can predict velocity, render the resulting audio, and back-propagate a reconstruction error. This makes velocity-relevant supervision possible even when direct note-level labels are absent. The cost is also direct: differentiable renderers are not free reproductions of every commercial synthesiser. They tend to specialise to one instrument family, and their fidelity to real recordings depends on how much of the timbral, recording-chain, and room behaviour the renderer captures. The choice of differentiable renderer therefore conditions what the downstream model can plausibly learn.

2.7.3 Analysis-by-synthesis and label-scarce velocity recovery

The combination of differentiable rendering and prediction networks defines an *analysis-by-synthesis* setting. A network proposes control values for an observed recording; the renderer turns those values back into audio (or into an intermediate audio-derived quantity); a loss compares the rendered output with the observed signal. This paradigm becomes the central tool when true note-level velocity labels are unavailable, because the renderer takes the place of a label by linking predicted control to a measurable property of the recording.

Two design axes matter for this thesis. The first is the comparison space. *Signal-domain* losses (waveform or spectral) can be applied directly to the rendered output, but they conflate velocity with timbre, recording chain, and room response, so the gradient signal is dominated by factors unrelated to velocity. *Parameter-domain* or perceptual proxy losses instead extract a small set of velocity-relevant acoustic descriptors (e.g. note-wise loudness, attack envelope) from both the rendered and the observed audio and compare them in that lower-dimensional space. The latter is closer in spirit to the parameter-extraction proxies used in spectral-modelling research and trades waveform fidelity for stronger alignment with the velocity target. The second axis is the renderer's domain coverage. A differentiable renderer adapted to piano need not transfer to guitar or violin without further work [109, 110], which is one of the practical

constraints of cross-instrument analysis-by-synthesis.

For expressive AMT, this strategy is attractive but delicate. It makes label-scarce learning possible, yet it also ties the learning signal to the adequacy of the renderer: a proxy objective can reward compatibility with the synthesiser more than compatibility with real instrumental control, and reconstruction error is therefore not the same thing as accurate velocity recovery. Chapter 6 adopts the analysis-by-synthesis route explicitly under these constraints. It contrasts a waveform-domain DDSP-style supervision interface with a low-dimensional parameter-space proxy built from a frozen SoundFont teacher, and treats proxy supervision as a method family with explicit limits rather than as a universal substitute for true note-level labels.

2.8 Datasets, Evaluation, and Reproducibility Constraints

The empirical claims of this thesis are conditioned on what each available dataset can support. Table 2.7 summarises the representative corpora relevant to expressive label recovery, listing their instrument coverage, scale, and which expressive modalities (MIDI velocity, score, and score dynamics) are actually available. The discussion that follows turns these modality differences into chapter-specific constraints.

2.8.1 Dataset modalities and what each chapter can learn

The thesis relies on datasets with very different modality combinations. Audio–MIDI datasets such as MAESTRO support supervised note-level velocity learning. Cross-dataset piano corpora such as SMD and MAPS stress generalisation under changed recording conditions. Score-aligned corpora such as MazurkaBL support score-facing dynamics research because they provide beat-level structure and expressive annotations. Other datasets are valuable but incomplete for the present thesis because they may lack reliable note-level velocity, verified score dynamics, or stable redistribution conditions.

These modality differences are not an implementation detail. They determine which claim can be made at all. Chapter 3 needs reliable symbolic velocity targets. Chapter 4 additionally needs an aligned score. Chapter 5 needs score-facing dynamics with metrical structure. Chapter 6 needs candidate non-piano corpora, but cannot assume direct velocity supervision of MAESTRO quality.

2.8.2 Evaluation for MIDI velocity

Velocity should be evaluated at note level rather than by naive frame-wise error. Otherwise, long sustained notes dominate the metric. Standard statistics include mean absolute error and related dispersion measures [23, 81]. However, numerical accuracy alone is not sufficient. A model can achieve a decent average error while producing over-smoothed or mechanically centred dynamics. Distribution-sensitive measures, visual inspection, and listening tests therefore play

Table 2.7: Representative datasets relevant to expressive label recovery in this thesis. Each dataset is summarised by its instrument family, scale, and the expressive modalities (MIDI, velocity, score, score dynamics) it provides.

Dataset	Instr.	Statistic			Modality			
		Hours	Performance	Compositions	MIDI	Velocity	Score	Dynamics
ATEPP [117]	piano	1007	11742	1580	✓	✓	✓ 43%	▲ 43%
▲ Audio is from YouTube (availability may vary); score dynamics are not verified.								
ASAP [13]	piano	92	1068	222	✓	✓	✓	▲
▲ Score dynamics (and other expressive markings) are not verified.								
MAESTRO v3 [5]	piano	172	1276	864	✓	✓	✗	✗
✓ High-quality piano dataset, recorded on Yamaha Disklavier pianos.								
SMD v2 [118]	piano	4.7	50	50	✓	✓	✗	✗
✓ Often used for cross-dataset evaluation; recorded on Yamaha Disklavier pianos.								
MAPS [119]	piano†	17.9	270	208	✓	✓	✗	✗
✓Count only ENSTDkAm and ENSTDkCl subsets (solo piano, Yamaha Disklavier).								
MazurkaBL [96]	piano	110	2000	44	✗	✗	✓	✓
✓ Dynamics and Expressive labels are explicitly verified. All Chopin compositions.								
Vienna4x22 [120]	piano	2.3	88	4	✓	✓	✓	✓
✓Contains 4 compositions from Chopin, Schubert and Mozart, each 22 performances.								
Batik-plays-Mozart [121]	piano	3.7	36	36	✓	✓	✓	✓
✓Contains 12 Mozart sonatas split to 36 movement-level score–performance pairs.								
GuitarSet [122]	guitar	3.0	360	30	✓	✗	✗	✗
✓ Good quality electric guitar dataset.								
GOAT [21]	guitar	5.9	172	n/a.	✓	✗	✓	✗
✓ Usage may still be constrained by copyright or redistribution terms.								
François Leduc [123]	guitar	4.0	78	n/a.	✓	✗	▲	✗
✓ By request only; released files include audio–MIDI pairs, but not the original scores.								
GAPS [124]	guitar	14.1	300	300	✓	✗	✓	✗
▲ Audio is collected from YouTube URLs (availability may vary).								
MusicNet [125]	ensemble	34.1	330	121	✓	▲	✗	✗
▲ Classical, multi-instrument; velocity is score-derived and not performance control.								
MID-FiLD [18]	ensemble	n/a.	4477	4477	✓	✓	✗	✗
▲ No human audio; MIDI is made by software rather than human performance.								

an important complementary role. This is especially evident in Chapter 3, where perceptual differences are not always fully captured by raw error.

Cross-dataset evaluation is also essential. Piano type, velocity calibration, microphone placement, and room acoustics all affect the mapping from performance action to recorded signal. A model that performs well only on the training dataset does not yet solve expressive AMT in a practical sense.

2.8.3 Evaluation for score dynamics

Score dynamics is usually evaluated as classification on a musically meaningful grid. F1 score is common because the class distribution is imbalanced and because some symbols occur much less frequently than others [97]. In the present thesis, beat-synchronous evaluation is especially important because it avoids arbitrary frame-level tolerance choices and aligns the prediction target with how the annotations are actually used in music notation.

Evaluation remains inherently limited by label ambiguity. Different editors or annotators may disagree on dynamic markings. Different editions of the same piece may contain different expressive symbols. This does not invalidate the task, but it does mean that score-dynamics performance must be interpreted with awareness of the annotation convention behind the dataset.

2.8.4 Evaluation under proxy supervision

When direct note-level velocity labels are absent, evaluation becomes weaker and must be stated more carefully. Audio reconstruction error after differentiable synthesis is informative, but it is not equivalent to accurate velocity recovery. A proxy-supervised study should therefore separate three questions. Does the synthesised audio become more plausible? Do the predicted control values display musically reasonable distributions and relative trends? Does any anchor subset with stronger labels show positive transfer?

This evaluation logic is directly relevant to Chapter 6. The contribution of that chapter is exploratory because the supervision and evaluation are weaker than in the piano chapters. Making that weakness explicit strengthens the thesis. It does not weaken it.

2.8.5 Reproducibility, alignment quality, and licensing

Expressive AMT research is strongly constrained by data quality and licensing. Many scores are copyrighted. Many recordings cannot be redistributed freely. Some datasets rely on external URLs or derived alignments whose long-term availability is uncertain. Alignment quality is another major factor. A method may fail not because the dynamics model is poor, but because the score or beat alignment is noisy.

For this reason, reproducible expressive AMT research should document preprocessing, alignment assumptions, train/validation/test protocols, and all modality constraints clearly.

The later thesis chapters follow this principle, and the final conclusion of the dissertation treats these constraints as design considerations rather than afterthoughts.

2.9 Summary of Gaps and Thesis Requirements

The review above leaves six requirements for a thesis on dynamics recovery.

- **R1 Preserve note content and timing while recovering dynamics.** Many expressive systems modify several performance dimensions jointly. The present thesis instead treats dynamics as a refinement or annotation layer. This requirement is most visible in Chapters 3, 4, and 6.
- **R2 Support heterogeneous data-availability settings.** Real corpora differ widely in what is available: symbolic MIDI only, audio plus aligned score, audio only, or audio with only proxy supervision. This requirement motivates the overall design of Chapters 3–6.
- **R3 Respect representation differences explicitly.** Loudness, MIDI velocity, and score dynamics are linked but not identical. The target representation must therefore be chosen deliberately and defended in each chapter. This requirement spans the whole thesis.
- **R4 Use musically meaningful coordinates for score-level outputs.** Score dynamics should not be treated as an arbitrary frame-wise label stream. It should be expressed on a beat-synchronous interface, supported by downbeat and phrase-level context, that supports interpretation, evaluation, and symbolic writing. This requirement motivates Chapter 5.
- **R5 Evaluate with both numerical and musical evidence.** Objective metrics are necessary but not always sufficient. Listening tests, qualitative inspection, distributional behaviour, and cross-dataset evaluation remain important. This requirement is visible in Chapters 3, 5, and 6.
- **R6 Treat beyond-piano transfer as a supervision problem rather than ordinary domain shift.** Once true note-level velocity labels disappear, the central issue is no longer only acoustic domain mismatch, but the absence of a directly measurable target. This requirement motivates Chapter 6, where proxy supervision is formulated as a renderer-conditioned route to label-scarce velocity recovery.

These requirements define the dissertation stance. Dynamics recovery is treated as controlled label restoration under structural preservation, not as unconstrained expressive generation. The four technical chapters instantiate this stance under progressively weaker or different evidence conditions: symbolic-only completion, score-informed audio refinement, audio-only score-dynamics recovery, and renderer-conditioned recovery when target-domain velocity labels are absent.

Chapter 3

Symbolic MIDI Velocity Completion as Image Colourisation

3.1 Chapter Overview and Position in the Thesis

This chapter addresses Scenario S1, the most controlled setting in the thesis map. Here the symbolic notes and their timing are already known, but the note-level velocity is missing, flattened, or musically uninformative. The chapter therefore asks a focused question: can useful note-level dynamics be recovered from symbolic structure alone, without changing the original composition?

This setting matters because it isolates the dynamics problem from the broader transcription problem. It is common in digital audio workstation workflows, step-entered MIDI, quantised MIDI, and corpus curation settings where note content already exists but expression does not. It also provides the cleanest test of the thesis claim that dynamics can be treated as a separate and editable target rather than as a side effect of full expressive performance generation.

This chapter makes three contributions. First, it shows that symbolic velocity completion can be formulated naturally as an image-colourisation problem on sparse piano-roll representations. Second, it develops a U-Net-based model with window attention and a sparsity-aware loss tailored to this representation. Third, it provides both objective and perceptual evidence that recovering only MIDI velocity can improve musical expressiveness in a controlled setting.

3.2 Task-Specific Related Work

MIDI velocity completion sits at the intersection of expressive rendering and symbolic refinement. Broad expressive-rendering systems often predict timing, articulation, and dynamics jointly [79, 80]. These systems are important, but they are not ideal when the practical goal is to preserve a user's note content and timing while improving only intensity. A velocity-only formulation is narrower, but it is also more controllable.

Recent symbolic velocity-prediction methods have typically treated the task as sequential prediction. Representative examples include convolutional autoencoders and sequence-to-

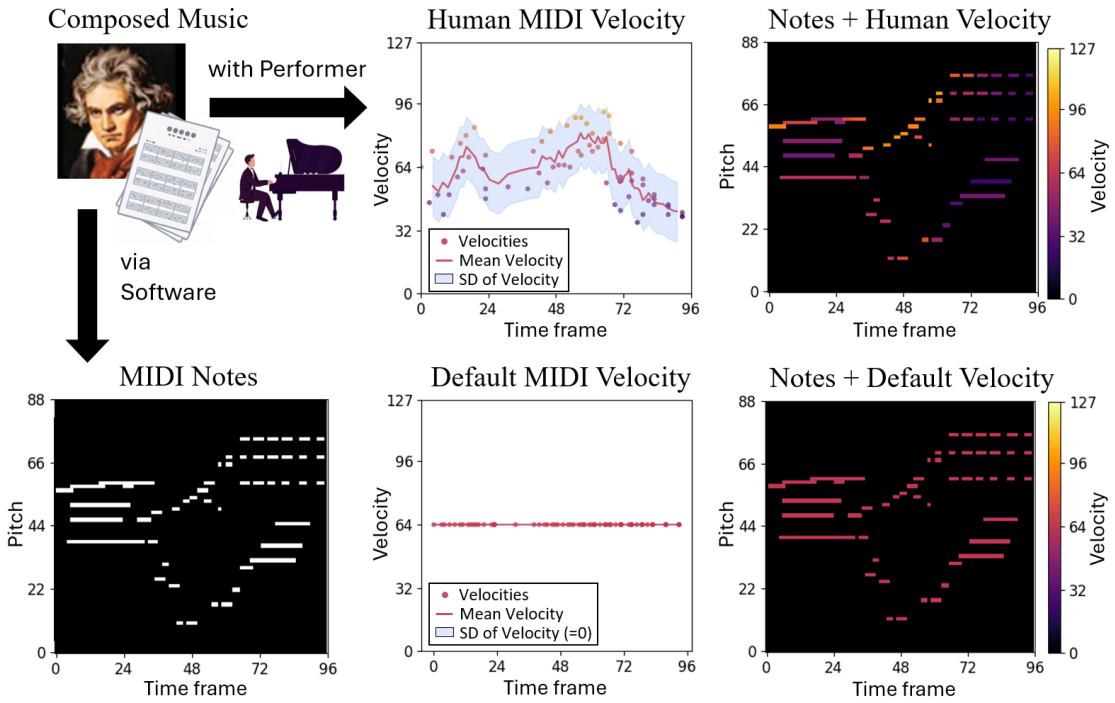


Figure 3.1: Comparison between MIDI notes with human-performed velocity and music-software default velocity (64 when unspecified). The standard deviation of velocity (SD_{velo}) represents the dispersion of velocities around their mean across pitches.

sequence models with attention [81, 82]. These models establish that note-level velocity can be learned from symbolic context. However, piano-roll data is sparse and polyphonic. The relevant structure is not only sequential. It is also spatial. Simultaneous notes create meaningful vertical patterns, and short local pitch–time neighborhoods often carry clear expressive cues.

This motivates the image-based view adopted in the present chapter. By treating symbolic MIDI as a sparse image and velocity as a colourisation target, the model can exploit local polyphonic structure more directly. U-Net and attention mechanisms are already successful in related music and audio tasks, but the present setting is distinct because it is purely symbolic and does not rely on acoustic input [23, 25]. This makes the chapter a natural starting point for the broader thesis.

3.3 Problem Setting

MIDI (Musical Instrument Digital Interface) is the dominant symbolic format in modern music production. A MIDI file that resembles sheet music can still sound mechanical when its note velocities are undefined or flattened to a default constant. In contrast, MIDI captured from human performance preserves the performer's control of intensity and therefore carries a substantial portion of the perceived musical expression. As discussed in the thesis introduction, dynamics is the expressive dimension targeted first in this thesis because it is both perceptually salient and directly editable. This chapter studies the most constrained setting in that map: recovering note-level MIDI velocity when the symbolic notes and their timing are already known.

This setting is practically important for two reasons. First, a large amount of MIDI used in digital audio workstation workflows is created by step input, quantisation, or low-cost controllers, leaving velocity flat. Second, many expressive rendering systems modify several performance dimensions jointly, including timing and sometimes note content, which is undesirable when the goal is to preserve the user's composition. By isolating velocity, the task becomes a precise and controllable refinement problem: the notes, onsets, and offsets remain fixed, and only note intensity is completed.

This chapter instead exploits the spatial structure of polyphonic symbolic music by treating MIDI as an image-colourisation problem. MIDI without velocity is represented as a binary pianoroll, while the target is a velocity-valued pianoroll. This formulation is especially natural for instruments such as piano, where multiple simultaneous notes create rich vertical patterns. Current dataset availability restricts the experiments here to piano data, but the formulation itself is not tied to piano alone.

3.4 Representation as Image-like MIDI Matrices

3.4.1 Matrix Representation

To process MIDI as images, we convert each performance into three matrices of size $T \times P$, where T is the number of time frames and $P = 88$ is the number of piano pitches. The three matrices are a binary onset roll O, marking note starts; a binary frame roll F, indicating note activation over time; and a velocity roll V, acting as the target colour intensity. In the implemented system, F is the network input, V is the prediction target, and O is used for onset masking in the loss. For the velocity roll, integer values in $[0, 127]$ are normalised to $[0, 1)$ to match the model output activation range. During post-processing, the model output is denormalised by scaling and rounding back to integer MIDI velocity. As illustrated in Figure 3.2, these matrices are highly sparse.

3.4.2 MIDI Segmentation

The segment duration is a key design variable in this formulation. Because a MIDI performance often exceeds three minutes, we split each piece into shorter segments to keep the computation tractable. The number of time frames T determines the temporal granularity of the matrix representation, so the timestep resolution is

$$\text{Resolution} = \frac{\text{Segment Duration}}{T}. \tag{3.1}$$

Unlike tasks that require fine temporal resolution to detect unknown note events, the present task already assumes that note timing is given. This makes a more computationally efficient timestep resolution feasible. With a fixed input size of $T = 96$, we explored segment durations of 5 s, 10 s, 15 s, and 20 s in hyperparameter search and found that 10 s yielded the best results. A

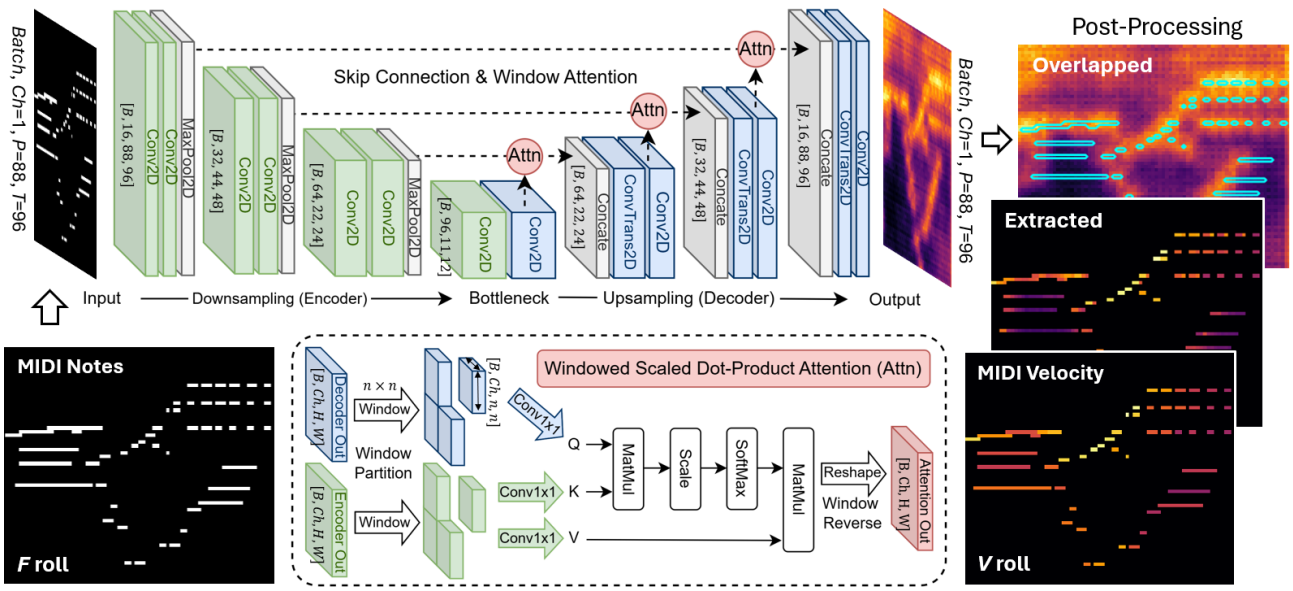


Figure 3.2: Proposed U-Net architecture. The model input (F roll) comprises 88 pitch bins and 96 time frames. Attn block denotes windowed scaled dot-product attention. The final velocity roll (V roll) is generated during post-processing by extracting velocity at note positions and assigning each note the velocity at its onset.

plausible explanation is that a 10-second window often captures a musically coherent phrase, for example roughly four measures at 120 BPM, providing enough context without overextending the receptive field.

3.5 Model Architecture and Loss

3.5.1 U-Net with Window Attention

The proposed architecture is shown in Figure 3.2. The U-Net extracts higher-level features through downsampling, while skip connections preserve local details together with broader structural patterns. All convolution blocks use 3×3 kernels, stride 1, and padding 1, followed by sigmoid activation and batch normalisation. Downsampling is implemented with a standard 2×2 MaxPool2D layer. For upsampling, we use transpose convolutions with 4×4 kernels and stride 2, following the image-colourisation variant in [126].

To address MIDI sparsity, we insert windowed scaled dot-product attention (window attention) [84] into the U-Net. Traditional self-attention [127] operates on the full feature map $X \in \mathbb{R}^{H \times W}$, where H and W denote height and width. Window attention instead partitions X into non-overlapping $n \times n$ windows and computes attention within each window. This reduces computational complexity while still improving feature aggregation [85].

The window size (n) balances local detail against broader dependency modelling. We explored $n \in \{1, 2, 4, 8\}$ and found that 2×2 window attention performed best in hyperparameter search. This suggests that attention is useful in this sparse setting, but overly large windows can compress information too aggressively and reduce effectiveness.

3.5.2 Sparsity-aware Loss Function

The proposed loss combines binary cross-entropy (BCE) with cosine similarity (CosSim), following [82], with $\alpha = 0.2$:

$$\mathcal{L}_{\text{Combine}} = (1-\alpha)\,\mathcal{L}_{\text{BCE}} + \alpha\,(1 - \text{CosSim})\,. \tag{3.2}$$

Here BCE optimises element-wise prediction error, while CosSim is computed pitch-wise and then averaged to capture the trend of velocity variation over time:

$$\text{CosSim} = \frac{1}{P}\sum_{p=1}^{P} \frac{\sum_{t=1}^{T} y_{t,p}\,\hat{y}_{t,p}}{\sqrt{\sum_{t=1}^{T} y_{t,p}^2}\,\sqrt{\sum_{t=1}^{T} \hat{y}_{t,p}^2}}, \tag{3.3}$$

$$\mathcal{L}_{\text{BCE}} = \frac{1}{TP}\sum_{t=1}^{T}\sum_{p=1}^{P} l_{\text{bce}}(y_{t,p}, \hat{y}_{t,p})\,. \tag{3.4}$$

In these expressions, $y_{t,p}$ and $\hat{y}_{t,p}$ denote the target and predicted velocity, respectively, and t and p index time and pitch.

To better match the structure of the task, we introduce two modifications. First, a masking operation $< m >$ based on the onset roll O ignores silent time steps and counts each note once at its onset. Second, a weighting operation $< w >$ emphasises the velocity-range boundaries, which are harder to predict accurately [23, 61, 128]. Motivated by the approximately Gaussian velocity distribution in Figure 3.3, we use a V-shaped weighting centred on 64 (normalised to 0.5), with empirical factor 3:

$$\mathcal{L}_{\text{BCE}}^{<m,w>} = \frac{1}{TP}\sum_{t=1}^{T}\sum_{p=1}^{P} O_{t,p}\left(1 + 3|V_{t,p} - 0.5|\right) l_{\text{bce}}(y_{t,p}, \hat{y}_{t,p})\,. \tag{3.5}$$

The final objective replaces $\mathcal{L}_{\text{BCE}}$ in Eq. (3.2) with $\mathcal{L}_{\text{BCE}}^{<m,w>}$. This loss focuses learning on note onsets while also penalising the underestimation of more extreme dynamic values.

3.6 Experimental Setup

3.6.1 Datasets

For training, we use the MAESTRO v3.0.0 dataset [5], recorded by skilled pianists on Yamaha Disklavier pianos during the International Piano-e-Competition. The default train/validation/test split is used. With 1,276 performances totaling more than 200 hours, MAESTRO provides a strong foundation for modelling MIDI velocity.

For evaluation, we use the Saarland Music Data (SMD) dataset [118], which contains 50 further performances recorded on a Yamaha Disklavier. We selected SMD for cross-dataset evaluation instead of the Piano-e-Competition dataset used in [81, 82], because the latter has substantial performance overlap with MAESTRO. The suitability of SMD is supported by

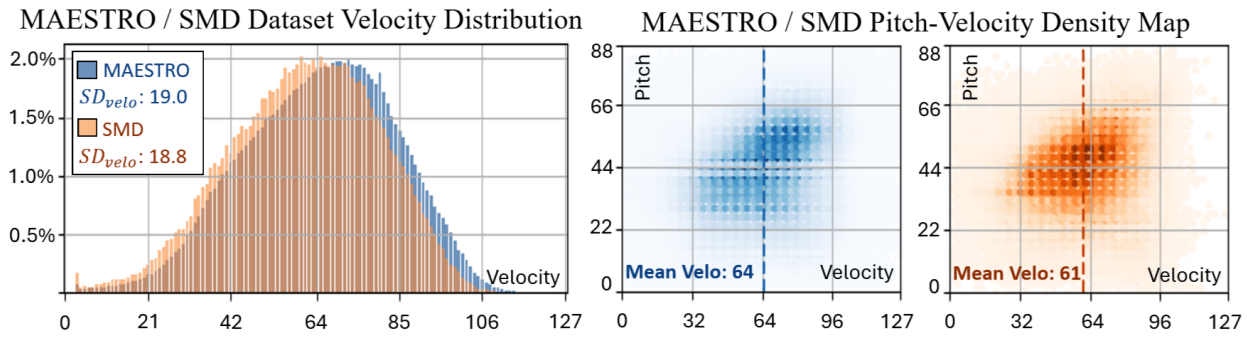


Figure 3.3: MIDI data distributions of the MAESTRO (blue) and SMD (orange) datasets, with density maps highlighting the similarity in their MIDI feature correlations.

Table 3.1: Selected SMD performances for the subjective listening test. SMD composer statistics and their overlap with the MAESTRO training set are also reported.

	MAESTRO train set		**SMD dataset**	
Composer	**Total Perf.**	**Total Dur.**	**Total Perf.**	**Selected Perf.**
F. Chopin	145	19.9 h	13	Op010-04
J. S. Bach	114	11.2 h	8	BWV849-02
L. van Beethoven	110	20.5 h	7	Op027No1-01
F. Liszt	93	16.0 h	3	-
R. Schumann	33	12.4 h	3	-
S. Rachmaninoff	29	4.2 h	3	Op036-02
J. Haydn	29	3.7 h	4	Hob017No4
W. A. Mozart	27	3.9 h	2	KV265
A. Scriabin	22	4.1 h	1	-
J. Brahms	20	6.1 h	3	-
B. Bartok	0	0 h	3	SZ080-03
M. Ravel	0	0 h	2	JeuxDEau

the similar distributional statistics in Figure 3.3. In addition, SMD is used for the subjective listening test, with eight selected performances listed in Table 3.1. Because SMD overlaps with MAESTRO in composer style but not in exact performances, it allows us to test both familiar and less familiar musical styles.

3.6.2 Training Setup

The trained models include the proposed U-Net and a re-implemented ConvAE baseline [81]. Following [129], continuous segments from the same piece are arranged into the same batch in order to preserve local musical continuity during training. Both models are trained for 300 epochs on the MAESTRO training set with learning rate 10^{-5}, batch size 3, and the same loss function. Training takes approximately 12 hours on an NVIDIA V100 32 GiB GPU using the Ranger21 optimiser. The top three checkpoints of each model, selected by validation performance on MAESTRO, are then tested on the MAESTRO test split and on SMD for cross-dataset evaluation.

3.6.3 Evaluation Metrics

Objective All objective metrics are computed on denormalised MIDI velocity in the original range of 0 to 127 after note-level post-processing. Let $\mathcal{I}$ denote the set of reference note onsets, and let V_i and $\hat{V}_i$ denote the reference and predicted velocity of note i, respectively. We report mean absolute error (MAE), mean squared error (MSE), standard deviation of absolute error (SD_{ae}), $\text{Recall}_{10\%}$, and the standard deviation of predicted velocity (SD_{velo}):

$$\text{MAE} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \left| V_i - \hat{V}_i \right|, \tag{3.6}$$

$$\text{MSE} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \left(V_i - \hat{V}_i \right)^2, \tag{3.7}$$

$$\text{SD}_{\text{ae}} = \sqrt{\frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \left(\left| V_i - \hat{V}_i \right| - \text{MAE} \right)^2}, \tag{3.8}$$

$$\text{Recall}_{10\%} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{1}\left(\left| V_i - \hat{V}_i \right| \leq 0.1 \times 127 \right), \tag{3.9}$$

$$\text{SD}_{\text{velo}} = \sqrt{\frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \left(\hat{V}_i - \bar{\hat{V}} \right)^2}, \quad \bar{\hat{V}} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \hat{V}_i. \tag{3.10}$$

Here $\mathbf{1}(\cdot)$ is the indicator function. $\text{Recall}_{10\%}$ uses an absolute tolerance of 10% of the full MIDI velocity range. MAE, MSE, and SD_{ae} quantify direct prediction error, whereas SD_{velo} summarises how much dynamic variation the system preserves. The latter is especially relevant here because a model can obtain moderate average error while still collapsing towards an unnaturally narrow dynamic range.

Subjective To complement the objective metrics, we also conduct a reference-based perceptual evaluation and summarise the listener responses as mean opinion scores (MOS) [130]. The protocol is MUSHRA-inspired rather than a strict implementation of the ITU procedure. Listeners hear the audio rendered from human-performed velocity as the reference and then rate each system output according to its perceived similarity to that reference. Ratings are collected with a continuous 0–5 slider whose five labelled intervals correspond to "bad", "poor", "fair", "good", and "excellent" similarity. The reported MOS is the average of these scores across listeners and stimuli, and 95% confidence intervals are reported to show rating consistency.

3.7 Quantitative Results

Tables 3.3 and 3.2 report the model performance when all trainable systems are trained only on the MAESTRO training set. The Flat model assigns a fixed velocity of 64, corresponding to the default behaviour of common music software. The Seq2Seq baseline uses pretrained weights from [82], while ConvAE [81] is re-implemented and trained within the same framework. Across both test conditions, the proposed U-Net outperforms the comparison systems on all reported objective metrics.

Table 3.2: Quantitative results on the MAESTRO test set. Arrows ↑ and ↓ indicate whether higher or lower values are better. Best score is highlighted in bold.

Model	MAESTRO test set				
	MAE ↓	**MSE** ↓	**SD$_{ae}$** ↓	**Recall$_{10\%}$** ↑	**SD$_{velo}$** ↑
Flat (all velocities set to 64)	14.8	333.8	10.2	48.5%	0
Seq2Seq [82]	13.5	286.1	9.9	53.8%	6.2
ConvAE [81] (re-implemented)	12.3	250.7	9.7	59.4%	9.7
U-Net (proposed)	**11.5**	**226.2**	**9.4**	**63.8%**	**10.7**

Table 3.3: Quantitative results on the SMD dataset. Arrows ↑ and ↓ indicate whether higher or lower values are better. Best score is highlighted in bold.

Model	SMD dataset				
	MAE ↓	**MSE** ↓	**SD$_{ae}$** ↓	**Recall$_{10\%}$** ↑	**SD$_{velo}$** ↑
Flat (all velocities set to 64)	15.3	367.5	10.4	49.2%	0
Seq2Seq [82]	15.1	356.9	10.9	48.8%	8.5
ConvAE [81] (re-implemented)	12.5	258.1	9.6	58.5%	9.8
U-Net (proposed)	**11.2**	**217.5**	**9.2**	**65.1%**	**11.1**

The metric behaviour warrants some discussion. When the predictions are visualised, as in Figure 3.4, two patterns become clear. First, accuracy metrics such as MAE, MSE, SD_{ae}, and $\text{Recall}_{10\%}$ usually distinguish the models well, but there are cases in which they fail to reflect perceptual differences. The Chopin Op. 10 No. 4 excerpt is one such example, where a mid-range local optimum produces superficially acceptable error values. Second, SD_{velo} is especially informative because it reflects the dispersion of predicted velocities around their mean. Higher values tend to reveal clearer differentiation between the hands and therefore a more human-like intensity profile.

3.8 Ablation Summary

Table 3.4 presents an ablation study evaluating key components of the proposed U-Net model.

Window attention mechanism. We evaluated different attention configurations to assess their impact. Removing attention reduced performance, which highlighted the attention mechanism's role in focusing on salient features and improved U-Net's performance. The 1×1 window attention (i.e., global attention) was tested alongside {2×2, 4×4, 8×8} windows, with GPU memory requirements gradually reducing. Among these, 2×2 window attention performed best. The result shows that while window attention mitigates MIDI sparsity, large windows over-compress information, reducing effectiveness.

MIDI segment duration. The analysis shows that reducing it from 20 to 10 seconds improves performance, while further reduction to 5 seconds slightly worsens the accuracy metrics. This is likely because a 10-second segment roughly encompasses four measures at 120 beats per

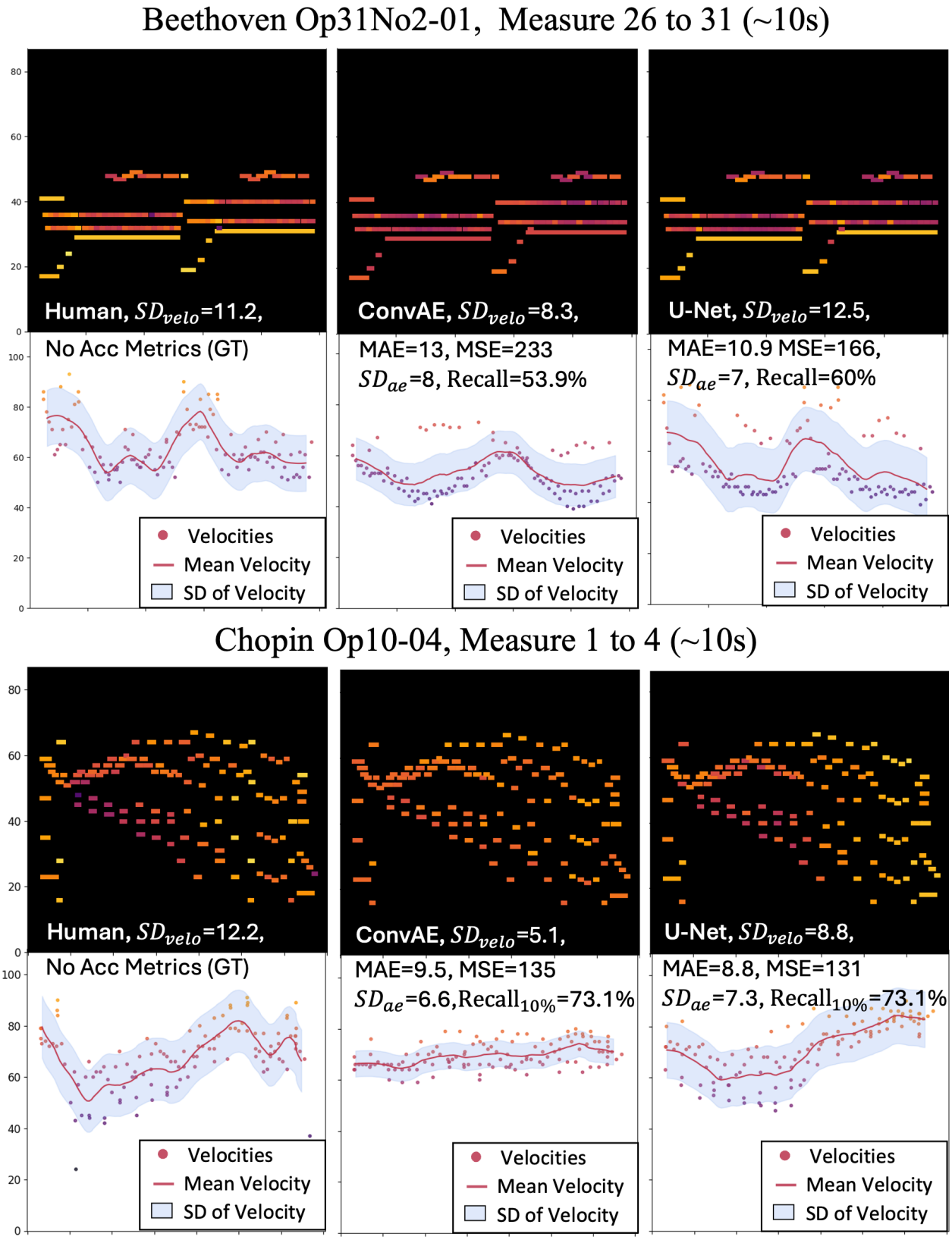


Figure 3.4: Comparison of human-performed velocity with ConvAE and U-Net predictions. The U-Net result is more human-like than ConvAE, but for Chopin Op. 10 No. 4 the standard accuracy metrics do not fully reflect this difference.

Table 3.4: Ablation study on the MAESTRO test set. *Attn.* denotes component ablation by adjusting the attention window, or removing attention. *Dur.* represents the MIDI segment duration, $< m, w >$ indicates loss with masking and weighting. An asterisk * marks the best configuration used for quantitative evaluation.

Model	Attn.	Dur.	Loss	MAESTRO test set				
				MAE ↓	MSE ↓	SD_{ae} ↓	$Recall_{10\%}$ ↑	SD_{velo} ↑
U-Net	1x1	10s	Comb $< m, w >$	11.7	237.7	9.7	62.3%	10.3
U-Net*	**2x2**	**10s**	**Comb** $< m, w >$	**11.5**	**226.2**	**9.4**	**63.8%**	10.7
U-Net	4x4	10s	Comb $< m, w >$	11.6	230.5	9.5	63.3%	10.7
U-Net	8x8	10s	Comb $< m, w >$	11.7	233.0	9.6	62.8%	10.6
U-Net	no attn	10s	Comb $< m, w >$	12.3	252.5	10.0	61.3%	9.9
U-Net	2x2	5s	Comb $< m, w >$	11.6	229.2	9.5	63.3%	11.0
U-Net	2x2	15s	Comb $< m, w >$	11.6	231.9	9.5	63.1%	10.5
U-Net	2x2	20s	Comb $< m, w >$	12.0	244.8	9.9	61.5%	10.6
U-Net	2x2	10s	BCE	13.6	304.5	10.7	55.0%	**12.8**
U-Net	2x2	10s	BCE $< m >$	11.6	228.2	9.4	63.4%	10.0
U-Net	2x2	10s	BCE $< m, w >$	11.5	227.6	9.4	63.6%	10.9

minute (BPM), forming a complete musical phrase that aids in semantic segmentation.

Loss function. Training with BCE loss alone produced a wide velocity dispersion (highest SD_{velo}), but this came with the lowest accuracy metrics. Applying the masking $< m >$ improved accuracy but reduced SD_{velo} as a trade-off. The weighting $< w >$ encouraged predictions to deviate from the mean, increasing both SD_{velo} and accuracy metrics. Finally, adding CosSim achieved the highest overall accuracy, with a negligible drop in SD_{velo}. This combined loss function $\mathcal{L}_{\text{Comb}}^{<m,w>}$ was recognised as the optimal configuration and applied in the training of both U-Net and ConvAE.

3.9 Perceptual Evaluation

A MUSHRA-inspired reference-based listening test was conducted to evaluate the human-likeness of performances generated by U-Net, ConvAE [81], and the Flat model.

Participants. We recruited 11 expert listeners aged 18 or above, selected for substantial musical background and experience with critical listening studies. Participants completed the survey anonymously through the Qualtrics platform [131].

Stimuli. The test used eight 10-second MIDI segments, each selected from one of the SMD performances listed in Table 3.1. For each segment, the original human-performed velocities were removed to create a common symbolic input. The Flat, ConvAE, and U-Net outputs were then rendered as stimuli, while the original MIDI with human-performed velocity served as the reference. All stimuli were rendered with the PianoTeq 8 plug-in using the Steinway Model D instrument.

Calibration. The dataset also provides corresponding recordings from the Yamaha Disklavier used in performance, which approximate what the pianist heard. To render the MIDI as faithfully as possible, we calibrated the PianoTeq 8 dynamics control against those recordings. Calibration compared momentary loudness correlation and loudness range, following BS.1770/EBU R128. The results in Table 3.5 show that a dynamics setting of 60 dB best matched the original recordings, and this setting was used for all listening-test stimuli.

Table 3.5: Metrics for comparing different PianoTeq configurations against the reference recordings. The best-matching configuration, used to calibrate all listening-test stimuli, is highlighted in bold.

Dynamics (dB)	**Loudness Δ ↓**	**Correlation Coefficient ↑**
50	1.8	0.9583
60	**0.3**	**0.9591**
70	0.9	0.9561
80	1.9	0.9535
90	3.1	0.9479

Procedure. Participants were instructed to use headphones because the relevant velocity differences are subtle and are easier to judge at adequate playback quality and level. In each trial, the audio rendered from the human-performed MIDI served as the fixed reference, and the rendered system outputs were judged relative to that reference. Following a MUSHRA-inspired reference-based procedure [130], participants rated each processed stimulus independently on a continuous 0–5 scale, where the labelled intervals corresponded to "bad", "poor", "fair", "good", and "excellent" similarity. The final scores were averaged across listeners to obtain MOS, and 95% confidence intervals were computed for reporting.

Table 3.6: Listening-test results. MOS values with 95% confidence intervals are reported. "Most seen" composers include {Chopin, Bach, Beethoven}; "less seen" composers include {Rachmaninoff, Haydn, Mozart}; and "unseen" composers include {Bartók, Ravel}. Best score is highlighted in bold.

Model	**SMD dataset (selected corpus)**			
	Most Seen Compo. MOS ↑	**Less Seen Compo. MOS ↑**	**Unseen Compo. MOS ↑**	**Overall MOS ↑**
Flat	1.58 ± 0.36	1.64 ± 0.44	1.18 ± 0.42	1.50 ± 0.37
ConvAE	1.93 ± 0.34	2.40 ± 0.39	1.80 ± 0.44	2.08 ± 0.33
U-Net	$\mathbf{3.10 \pm 0.38}$	$\mathbf{3.16 \pm 0.29}$	$\mathbf{2.67 \pm 0.46}$	$\mathbf{3.01 \pm 0.34}$

Results. Although a substantial gap to human performance remains, the proposed U-Net clearly outperforms the other systems. The Flat model performs worst, consistent with earlier findings that default velocity is musically inadequate [81]. The seen, less-seen, and unseen repertoire groups are best interpreted as descriptive groups rather than as a strict complexity-controlled comparison. Even so, the relatively narrow 95% confidence intervals across groups suggest that participants rated the stimuli with reasonable consistency.

3.10 Discussion

This chapter has verified that note-level dynamics can be recovered usefully even when the symbolic notes and timing are held fixed. It turns expressive enhancement into a controllable post-processing stage rather than a full performance-rendering problem, which is exactly the setting encountered in many MIDI-production and corpus-curation workflows.

The image-colourisation view is effective because it matches the structure of the data. Polyphonic note patterns create meaningful vertical and local spatial regularities that are awkward to express in purely sequential terms. The U-Net captures this local structure well, and the sparsity-aware loss prevents the model from spending most of its capacity on silent regions. The results suggest that this representation choice is a major reason the method surpasses the sequence-based baselines considered here.

The evaluation design is therefore deliberately broader than a single regression score. Objective error metrics quantify note-wise accuracy, SD_{velo} checks whether the predicted velocity distribution retains expressive spread, and the listening test verifies that the recovered layer changes perceived musical quality. This combination is important because a velocity-completion model can be numerically close to the target while still sounding too flat, or can sound more expressive while producing larger point-wise deviations.

The empirical scope of the chapter is piano-based, using datasets recorded with Yamaha Disklavier instruments. Within that scope, the cross-dataset MAESTRO-to-SMD results show that the image-colourisation formulation is not merely memorising one corpus.

3.11 Summary

This chapter studied the most controlled form of expressive-label recovery: the notes, onsets, and offsets are already fixed, and only the missing MIDI velocity layer is inferred. By casting the piano roll as an image-like representation, the proposed U-Net uses local pitch–time neighbourhoods and vertical chordal patterns that are weakened in a purely sequential formulation. Window attention and an onset-masked, velocity-aware loss are not incidental additions; they match the sparse structure of the target and prevent the model from optimising mainly around silence.

The chapter also shows why velocity completion should not be evaluated only as point-wise regression. The MAESTRO-to-SMD experiments indicate that the model transfers beyond its training corpus, while SD_{velo} and the listening test expose the musical problem that MAE alone can hide: a prediction may be close to the target and still sound too flat. This makes the chapter the symbolic-only baseline for the thesis. It preserves the composition exactly, treats dynamics as an editable layer, and establishes that such a layer has perceptual value before any audio evidence is introduced.

Chapter 4

Score-Informed Transformer Correction for AMT Velocity Estimation

4.1 Chapter Overview and Position in the Thesis

This chapter addresses Scenario S2 of the thesis. In this setting, a performance audio recording and an aligned symbolic score are both available. This situation is common in music education, rehearsal analysis, and curated audio–score archives. The score supplies reliable note content and approximate temporal structure. However, its expressive information is often incomplete, simplified, or unreliable. At the same time, modern automatic music transcription (AMT) systems can estimate note-level MIDI velocity from audio, but these estimates remain noisy and sensitive to acoustic domain shift. The central question of this chapter is: *when note events are already known from a score, how should score information be used to improve AMT-estimated velocity without redesigning the whole transcription backbone?*

Recent progress in velocity-enabled piano AMT has supported the creation of expressive piano corpora [6, 117, 132–134] and downstream applications, such as expressive rendering [135], music education [10], and music generation [136]. Although many audio recordings with aligned scores are available, reliable note-level dynamics, i.e., MIDI velocity, remain much less available. In many real collections, a score exists, but its velocity values are either quantised, manually entered, inherited from notation defaults, or simply unrelated to the performed dynamics. This creates a natural score-informed correction problem rather than a pure end-to-end transcription problem.

The present chapter keeps the note-level velocity target but introduces real audio and aligned score support. AMT backbones evolve rapidly, from CRNN designs to frequency–time Transformers and event-based models [4, 69, 71, 74]. If each score-informed system requires a new early-fusion architecture, then any gain in velocity estimation is tightly coupled to one backbone generation. This chapter instead asks whether score information can be injected *late*, as a lightweight correction layer, so that the same idea remains reusable as AMT front ends change. This design choice is what distinguishes the chapter from earlier score-informed work.

4.2 Task-Specific Related Work and Motivation

Score-informed MIDI velocity estimation developed in parallel with velocity-enabled AMT. Earlier studies estimated note intensity from piano recordings with hand-crafted acoustic measurements, statistical analysis, or score-informed factorisation [7, 60, 93, 94]. These studies established the feasibility of the task, but their dependence on manual parameterisation limited portability across datasets and recording conditions. More recent work moved to deep learning and showed that aligned score information can substantially improve note-level dynamics estimation [23, 61, 83]. The most competitive of these systems use feature fusion, where score information is injected into the acoustic network during feature extraction.

Those feature-fusion approaches are effective, but they also create a practical bottleneck. The score-informed mechanism becomes entangled with the acoustic encoder. As a result, the whole system must usually be re-engineered for each new AMT backbone. This is awkward in a fast-moving research area where strong transcription front ends already exist and continue to improve. For a thesis contribution, accuracy alone is not enough. The method should also expose a reusable principle. This is the main reason for the late-correction stance adopted in this chapter.

This chapter is also closely related to velocity-enabled piano AMT. Modern systems such as HPT, HPPNet, hFT-Transformer, and Transkun v2 already estimate MIDI velocity together with note events [4, 69, 71, 74]. However, audio-only velocity estimation remains acoustically fragile. Recent analysis shows that transcription models can absorb strong sound and recording biases, which then harm cross-dataset behaviour [137]. This makes score-informed refinement attractive even when the AMT backbone is already strong. The role of the aligned score is not to replace audio. Its role is to provide acoustics-invariant structure that constrains how the audio-driven estimate should be corrected.

Finally, the chapter should be distinguished from audio–score alignment work. Alignment studies correct *when* a note occurs in the score relative to the audio [14, 138, 139]. The present problem is different. We assume that note content and note timing are already available through alignment, and we focus on *how strongly* each aligned note was played. This narrow focus is deliberate. It lets the chapter isolate note-level dynamics recovery as a standalone problem.

4.3 Problem Setting

The task studied in this chapter is *score-informed MIDI velocity estimation*. Given a piano performance audio recording and a time-aligned score, we seek to output MIDI notes with corrected note-level velocity values. The score is assumed to provide the note events that should receive a velocity value. The remaining problem is the estimation or correction of velocity at each note onset.

This setting is practically important for two common use cases. First, educational scenarios often provide a reference score together with student performance audio. Second, large online archives often provide aligned audio–score pairs but do not provide trustworthy expressive labels.

In both cases, an audio-only AMT system leaves dynamics analysis incomplete. A score-informed system can use symbolic structure to stabilise the estimate while retaining acoustic evidence from the performed sound.

Two assumptions should be stated clearly. First, the method is designed to correct velocity, not to determine whether a note exists. Second, the score alignment is assumed to be sufficiently reliable to define note events. This does not mean the score must be perfectly aligned. Indeed, later sections explicitly test robustness under moderate artificial timing shifts. However, the problem formulation still presumes that the score and audio refer to the same performed notes.

Relative to the rest of the thesis, this chapter therefore sits between symbolic completion and higher-level dynamic analysis. Unlike Chapter 3, it operates on real audio and must cope with acoustically induced error. Unlike Chapter 5, its output remains note-level MIDI velocity rather than broader dynamic categories or change patterns. This middle position makes it a useful bridge in the thesis narrative.

4.4 Modular Correction Principle

As discussed in Section 4.2, prior score-informed designs inject score information during acoustic feature processing and therefore couple the score-informed mechanism to a specific acoustic encoder. The modular alternative pursued here moves score information to a late-correction stage that sits after the AMT velocity branch. This relocation gives the design three concrete properties that are exploited in the rest of the chapter: it preserves compatibility with existing AMT pipelines, it isolates dynamics refinement from note transcription, and it allows the same correction module to be reused across acoustically different AMT backbones. Figure 4.1 illustrates the distinction.

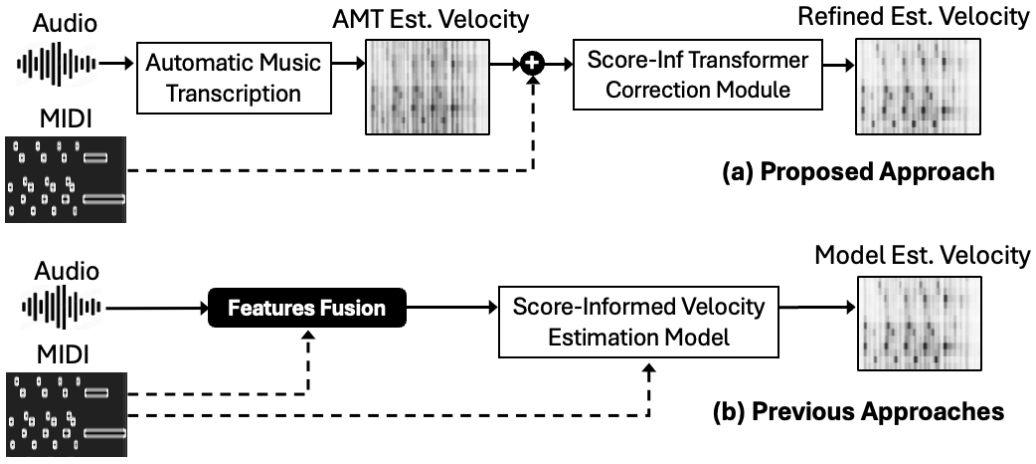


Figure 4.1: Conceptual comparison between the proposed modular correction approach and earlier score-informed feature-fusion approaches.

The interface of the proposed system is intentionally simple. The AMT backbone first predicts a preliminary velocity map from audio. The aligned score is then converted into symbolic features at the same time resolution. A lightweight score-informed module finally corrects the preliminary estimate at note onsets. This design has three practical advantages.

First, it keeps the acoustic front end reusable. Second, it allows the correction module to be attached to several AMT backbones with only small engineering changes. Third, it constrains the score-informed stage to a narrow task. The module adjusts *how loudly* a note is played. It does not decide *whether* the note exists.

This separation of roles is important. The acoustic branch still provides the initial evidence for performance intensity. The score does not overwrite that evidence. Instead, it regularises the estimate by telling the model where notes occur and how they are organised in time. Later results will show that this division of labour is strong enough to improve both in-domain accuracy and cross-dataset behaviour.

4.5 Score-Informed Correction Module

Figure 4.2 shows the complete framework. The score-informed module is attached after the velocity branch of a chosen AMT system. Only the velocity estimate is corrected. The rest of the transcription backbone remains unchanged.

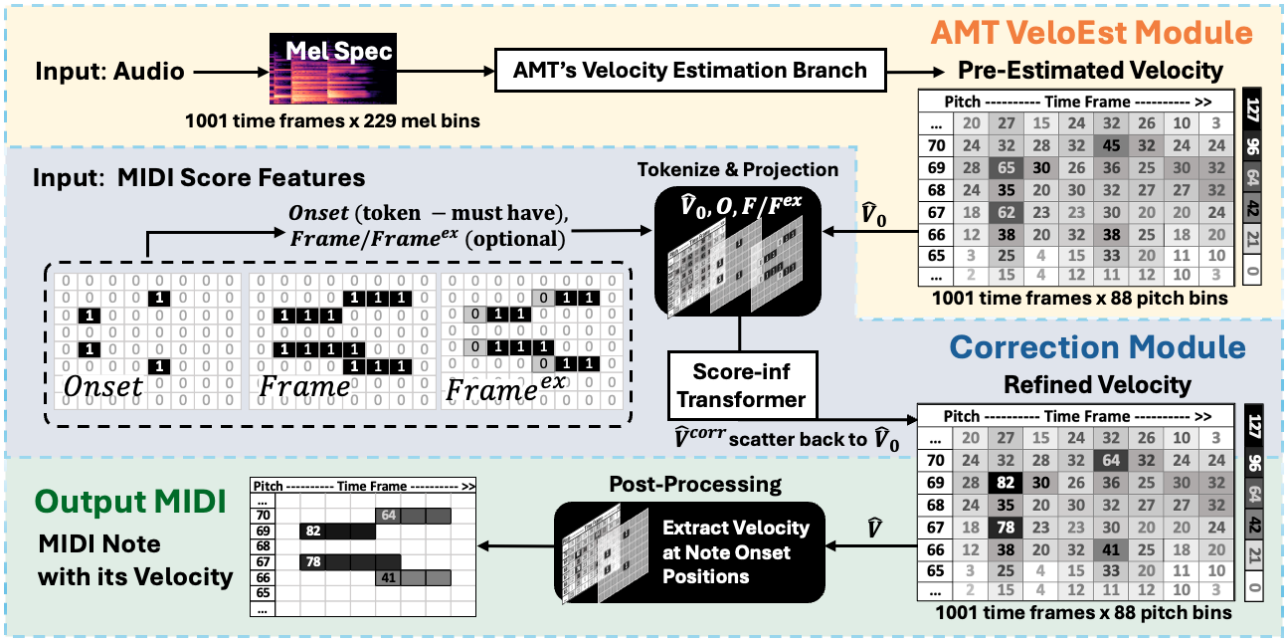


Figure 4.2: Proposed score-informed MIDI velocity estimation framework. The AMT backbone provides a preliminary velocity estimate. The score-informed module refines this estimate using features extracted from the aligned MIDI score.

4.5.1 MIDI score features

Consistent with the standard paradigm of score-informed MIDI velocity estimation [23], we rasterise the aligned score into pianoroll-like binary matrices at the same time resolution as the AMT output. Let T be the number of frames and P the number of piano keys. In our setting, $T = 1001$ and $P = 88$. We define the onset matrix $\mathbf{O} \in \{0, 1\}^{T \times P}$, the frame matrix

$\mathbf{F} \in \{0,1\}^{T\times P}$, and the onset-excluded sustain matrix $\mathbf{F}^{\text{ex}} \in \{0,1\}^{T\times P}$ as

$$\mathbf{F}^{\text{ex}} = \mathbf{F} - \mathbf{O}. \tag{4.1}$$

We further define the nonzero index sets

$$\mathcal{I} = \{(t,p) \mid O_{t,p} = 1\}, \qquad \mathcal{J} = \{(t,p) \mid F_{t,p} = 1\}. \tag{4.2}$$

The onset set $\mathcal{I}$ is used for tokenisation in the Transformer correction module. The frame set $\mathcal{J}$ is used for loss masking.

4.5.2 Architecture

The proposed architecture adopts a modular design. AMT systems are usually multi-task models that estimate onset, offset, frame, and velocity simultaneously. We isolate the velocity estimation branch of the chosen AMT baseline and extend it with a score-informed correction module. To make the design choice explicit, we study two correction modules: an early frame-wise BiLSTM retained for ablation, and a token-based Transformer used in the final system.

Velocity estimation branch in AMT. This component is identical to the velocity branches of the selected AMT baselines, namely HPT [4], HPPNet [74], and DynEst [26]. All settings follow the original implementations and public repositories. For a fair comparison, all baselines use the same log-Mel spectrogram input with a 22.05 kHz sampling rate, 2048-point FFT, 100 FPS, Hann window, and 229 mel bins. This yields an input tensor of size 1001×229. The velocity branch outputs a preliminary velocity map $\hat{\mathbf{V}}_0 \in [0,1]^{T\times P}$.

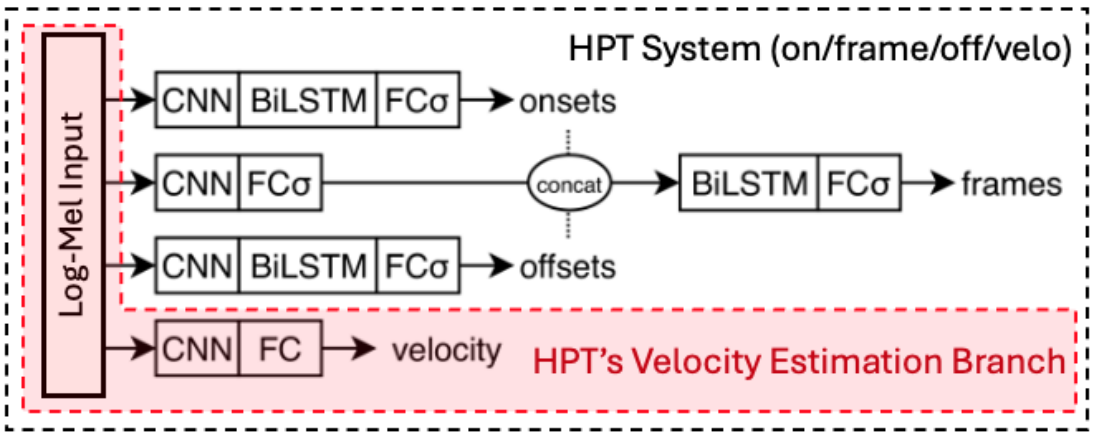


Figure 4.3: Velocity Estimation Branch in High resolution piano transcription (HPT) system [4].

Early design: score-informed BiLSTM correction module. As an early design, we first explored a frame-wise bidirectional LSTM (BiLSTM) as the correction module. At each frame t, the preliminary velocity estimate is concatenated with aligned score features along the feature dimension:

$$\mathbf{z}_t = \left[\hat{\mathbf{V}}_{0,t,:};\, \mathbf{O}_{t,:};\, \mathbf{F}_{t,:}\right] \quad \text{or} \quad \mathbf{z}_t = \left[\hat{\mathbf{V}}_{0,t,:};\, \mathbf{O}_{t,:};\, \mathbf{F}^{\text{ex}}_{t,:}\right], \tag{4.3}$$

where $\mathbf{z}_t \in \mathbb{R}^{3P}$. The sequence $\{\mathbf{z}_t\}_{t=1}^{T}$ is processed by a BiLSTM with 256 hidden units in each direction, which yields hidden states $\mathbf{h}_t \in \mathbb{R}^{512}$. A fully connected layer with Sigmoid activation then maps each $\mathbf{h}_t$ to a refined velocity vector $\hat{\mathbf{V}}^{\text{corr}}_{t,:} \in [0,1]^P$. This variant uses the same score information as the Transformer module, but operates on the full frame sequence rather than onset tokens. We retain it because it provides a strong and intuitive recurrent baseline for analysis.

Final design: score-informed Transformer correction module. Our final design replaces the frame-wise BiLSTM with an onset-token Transformer correction module. For computational efficiency, we tokenise only the score onset coordinates in $\mathcal{I}$. This yields a variable-length sequence of $N = |\mathcal{I}|$ tokens per segment. For each onset token $i = (t_i, p_i) \in \mathcal{I}$, we form an embedding

$$\mathbf{x}_i = \mathbf{e}^{(t)}_{t_i} + \mathbf{e}^{(p)}_{p_i} + \mathbf{W}_f \mathbf{f}_i, \tag{4.4}$$

where $\mathbf{e}^{(t)}_{t_i}$ and $\mathbf{e}^{(p)}_{p_i}$ are learnable time and pitch embeddings. The feature vector $\mathbf{f}_i$ contains the onset-wise preliminary velocity $\hat{V}_{0,t_i,p_i}$ together with optional score-derived features from $\mathbf{F}$ or $\mathbf{F}^{\text{ex}}$, projected by $\mathbf{W}_f$. Stacking all embeddings yields $\mathbf{X} \in \mathbb{R}^{N \times d}$. This sequence is processed by a lightweight Transformer encoder implemented as a streamlined Conformer-like block [140], as shown in Figure 4.4.

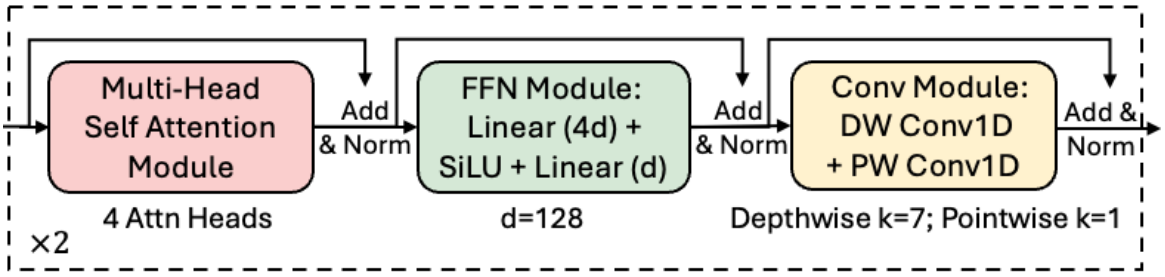


Figure 4.4: Lightweight Conformer-like Transformer encoder used in the score-informed correction module.

We use an embedding size of $d = 128$, two encoder layers, and four attention heads. A dropout rate of 0.1 is applied after each sub-module. Given the encoder outputs, a linear head predicts a scalar residual δ_i for each token. The corrected velocity is then obtained by a bounded residual update:

$$\hat{V}^{\text{corr}}_{t_i,p_i} = \text{clip}\Big(\hat{V}_{0,t_i,p_i} + \alpha \tanh(\delta_i),\, 0,\, 1\Big), \tag{4.5}$$

where α controls the maximum correction magnitude. In all experiments, $\alpha = 0.2$. The corrected values replace the preliminary estimates at onset positions. Non-onset entries remain unchanged.

The two correction modules reveal the main architectural contrast in this chapter. The BiLSTM treats the task as dense frame-wise sequence correction. The Transformer treats it as sparse onset-wise residual refinement. Both use the same score source. The later analysis shows that this difference in representation and inductive bias has a clear effect on generalisation.

4.5.3 Loss function

All models are trained with the hybrid loss adopted in prior score-informed work [23]. It combines frame-wise binary cross-entropy with onset-wise ℓ_1 loss:

$$\mathcal{L} = \frac{1}{|\mathcal{J}|} \sum_{(t,p)\in\mathcal{J}} \ell_{\text{bce}}\big(V_{t,p}, \hat{V}_{t,p}\big) + \frac{1}{|\mathcal{I}|} \sum_{(t,p)\in\mathcal{I}} \left|V_{t,p} - \hat{V}_{t,p}\right|. \tag{4.6}$$

Here $\mathbf{V}$ is the ground-truth velocity target and $\hat{\mathbf{V}}$ is the predicted velocity output of either the baseline AMT branch or the score-informed system. This hybrid supervision is useful because it constrains both the local onset value and the broader note sustain region, while still keeping note-level precision central to the objective.

4.5.4 Post-Processing

MIDI velocity is defined at note onset. It reflects the instantaneous intensity of the keystroke rather than a time-varying loudness curve [7]. This onset-centred definition is also consistent with the physical mechanism of the piano: once a key has been struck, subsequent finger pressure does not further control the sounding intensity. This differs from bowed strings such as violin, where bow pressure, bow speed, and vibrato can shape the sound after onset. For piano, the note-level velocity target is therefore naturally tied to the attack.

In our setting, a time-aligned MIDI score is available, which provides the note positions including onsets and duration (note frames). We therefore bypass AMT-style note detection and instead focus only on estimating the velocity value for each given note. During post-processing, the predicted velocity at the onset is taken and then assigned to the whole note event, consistent with the MIDI specification.

4.6 Experimental Protocol

The experiments are designed to answer four questions. First, does the score-informed correction module improve a strong AMT baseline in-domain. Second, how do the BiLSTM and Transformer correction designs differ in generalisation behaviour. Third, can the same correction principle be attached to several AMT backbones with only small overhead. Fourth, how robust is the final score-informed design to moderate audio–score misalignment.

4.6.1 Datasets

For training, we use the MAESTRO v3.0.0 dataset [5] with its default train, validation, and test split. MAESTRO contains 1,276 Yamaha Disklavier piano performances and more than 200 hours of precisely aligned audio–MIDI pairs. The Disklavier setup is particularly valuable here because it provides high-fidelity note-level velocity targets together with realistic acoustic recordings. For out-of-domain evaluation, we use the SMD dataset [118]. All models are trained

only on the MAESTRO training split. This deliberately strict protocol exposes cross-dataset behaviour clearly.

4.6.2 AMT backbones and training setup

The chapter uses three representative velocity-enabled AMT backbones. HPT [4] is adopted as the main testbed because it is lightweight and widely used. HPPNet [74] provides a parameter-efficient alternative. DynEst [26] provides a recent dynamics-oriented baseline. For each selected model, we retrain its velocity branch as an audio-only baseline and compare it with a jointly trained counterpart that includes the proposed score-informed module. For HPT, we evaluate both the BiLSTM and the Transformer correction designs. For the broader cross-backbone comparison and robustness study, we use the Transformer design because it becomes the preferred final module after the generalisation analysis.

Each model is trained for 120k iterations on a single NVIDIA RTX 3090 with 24 GiB memory. We use Adam with a learning rate of 1×10^{-4}. The learning rate is multiplied by 0.9 every 10k iterations. The batch size is 12. The random seed is 86. We select the best checkpoint by validation-set performance and then evaluate on the test sets.

4.6.3 Evaluation metrics

We retain the note-level evaluation protocol introduced in Section 3.6.3 of Chapter 3. Specifically, the present chapter reports three quantitative metrics on the denormalised MIDI velocity scale $[0, 127]$: the mean absolute error (MAE), the standard deviation of the absolute error (SD_{ae}), and $\text{Recall}_{10\%}$. Their formal definitions follow exactly the notation and formulas given in Eqs. (3.6), (3.8), and (3.9). In this chapter, the evaluation set $\mathcal{I}$ is onset-masked by the aligned score, so the comparison is carried out only at reference note positions. This yields a note-level evaluation and avoids the temporal bias of frame-wise scoring, where sustained notes would otherwise dominate the error.

A brief clarification is needed for the recall-style metric, because some earlier score-informed studies reported a different quantity under the name *Recall* [23]. In those studies, the predicted velocity map was sparse, so the goal was simply to check whether the model produced a nonzero velocity value at the reference note position:

$$\text{Recall} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} \mathbf{1}\left(\hat{V}_i > 0\right). \tag{4.7}$$

This quantity measures note-position coverage rather than the accuracy of MIDI velocity itself. In our approach, and in most modern AMT baselines, the predicted velocity maps are dense as shown in Figure 4.5. After post-processing, almost every aligned note onset therefore receives a nonzero estimated velocity, so the older Recall quickly saturates near 100% and becomes uninformative.

For this reason, the present chapter uses $\text{Recall}_{10\%}$, following the same definition as Chapter 3.

This metric counts a note as correct only when the predicted velocity falls within a 10% tolerance on the MIDI velocity scale. It is consistent with the velocity criterion used in the standard $NoteOn/OffwVelo$ evaluation implemented in the `mir_eval` package [141], which is widely used in music transcription research. Using this convention makes the evaluation easier to reproduce and helps maintain comparability with established AMT benchmarks.

4.7 Results and Analysis

4.7.1 Validation on the score-informed correction

We first attach the proposed modules to HPT and denote the resulting systems as Score-HPT. HPT is used as the initial testbed because it is lightweight and prevalent in recent piano transcription work. This lets us isolate the effect of score-informed correction before extending the method to other backbones. Table 4.1 summarises the results on the MAESTRO test set.

Table 4.1: Comparison of HPT and Score-HPT on the MAESTRO test set. Lower values are better for MAE and SD_{ae}, while higher values are better for $\text{Recall}_{10\%}$. Best score is highlighted in bold.

Model & Inputs	**MAESTRO test set**			
	MAE ↓	SD_{ae} ↓	$\text{Recall}_{10\%}$ ↑	# Params
HPT				
audio	4.6	6.1	80.6%	4.1M
Score-HPT (BiLSTM applied)				
audio + onset	4.2	5.2	88.2%	5.8M
audio + onset + frame	3.6	**4.5**	95.6%	7.5M
audio + onset + frame_{ex}	**3.5**	**4.5**	**95.8%**	7.5M
Score-HPT (Transformer applied)				
audio + onset	4.0	4.8	93.8%	5.0M
audio + onset + frame	4.0	4.7	93.9%	5.0M
audio + onset + frame_{ex}	3.9	4.7	94.0%	5.0M

Two observations matter here. First, both correction modules consistently improve the audio-only HPT baseline. This confirms that aligned score information is useful even when inserted only after the AMT velocity branch. Second, the BiLSTM achieves the strongest in-domain fit on MAESTRO, especially with onset + frame_{ex}. This shows that dense frame-wise recurrent correction is expressive enough to model the task very closely. However, the later generalisation analysis will show that the strongest in-domain fit is not the same as the best final design. The Transformer reaches slightly higher MAE on MAESTRO, but it does so with fewer parameters and with a correction mechanism that is easier to transfer across backbones.

Figure 4.5 gives a qualitative view of this effect. Compared with the baseline, Score-HPT produces a cleaner onset-centred velocity pattern. This is consistent with the intended role of MIDI velocity as an onset attribute. The score-informed module does not remove the dense map

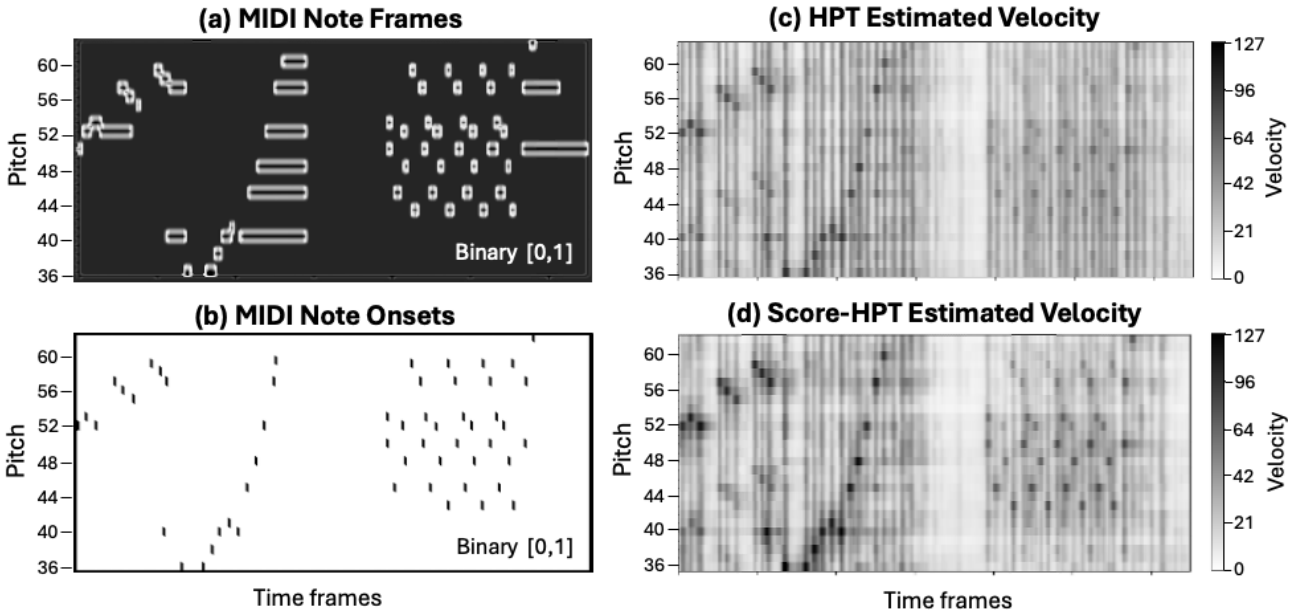


Figure 4.5: Qualitative comparison between HPT and Score-HPT (Transformer applied). The refined estimate is more concentrated around onset locations while remaining fully populated.

predicted by the AMT branch. Instead, it reshapes the estimate so that the onset locations become more informative.

4.7.2 Cross-dataset generalisation

To study generalisation, we track learning curves on the MAESTRO validation set and on a held-out 10% subset of SMD, which does not provide an official validation split. Figures 4.6 show that all score-informed HPT variants outperform the plain HPT baseline on both in-domain and out-of-domain data. For the Transformer family, the onset + frame condition gives the strongest overall cross-dataset behaviour, although the difference between score feature variants remains modest.

A clear pattern appears during training. Performance on MAESTRO improves steadily. Performance on SMD peaks earlier and then degrades. This indicates that prolonged training overfits to MAESTRO-specific acoustics. Similar sound biases in AMT velocity estimation were also reported in [137]. The score-informed module does not remove this problem completely. However, it shifts the error curves downward and stabilises out-of-domain behaviour. This supports the view that the aligned score acts as an acoustics-invariant structural prior.

The contrast between the BiLSTM and the Transformer is especially informative. The BiLSTM tends to fit MAESTRO more aggressively, but its out-of-domain degradation is also more pronounced. This behaviour is consistent with the way the two modules process the sequence. The BiLSTM consumes all 1001 frames, including many highly redundant sustain frames. Adjacent frames often carry very similar acoustic-envelope information. A recurrent model can therefore reduce training loss by memorising dataset-specific loudness trajectories, microphone responses, and reverberation patterns that repeat within MAESTRO recordings. Its bidirectional hidden state also entangles each correction with broad frame-level acoustic context.

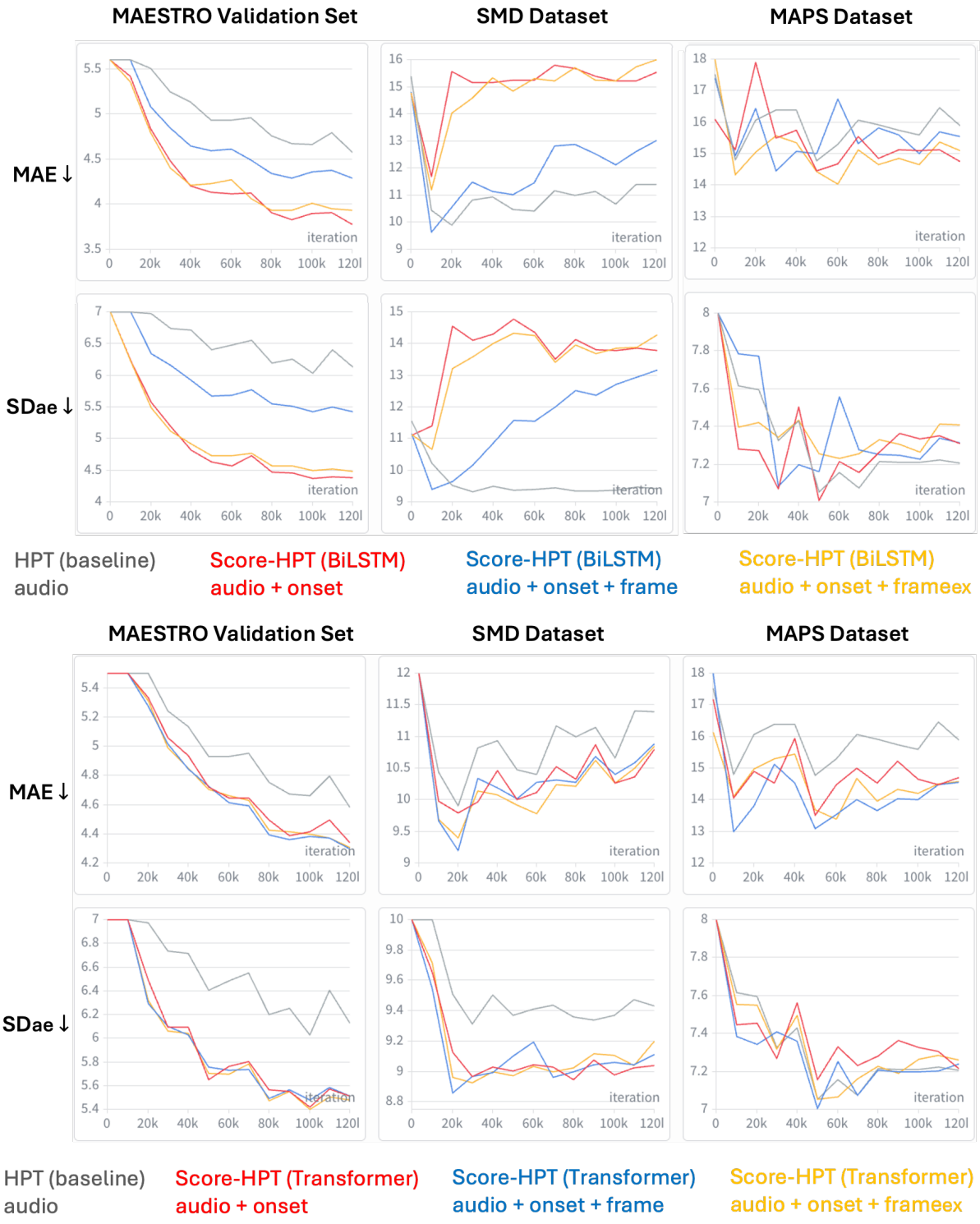


Figure 4.6: Learning curves for HPT and Score-HPT variants using the BiLSTM and Transformer correction module. All models are trained on the MAESTRO training split and evaluated every 10k iterations.

That context is helpful in-domain, but it is also where room and recording biases reside. The BiLSTM variant is additionally larger than the Transformer variant, which further increases its capacity to absorb these biases.

The Transformer has a different inductive bias. It operates only on onset tokens rather than on the full frame lattice. This removes most of the redundant sustain frames that are easiest to overfit. Self-attention then models note-to-note relations directly on the symbolic event sequence, while the bounded residual update in Eq. (4.5) constrains the magnitude of each correction. In effect, the Transformer is encouraged to learn *where* and *how much* the preliminary velocity should be adjusted, instead of re-modelling the full acoustic envelope. This yields a slightly less aggressive in-domain fit than the BiLSTM, but better cross-dataset stability. For this reason, the Transformer becomes the main correction module in the remainder of the chapter.

4.7.3 Comparison with existing systems

We next compare the Transformer-based correction strategy with existing score-informed systems and representative AMT models on the SMD dataset. To keep the comparison fair, all models follow the same training protocol and use only the MAESTRO training split for learning. Following the generalisation analysis above, we use onset + frame for all proposed Transformer-based variants. Table 4.2 summarises the results. Specifically, $\text{Recall}_{10\%}$ results of hFT-Transformer and Transkun v2 are obtained from the MIREX2024: Polyphonic Transcription challenge.[1]

Table 4.2 shows that Score-HPT and Score-DynEst achieve the best MAE and SD_{ae} under this protocol. They outperform dedicated score-informed FiLM models as well as several strong velocity-enabled AMT systems. The result is notable because the added correction module is small. The parameter overhead is about 1M.

The same correction principle is also effective beyond HPT. When attached to HPPNet and DynEst, it again improves the corresponding baseline. This supports the main thesis claim of the chapter. The benefit does not depend on one transcription backbone. Instead, the method behaves as a reusable refinement layer. Even when HPPNet and DynEst are retrained with log-Mel spectrograms rather than with their original CQT or Bark-scale specific loudness inputs, the score-informed module still yields a consistent gain. This strengthens the argument that the correction principle itself is robust.

One further detail is worth noting. Although Transkun v2 defines a very strong state-of-the-art (SOTA) in note transcription, it is less competitive in velocity MAE and SD_{ae}. A plausible reason is that its core SemiCRF design is primarily optimised for note-event modelling rather than for precise dynamics regression. This observation reinforces the main claim of the chapter: strong note transcription does not automatically imply strong note-level velocity estimation. A dedicated but lightweight correction layer can still add value.

[1] https://music-ir.org/mirex/wiki/2024:Polyphonic_Transcription_Results

[2] FiLM-UNet reported 89.7% in their paper [23] is standard Recall, a less strict metric than our $\text{Recall}_{10\%}$.

Table 4.2: Comparison on the SMD dataset. All models are trained only on the MAESTRO training split. Best score is highlighted in bold.

Model	SMD dataset			
	MAE ↓	SD_{ae} ↓	$Recall_{10\%}$ ↑	# Params
Score-informed methods				
DiffVel [83]	19.7	13.1	–	–
FiLM Conv [61]	15.1	12.3	–	–
FiLM U-Net [23]	9.9	7.8	73.0% [2]	8.8M
Proposed (Transformer applied)				
Score-HPT	**8.0**	**6.9**	80.3%	5.0M
Score-HPPNet	9.2	7.3	79.3%	1.5M
Score-DynEst	**8.0**	**6.9**	77.6%	14.3M
AMT systems				
HPT [4]	10.0	8.8	62.5%	4.1M
HPPNet (no CQT) [74]	10.6	9.0	60.2%	0.5M
DynEst (no BSSL) [26]	10.4	8.8	59.6%	13.3M
hFT-Transformer [69]	9.9	7.3	76.5%	5.5M
Transkun v2 [71]	13.3	11.9	**82.1%**	13.6M

4.7.4 Robustness to audio–score misalignment

Score alignment quality is a central condition for score-informed correction. This chapter therefore tests that condition directly. To evaluate robustness under imperfect synchronisation, we conduct timing-shift experiments using artificial score displacements from ±0.1 to ±2.5 seconds, following [23]. These shifts are applied to the SMD evaluation data. For example, a +0.1 shift means that the original aligned score is intentionally delayed by 0.1 seconds. All models evaluated here are the same final models used in the cross-backbone comparison above. They are trained only on perfectly aligned MAESTRO data. The results are summarised in Figure 4.7.

A methodological point is important here. In the earlier benchmark of [23], shifted scores were used as model inputs but the original unshifted score was still used during post-processing. That choice follows the earlier evaluation tradition of [61], but it introduces a temporal inconsistency. If a model is conditioned on a shifted score, then its output is tied to that shifted note grid. Post-processing should therefore use the same shifted score. We adopt this internally consistent protocol in the present chapter. Under this refinement, FiLM U-Net is substantially more robust than originally implied by the earlier report.

Applying this refined protocol, Score-HPT achieves the strongest robustness curve on SMD. More broadly, all score-informed methods degrade only moderately relative to the perfectly aligned case. Given that MIDI velocity spans the range 0 to 127, the observed increase of only a few units in MAE or SD_{ae} under even large global shifts remains small in absolute terms. Minor shifts can even lead to slight improvements for some proposed variants. A plausible

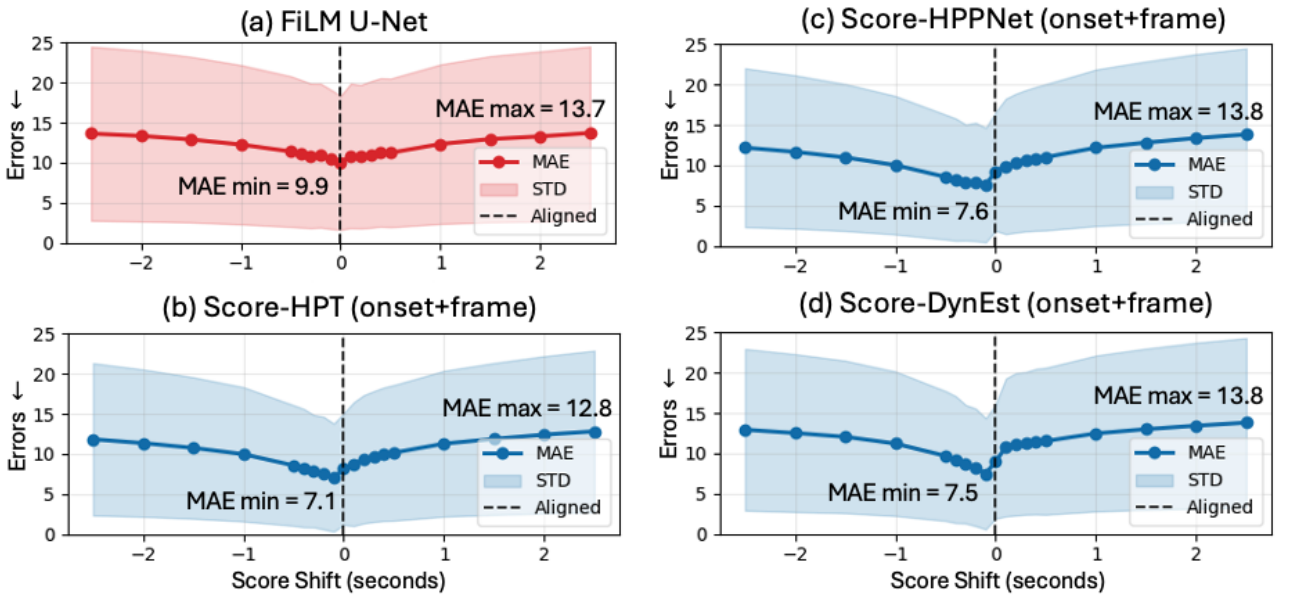


Figure 4.7: Robustness evaluation on data derived from the SMD dataset, using audio–score misalignments of ±0.1, 0.2, 0.3, 0.4, 0.5, 1, 1.5, 2, and 2.5 s.

interpretation is that the AMT branch already produces temporally fuzzy onset-wise velocity estimates, and a small score displacement may occasionally compensate for that fuzziness. The important conclusion is not that alignment no longer matters. Rather, it is that the proposed late-correction design is surprisingly tolerant to moderate global timing mismatch. This makes the method more realistic for educational and archival use, where score synchronisation is often good but not perfect.

4.8 Discussion

The main finding of this chapter is that score information is most useful when it is treated as a structural prior rather than as a full replacement for the acoustic model. The AMT backbone still supplies the initial estimate of performance intensity. The score-informed module only performs a targeted correction at onset locations. This division of labour keeps the system lightweight and explains why the gain transfers across backbones. It also clarifies why score information helps cross-dataset behaviour. The score does not change with microphone placement, room acoustics, or recording chain. It regularises a prediction target that is otherwise acoustically fragile.

The retained BiLSTM experiment makes this point sharper. The BiLSTM results show that score-informed correction is not inherently tied to attention. A recurrent module can also exploit score support and can even fit the training domain more strongly. However, the Transformer is a better final design because its inductive bias matches the task more closely. Velocity is a note-onset attribute. The onset-token Transformer models note events directly. The BiLSTM instead processes a long dense frame sequence in which many frames correspond to sustain and acoustic decay rather than to distinct note decisions. This is precisely where overfitting to recording-specific envelopes becomes easier. The Transformer therefore trades a small amount

of in-domain fit for better portability and generalisation.

The robustness experiments also refine the role of the alignment assumption. The present test perturbs the score with a *global* shift, which is the relevant condition for moderately imperfect synchronisation in curated score–audio archives. Within this tested condition, late correction remains stable even when the score grid is displaced by a surprisingly large amount. The practical bottleneck is therefore narrower than a strict perfect-alignment assumption might suggest: the method does not require sample-perfect score synchronisation to be useful.

4.9 Summary

This chapter moved from symbolic completion to score-informed audio refinement. The task is still note-level MIDI velocity, but the estimate now starts from an AMT velocity branch and is corrected using aligned score structure. The proposed Transformer correction module deliberately sits late in the pipeline: it does not redesign the acoustic encoder, and it does not decide which notes exist. It only adjusts the velocity attached to known score onsets.

The main result is therefore a design principle rather than just a lower error table. Aligned score information acts as an acoustics-invariant prior for a target that is otherwise sensitive to recording condition and transcription backbone. This explains why the same correction idea improves HPT, HPPNet, and DynEst, strengthens MAESTRO-to-SMD/MAPS generalisation, and remains stable under moderate global score–audio shifts. The BiLSTM comparison further sharpens the point: a denser sequence model can fit the training domain, but the onset-token Transformer better matches the object being estimated. In the thesis sequence, this chapter bridges the purely symbolic setting and the later label-scarce setting by showing that score structure can stabilise velocity recovery without becoming entangled with low-level acoustic modelling.

Chapter 5

Audio-to-Score Dynamics via Joint Estimation of Piano Dynamics and Metrical Structure

5.1 Chapter Overview and Position in the Thesis

Score-level dynamic markings such as *p*, *mf*, and *f* provide the principal human-readable description of musical intensity in notated music. These markings are inherently relative, are naturally indexed by the beat grid rather than by individual note onsets, and cannot in general be inferred from the symbolic score itself. This chapter addresses Scenario S3 of the thesis, in which performance audio is the only available input and the desired output is a beat-synchronous symbolic description of dynamics. Recovering such a representation requires not only the dynamic labels themselves but also the metrical structure needed to anchor them on a score.

Instead of recovering note-level velocity, which is precise and directly editable, the present chapter targets a more interpretable score-facing representation. It therefore completes the thesis map by addressing the setting in which audio is the only available input and the desired output is a beat-synchronous symbolic description rather than raw note-level control.

5.2 Task-Specific Related Work

Audio-to-score dynamics recovery is related to, but distinct from, note-level velocity estimation. A large part of the expressive-performance literature studies how performed loudness relates to symbolic dynamic markings and how those markings vary with musical context [17, 30, 58]. This literature makes two points that matter here. First, score dynamics is musically meaningful and worth recovering. Second, it is inherently ambiguous and cannot be reduced to a fixed physical threshold.

A second relevant line of work concerns metrical structure estimation. Beat and downbeat tracking has a strong literature in MIR, with modern deep models achieving increasingly robust

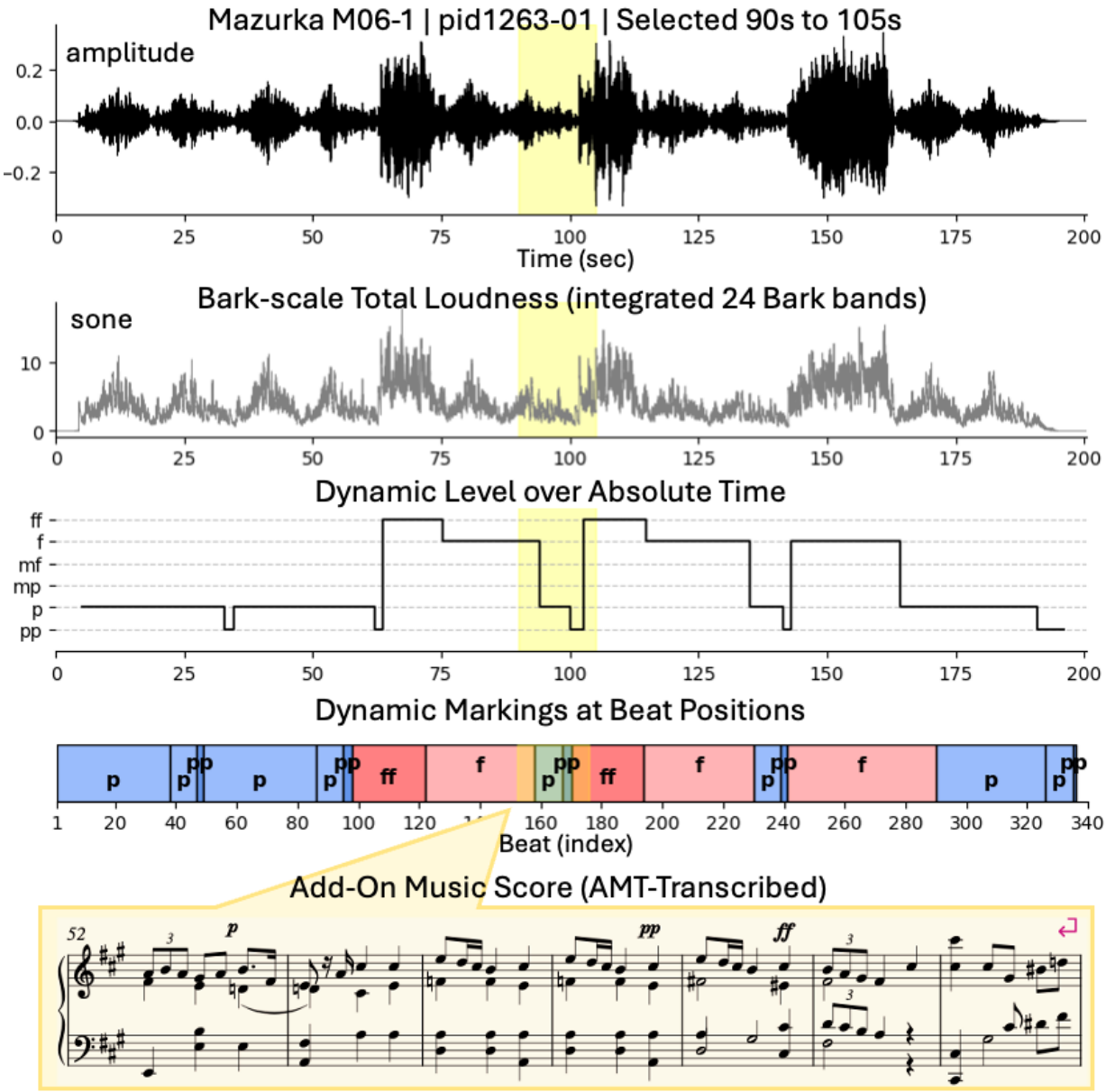


Figure 5.1: Workflow for adding dynamic markings to an AMT-transcribed score from audio. The highlighted region (90–105 s of Chopin Mazurka Op. 6 No. 1) shows a case study. Dynamic markings are aligned with the musical beat grid, and a change point indicates a contextual shift in the dynamic levels.

results [99–101]. For the present chapter, beat and downbeat estimation is not only an auxiliary task. It is part of the output interface because score dynamics is easiest to interpret and evaluate on a beat grid.

These observations motivate a joint rather than isolated formulation. Predicting score dynamics without metre produces labels that are hard to place. Predicting metre without dynamics misses the expressive target. The present chapter therefore studies score-facing dynamics recovery through joint estimation of dynamic levels, change points, beats, and downbeats.

5.3 Problem Setting and Targets

Chapter 2 already introduced the ambiguity of score dynamics, the distinction between score dynamics and MIDI velocity, and the role of beats and downbeats as a musically meaningful coordinate. Rather than repeat that background, this chapter focuses on the concrete transcription problem studied in this work.

Given a piano performance audio signal, the system predicts four synchronised output sequences on a frame grid: dynamic levels, dynamic change points, beats, and downbeats. The dynamic targets are the score-facing outputs, while beats and downbeats provide the metrical structure required to align these labels to a symbolic score. This joint design serves two practical modes of use. In the first mode, a reliable beat grid is already available from an aligned score or a score-level AMT system, and the predicted dynamic labels are attached to that grid. In the second mode, no score is available, and the model first predicts beats and downbeats by itself, then uses the estimated grid to place the dynamic labels.

More specifically, the dynamic-level target is formulated as a six-class frame-wise prediction problem, including one *blank* class before the first annotated marking and the five score-dynamics classes used in MazurkaBL: *pp*, *p*, *mf*, *f*, and *ff*. The change-point target is binary and indicates whether a contextual transition between adjacent dynamic regimes occurs at a given time. Beats and downbeats are also binary frame-wise targets. At inference time, the frame-wise outputs are converted into beat-synchronous score annotations by sampling the dynamic class at beat locations and snapping detected change points to the nearest beat.

This task directly matches the role assigned to Chapter 5 in the thesis plan: score-level dynamics should be represented as a beat-synchronous interface comprising dynamic levels, change points, and metrical structure. It also supports the broader thesis objective of recovering expressive labels without changing the underlying note content or timing.

5.4 Perceptually Grounded Features and Multi-Scale Backbone

5.4.1 Audio feature representations

Log-Mel spectrograms serve as the standard input representation in many modern MIR systems, providing a natural baseline for both dynamics estimation and beat/downbeat tracking tasks. To ensure a fair comparison with advanced beat tracking models [100, 101], their widely standardised audio preprocessing configuration is adopted throughout this chapter. Specifically, the audio is downmixed to mono and resampled to 22.05 kHz. A Hann-windowed STFT is then applied using a window length of 1024 and a hop size of 441 samples, followed by a projection onto 128 Mel bands. This yields a precise frame rate of 50 fps. Under this setting, the log-Mel input is represented as $\mathbf{X}_{\text{Mel}} \in \mathbb{R}^{128\times T}$, where T denotes the number of temporal frames after feature extraction.

Recent studies have started to question whether log-Mel features are the most suitable representation for musical dynamics. Narang et al. [24] compared log-Mel and Bark-scale loudness-based features for singing voice dynamics estimation and reported that Bark-scale features consistently outperformed log-Mel inputs. This observation is particularly relevant to the present study because score dynamics are closer to perceived intensity than to raw spectral energy. For piano dynamics, this motivation is also consistent with the use of specific loudness in earlier studies on performed dynamics [30]. BSSL represents loudness over auditory critical bands, and therefore keeps the perceptual distribution of intensity across frequency regions. This makes it more informative than a single integrated loudness curve, while being more compact and perceptually aligned than a high-dimensional spectral magnitude representation.

A core challenge was the implementation of the BSSL extractor. MOSQITO [142] provides an ISO 532-1:2017-compliant pipeline [40] appropriate for vocal settings [24], but its 48 kHz-oriented workflow is overly constrained. This is unsuitable here because the MazurkaBL dataset is natively 44.1 kHz, while the baselines for comparison are all processed under a 22.05 kHz, 50 fps feature configuration. Using MOSQITO would require an additional resampling (44.1 kHz to 48 kHz) and different FFT parameters, making the feature comparison unbalanced and less efficient. We instead implemented a custom BSSL extractor in PyTorch by reproducing the `ma_sone` function from Pampalk's Music Analysis MATLAB toolbox [143], following the algorithmic chain in [37]. The mathematical formulation of this extractor is given in Chapter 2. Using the same STFT parameters as the log-Mel input, the BSSL extractor produces $\mathbf{X}_{\text{BSSL}} \in \mathbb{R}^{22\times T}$. We use the full BSSL matrix rather than compressing it into the BSTL, because the latter would discard the frequency-band structure that may carry useful timbral and register-dependent cues for piano dynamics.

5.4.2 Shared multi-scale encoder

A central modelling challenge is the conflict between the temporal context and frame-level precision required by the four tasks. While metrical tasks (beats and downbeats) rely on high temporal resolution to localise sharp transients [99], dynamics-related tasks (dynamics and change points) depend on the surrounding phrase, local contrast, register, and relative loudness context [24], thereby requiring extended context lengths and stable spectral features. We therefore adapt the multi-scale network of [102] as the shared encoder. Since this architecture was originally designed for playing techniques spanning different temporal scales, it is exceptionally well-equipped to resolve the competing receptive-field requirements of our four tasks.

As shown in Fig. 5.2, given an $F \times T$ time–frequency feature, where F is the number of frequency bands and T is the number of time frames, the encoder first applies 2D batch normalisation and then forms three temporal branches at resolutions T, T/s, and T/s^2, where s denotes the temporal scaling factor. The full-resolution branch preserves frame-level timing evidence, the middle branch provides an intermediate representation, and the coarsest branch gives the largest effective receptive field. Following [102], residual blocks are used for local feature extraction, while self-attention blocks are applied to strengthen long-range temporal modelling. After multi-scale fusion, the encoder outputs a shared latent sequence $\mathbf{Z} \in \mathbb{R}^{T\times 8}$, which is then passed to the MMoE decoder and task-specific heads.

Unlike the reference architecture which fixes the scaling factor to 3 [102], we treat s as a tunable hyperparameter to explicitly control the trade-off between temporal precision and contextual abstraction. If s is too small, the three branches become insufficiently differentiated and the encoder behaves close to a single-resolution model with redundant side branches. If s is too large, the coarsest branch provides stronger context but may over-compress information that is still useful for event localisation. Under our 50 fps feature rate, the empirically best value $s = 5$ means that the coarsest branch operates at approximately 2 fps, giving the attention blocks a substantially longer temporal view while the full-rate branch still preserves 20 ms frame-level timing. The ablation study in Section 5.7.2 discusses this effect in more detail.

The feature choice also interacts with the model design. Using 22-band BSSL instead of 128-bin log-Mel reduces the frequency dimension entering the convolutional encoder, lowering memory usage and making longer segments feasible. This allows the model to capture phrase-level dynamics while remaining compact, as further confirmed in Section 5.7.2 Ablation Study.

5.5 Joint Estimation with Task-Aware Decoding

5.5.1 MMoE decoder

The shared encoder provides a common latent representation, but the tasks remain acoustically diverse. To reduce negative transfer, we use a Multi-gate Mixture-of-Experts (MMoE) layer [27] as a task-aware decoder.

The MMoE module contains a pool of eight shared experts. Each expert is a lightweight

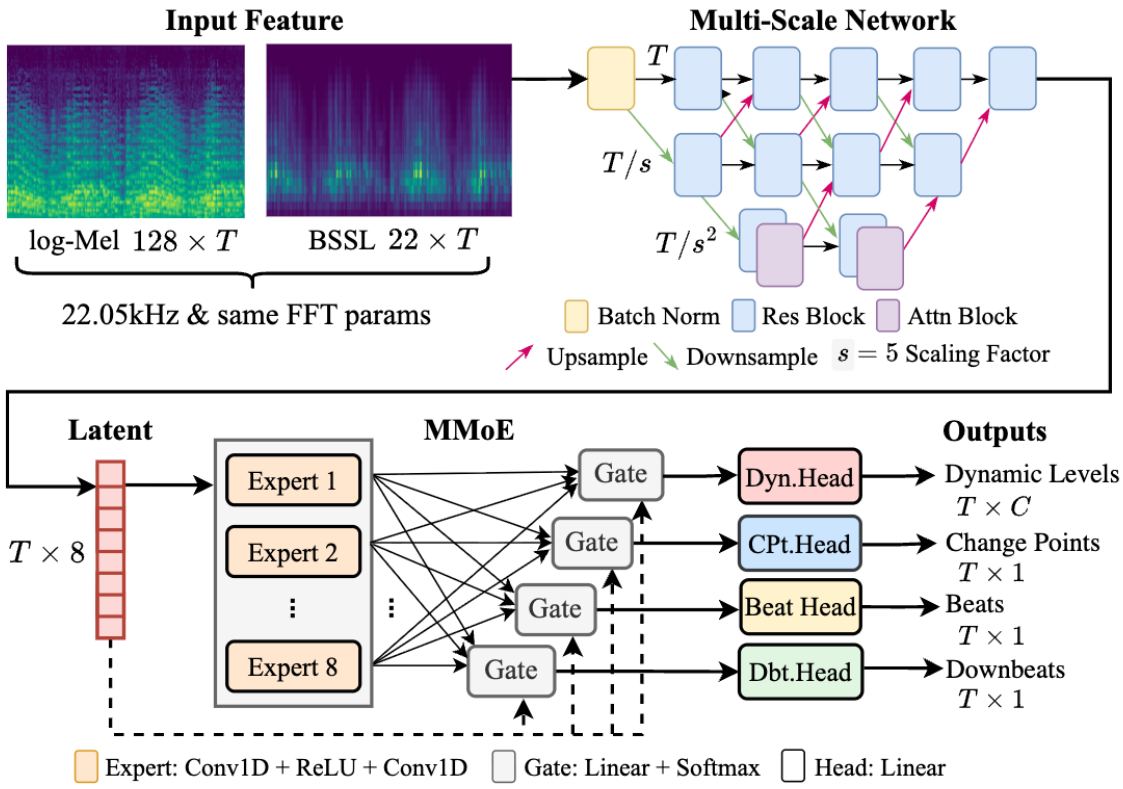


Figure 5.2: Architecture of the proposed multi-task framework. A three-branch multi-scale network encodes either log-Mel or BSSL input at temporal resolutions T, T/s, and T/s^2, producing a latent sequence of shape $T \times 8$. An 8-expert MMoE with four gates forms task-specific features, followed by linear heads for the four targets. The scaling factor s is a tunable hyperparameter.

temporal convolutional block with two sequential 1D convolutional layers of kernel size 3 and length-preserving padding, separated by a ReLU activation. All experts process the shared latent sequence $\mathbf{Z}$ in parallel.

For each task k, a dedicated gating network $G_k(\cdot)$ acts as a dynamic router. At time step t, the gate takes the latent feature $\mathbf{z}_t$ and outputs a softmax-normalised weight vector $\mathbf{w}_k(t) \in \mathbb{R}^8$. The task-specific feature vector is then computed as a weighted combination of expert outputs:

$$\mathbf{y}_{k,t} = \sum_{i=1}^{8} \mathbf{w}_{k,i}(t) \cdot \mathbf{e}_{i,t}, \qquad \mathbf{w}_k(t) = \mathrm{Softmax}(G_k(\mathbf{z}_t)). \tag{5.1}$$

Here, $\mathbf{e}_{i,t}$ is the output of expert i at time step t. Repeating this process frame by frame yields the task-specific sequence $\mathbf{Y}_k = [\mathbf{y}_{k,1}, \ldots, \mathbf{y}_{k,T}]^\top \in \mathbb{R}^{T \times 8}$, which is then mapped to logits by a separate linear head for each task.

5.5.2 Task-specific post-processing

The model outputs frame-wise probabilities, which are converted into discrete musical events through tailored post-processing. For beat and downbeat detection, we adopt the post-processing procedure from [101]: probabilities are thresholded at 50%, and peak picking is applied within a 7-frame local neighbourhood (radius 3 at 50 fps). For dynamic markings, the dynamic level at each beat is determined by taking the `argmax` of the class probabilities at that frame. When

an external beat grid is available, this sampling step is performed on that grid; otherwise, the model's own beat predictions are used, which gives the fully end-to-end mode considered in this chapter.

Change points are determined in two stages. First, we identify all frames whose change-point probability exceeds 75%. Second, we snap each candidate to the nearest detected beat. This beat-snapping step is motivated by the annotation style of MazurkaBL, where dynamic markings are beat-aligned rather than freely placed in continuous time [96]. The step also keeps evaluation consistent with prior work [97].

5.5.3 Loss function

Training uses a composite multi-task objective,

$$\mathcal{L}_{\text{MTL}} = \mathcal{L}_{\text{Dyn}} + \mathcal{L}_{\text{CPt}} + \mathcal{L}_{\text{Beat}} + \mathcal{L}_{\text{Dbt}}. \tag{5.2}$$

The dynamic head is trained with beat-masked cross-entropy. Let $\mathbf{d}_t \in \mathbb{R}^C$ denote the dynamic logits at frame t, let $c_t \in \{1, \ldots, C\}$ denote the reference class, and let $m_t^{\text{beat}} \in \{0, 1\}$ be a mask that is one only at annotated beat frames. Then

$$\mathcal{L}_{\text{Dyn}} = \sum_{t=1}^{T} m_t^{\text{beat}} \, \text{CE}(\mathbf{d}_t, c_t). \tag{5.3}$$

In other words, only beat-aligned frames contribute to the dynamic loss, while inter-beat frames are ignored. This directly reflects the annotation style of MazurkaBL and prevents the dynamic head from learning spurious inter-beat fluctuations between neighbouring beats.

For the three binary tasks, i.e., change points, beats, and downbeats, we adopt the shift-tolerant weighted BCE from Beat This [101]. We first write the weighted BCE in the familiar form

$$\text{BCE}_\alpha(y, \hat{y}) = -\alpha y \log \hat{y} - (1 - y) \log(1 - \hat{y}), \tag{5.4}$$

where α is the positive-class weight. For each binary task $k \in \{\text{CPt}, \text{Beat}, \text{Dbt}\}$, let $y_t^{(k)} \in \{0, 1\}$ and $\hat{y}_t^{(k)} \in (0, 1)$ denote the target and predicted probability at frame t, and let $\mathcal{P}_r(\cdot)$ denote one-dimensional max-pooling over a temporal radius r. The shift-tolerant loss is written as

$$\mathcal{L}_k^+ = \sum_{t=1}^{T} y_t^{(k)} \, \text{BCE}_{\alpha_k}\left(1, \mathcal{P}_3(\hat{\mathbf{y}}^{(k)})_t\right), \tag{5.5}$$

$$\mathcal{L}_k^- = \sum_{t=1}^{T} \left(1 - \mathcal{P}_6(\mathbf{y}^{(k)})_t\right) \text{BCE}_{\alpha_k}\left(0, \mathcal{P}_3(\hat{\mathbf{y}}^{(k)})_t\right), \tag{5.6}$$

$$\mathcal{L}_k = \mathcal{L}_k^+ + \mathcal{L}_k^-. \tag{5.7}$$

The positive term applies BCE to the local maximum prediction within a 7-frame neighbourhood, so a small temporal shift around the annotation is tolerated. The negative term applies BCE

only outside a wider exclusion zone created by $\mathcal{P}_6(\mathbf{y}^{(k)})$, so frames close to a positive label are not penalised as hard negatives. This design is useful for beat-like targets because the positive frames are extremely sparse and the annotations are not perfectly precise at frame level. In implementation, the same loss is applied on logits for numerical stability.

5.6 Experimental Setup

5.6.1 Dataset and protocol

We use the MazurkaBL dataset [96], a score-aligned corpus of 2,098 solo piano recordings covering 44 Chopin Mazurkas. After excluding two mazurkas with irregular dynamics annotations (M06-4 and M63-2), 1,999 recordings are used. Although newer datasets such as EME33 [144] exist, MazurkaBL remains the largest publicly available resource for studying notated dynamics against performed loudness from audio. Its provision of score-aligned beat times and verified expressive markings makes it particularly suitable for the task of beat-synchronous score-dynamics transcription. The choice to train on MazurkaBL is therefore deliberate: among available solo-piano corpora, it provides the most complete supervision for the joint target studied in this chapter, rather than only note-level velocity or metrical annotations.

For cross-dataset evaluation, we additionally use the Vienna4x22 Piano Corpus [120] and Batik-plays-Mozart [121]. Both external corpora are used only as test sets. Vienna4x22 contains repeated performances of four classical piano excerpts, whereas Batik-plays-Mozart contains movement-level Mozart piano performances with matched symbolic information. They are converted into the same HDF5-style schema as MazurkaBL, including waveform, beat, downbeat, measure, change-point, and dynamic-label fields. The conversion uses the available MusicXML scores and score–performance match files, so that all three corpora can be evaluated with the same post-processing and metric code. This design keeps the cross-dataset experiment focused on corpus transfer rather than on differences in file format or evaluation implementation.

Table 5.1 summarises the five-level dynamic-label distributions used by the evaluation pipeline. It is included because the three corpora share the same score-dynamics vocabulary but differ markedly in their label priors. MazurkaBL has a large *p* class and a substantial loud region; Vienna4x22 is strongly concentrated in *pp*, *p*, and *mf*; Batik-plays-Mozart is concentrated mainly around *p* and *f*. A classifier trained only on MazurkaBL therefore learns not only an acoustics-to-symbol mapping, but also the prior probability of each symbolic label under the Chopin-mazurka annotation convention.

We use 5-fold cross-validation stratified at the mazurka-opus level. This split is deliberately piece-aware: performances of the same mazurka do not appear in both training and test folds. Given the limited size of the corpus, this provides a data-efficient way to test generalisation to unseen pieces within MazurkaBL, while avoiding the overly optimistic estimates that can arise when different performances of the same work leak across splits. For each fold, the model is trained for 120 epochs, and the best checkpoint is selected on the validation set using F1 score. Optimisation uses AdamW with learning rate 3×10^{-4}, batch size 10, and fixed random seed 86.

Table 5.1: Beat-level distribution of five-level dynamic labels in the MazurkaBL training set, Vienna4x22, and Batik-plays-Mozart corpora. All three corpora share the same five score-dynamics labels, but their label priors differ substantially.

Label / statistic	MazurkaBL train	Vienna4x22	Batik-plays-Mozart
pp	7.0%	25.5%	0.8%
p	54.4%	44.9%	52.9%
mf	5.7%	27.0%	0.8%
f	29.9%	2.3%	45.5%
ff	3.1%	0.3%	0.1%
# beats	685,209	13,618	23,662
# change events	19,033	396	2,327

During training, audio is sliced into 60-second segments with 50% overlap. No overlap is used at evaluation time. Both BSSL and log-Mel features are extracted with the same STFT configuration to ensure identical temporal resolution. The empirically optimal temporal scaling factor is $s = 5$. The model predicts six dynamic classes: one *blank* class and the five annotated dynamic levels *pp*, *p*, *mf*, *f*, and *ff*. On an NVIDIA RTX 3090 (24 GiB), a full 5-fold run requires approximately 20 hours, with peak GPU memory usage around 4 GiB.

Beyond the in-domain MazurkaBL evaluation, Section 5.7.3 reports a test-only cross-dataset evaluation. For each architecture and front-end variant, the five MazurkaBL fold models are all evaluated on Vienna4x22 and Batik-plays-Mozart without fine-tuning. For the multitask variants, the external table selects the fold with the best dynamic F1 on the target corpus and reports the other task scores from the same checkpoint. For the single-task variants, each task has its own fold models, so the best fold is selected separately for the corresponding task. This protocol is intended as a reproducible transfer diagnostic: it asks whether a MazurkaBL-trained fold checkpoint can be reused on an external classical-piano corpus, while the main MazurkaBL table remains the within-corpus benchmark reported as mean and standard deviation over all folds. The per-fold outputs are retained so that the selected fold can be traced.

5.6.2 Baselines

We compare the proposed model against both single-task baselines and task-specific baselines. These baselines are intentionally heterogeneous: some address only dynamics or change points, while others address only beats and downbeats. Accordingly, the dashes in Table 5.2 indicate tasks that a baseline does not support, rather than experiments that were omitted.

Single-task multi-scale network. We establish single-task baselines by adapting the multi-scale network from [102], integrating its public implementation into our data pipeline, and training one independent model per target. Each single-task model keeps the same feature extraction, audio segmentation, encoder depth, optimiser, and post-processing settings as the proposed system, and differs only in that it solves one target at a time instead of learning all four jointly. Each model uses the corresponding task-specific loss from the multi-task objective, with

scaling factor $s = 5$. The parameter count reported in Table 5.2 is therefore for one single-task model instance; solving the four tasks independently would require four such models.

Dynamic and change-point baselines. For dynamics and change points, we report published results from representative methods in [97], namely the Artificial Neural Network (ANN) for dynamics and the Pruned Exact Linear Time (PELT) algorithm for change-point detection. These baselines address only their own target and are not trained jointly with metrical estimation. They are also tuning-intensive and not end-to-end, so a faithful re-implementation would introduce many secondary design choices unrelated to the main contribution of this chapter. We therefore report their published scores directly, as in the ICASSP paper.

Beat and downbeat baselines. For beats and downbeats, we include the time-convolutional network with dynamic Bayesian network post-processing [99] and the recent SOTA transformer model Beat This [101]. Both are joint beat/downbeat systems and therefore populate only the beat-related columns in Table 5.2. We retrain them from scratch on MazurkaBL using their public implementations and the same 5-fold protocol as ours. To avoid turning these baselines into modified systems, we preserve their original log-Mel front-end and task formulation.

Finally, the rows marked *w/o. BSSL* in Table 5.2 are not additional baselines from the literature. They are feature-control variants of our own architectures in which the 22-band BSSL input is replaced with a 128-bin log-Mel input under otherwise matched settings. This isolates the effect of the front-end from the effect of the model architecture.

5.6.3 Evaluation metrics

All four tasks are evaluated using F1 score. In its standard form,

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \tag{5.8}$$

$$R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \tag{5.9}$$

$$F_1 = \frac{2PR}{P + R}. \tag{5.10}$$

Here, TP, FP, and FN denote true positives, false positives, and false negatives, respectively.

For beat and downbeat tracking, the evaluation is event-based rather than frame-based. In other words, beats and downbeats are treated as binary time-stamped events, not as continuous labelled regions. We therefore count matches by one-to-one event alignment: a predicted beat or downbeat is a true positive only if it can be matched to an unmatched reference event within a ± 70 ms window; otherwise it is a false positive, and any unmatched reference event becomes a false negative. We keep this ± 70 ms tolerance because it is the standard F1 setting used by recent beat-tracking work and by `mir_eval`, which makes the present results directly comparable with the literature [99, 101, 141]. It should therefore be read as an evaluation convention rather than a musically fundamental constant.

Dynamics and change points follow the evaluation protocol of [97]. For dynamics, the frame-wise output of the dynamic head is sampled at each ground-truth beat location and then discretised into score labels. This turns the problem into beat-wise multi-class classification on the same metrical grid as the annotations. Let $\mathcal{C} = \{pp, p, mf, f, ff\}$ denote the five musically meaningful classes, excluding the *blank* class. We compute class-wise F1 as

$$F_{1,c} = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad c \in \mathcal{C}, \tag{5.11}$$

and report the macro-average

$$F_1^{\mathrm{Dyn}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} F_{1,c}. \tag{5.12}$$

The *blank* class is excluded because it only covers the region before the first annotated marking and is not itself a score-dynamics symbol.

For change points, the predicted times are first snapped to the nearest ground-truth beat before evaluation. The task then reduces to binary classification on beat indices: a predicted change point is counted as correct only if its snapped beat index exactly matches a reference change-point index. Since both dynamics and change points are evaluated on the same beat grid as the annotations, no additional timing tolerance is required for these two tasks.

5.7 Results, Case Study, and Discussion

5.7.1 Main results

Table 5.2 shows that the proposed multi-task framework gives the strongest overall results on MazurkaBL. With BSSL input, it achieves the best scores for dynamics, change points, and beats, while the log-Mel variant of the same framework achieves the best downbeat F1. The BSSL configuration is therefore the best overall trade-off, and it is the configuration adopted as the main system in this chapter.

The multi-task formulation is consistently better than the corresponding single-task counterpart under the same BSSL front-end. Relative to the single-task BSSL model, the proposed multi-task model improves dynamic F1 by 3.8 points, change-point F1 by 5.1 points, beat F1 by 0.1 points, and downbeat F1 by 10.2 points. These gains indicate that joint learning is useful rather than harmful in this setting, provided that the shared representation is combined with task-aware decoding.

The feature comparison also reveals an important trade-off. BSSL is better for dynamics, change points, and beats, whereas log-Mel remains slightly better for downbeats. The practical advantage of BSSL lies in its compactness: the input dimensionality drops from 128 mel bins to 22 Bark bands, reducing the model size from 14.7 M parameters to 0.5 M in the multi-task setting. This drastic reduction enables much longer input sequences and makes long-context training feasible on modest hardware.

This result should not be read as a dimensionality effect alone. In the terms of Chapter 2,

Table 5.2: Performance comparison of the proposed model against all baselines on the MazurkaBL dataset. Per-task F1 scores (%) are reported as mean ± standard deviations over 5-fold cross-validation. PELT is an algorithmic method with no trainable parameters, while ANN did not report this attribute. The single-task parameter count is reported per model instance. The best score is highlighted in bold, and the second-best is underlined.

Method	Feature	MazurkaBL dataset				
		Dyn. F1	CPt. F1	Beat F1	Dbt. F1	Params
ANN [97]	BSSL	29.4	–	–	–	n/a
PELT [97]	BSSL	–	10.8	–	–	n/a
TCN+DBN [99]	log-Mel	–	–	60.9 ± 1.8	30.4 ± 1.3	0.1 M
Beat This [101]	log-Mel	–	–	80.5 ± 2.7	52.8 ± 6.2	20.3 M
Single-task Multiscale	BSSL	50.6 ± 10.1	21.0 ± 9.9	<u>84.0 ± 1.5</u>	45.0 ± 1.7	0.4 M
w/o. BSSL	log-Mel	50.4 ± 11.1	17.5 ± 5.4	83.8 ± 1.8	54.7 ± 7.5	13.3 M
Multitask Multiscale	BSSL	**54.4 ± 8.9**	**26.1 ± 9.7**	**84.1 ± 1.3**	<u>55.2 ± 4.2</u>	0.5 M
w/o. BSSL	log-Mel	<u>50.8 ± 10.9</u>	<u>23.1 ± 6.1</u>	83.7 ± 1.7	**58.5 ± 6.2**	14.7 M

BSSL preserves the Bark-wise distribution of specific loudness after auditory weighting and masking, whereas a scalar loudness curve would discard that structure and a conventional log-Mel spectrogram does not encode it explicitly. The gains in dynamic and change-point prediction are therefore consistent with the thesis-level claim that score-dynamics recovery benefits from a representation closer to perceived loudness than to raw spectral magnitude.

5.7.2 Ablation study

To validate the design choices of the proposed model, we conduct an extended ablation study against the full BSSL configuration. The full model uses the MMoE decoder, temporal scaling with $s = 5$, data augmentation, and 60-second audio segments. Table 5.3 evaluates the architecture across three dimensions: the removal of key model components, the variation of the temporal scaling factor s, and the adjustment of the input audio segment length. This organisation is deliberate because it directly reflects our core modelling hypothesis. Since macro-structural dynamics and metrical beats/downbeats inherently prefer conflicting temporal contexts, temporal scaling and audio segment length act as the primary mechanisms to resolve this tension, making their analysis critical.

The component ablations show that all major parts of the model contribute to the final performance, but not in the same way. The most diagnostic result is the removal of temporal scaling. Setting $s = 1$ reduces the average F1 from 55.0 to 46.5, with particularly large drops in change-point F1 and downbeat F1. This indicates that the performance gain does not merely come from using a deeper convolutional encoder, but from exposing the downstream tasks to genuinely different temporal resolutions. Removing MMoE also degrades all four tasks, supporting the view that the four heads have different preference on the same shared latent. The overlap-based data augmentation gives a further improvement, which is expected for this

Table 5.3: Ablation study of the multitask multiscale network with BSSL as input. Per-task F1 scores (%, mean over 5 folds) and their arithmetic average are reported. The full model uses MMoE, temporal scaling $s = 5$, 60-second audio segments, and data augmentation. The best score is highlighted in bold, and the second-best is underlined.

Setting	MazurkaBL dataset				
	Dyn. F1	CPt. F1	Beat F1	Dbt. F1	Average
Multitask Multiscale Network (s=5, 60s)	<u>54.4</u>	<u>26.1</u>	**84.1**	**55.2**	**55.0**
w/o. MMoE	52.8	22.0	82.9	51.8	52.4
w/o. Data Augmentation	50.5	19.6	83.2	51.7	51.2
w/o. Temporal Scaling (s=1)	50.5	13.3	80.3	41.9	46.5
uses Temporal Scaling s=3	51.1	<u>26.1</u>	**84.1**	<u>54.3</u>	53.9
uses Temporal Scaling s=7	54.0	25.8	<u>83.8</u>	53.8	<u>54.4</u>
uses Temporal Scaling s=9	52.7	25.4	83.2	50.3	52.9
uses 120s Audio Segment	52.3	25.9	81.1	46.8	51.5
uses 90s Audio Segment	**56.8**	**27.8**	82.9	46.6	54.2
uses 30s Audio Segment (as in [101])	49.1	19.2	83.4	52.7	51.1

dataset because change points and downbeats are sparse events. Overlapping audio segments increase the chance that such events appear in varied local contexts during training.

The temporal-scaling results give a detailed view of the multi-scale encoder. The reference architecture of [102] uses a scaling factor of 3, but this value is not optimal for our tasks combination. With $s = 3$, beat F1 and change-point F1 remain strong, but the average score is lower than the full model. Increasing the factor to $s = 5$ gives the best overall balance. A slightly larger value, $s = 7$, remains competitive and gives the second-best average score, but further increasing the factor to $s = 9$ causes a clear degradation, especially for downbeats. This pattern suggests that the model needs a sufficiently coarse branch to capture long-range dynamic context, but excessive temporal compression begins to remove information that cannot be fully recovered for event localisation. The best value is therefore not simply the largest receptive field, but the best compromise between contextual abstraction and temporal precision.

The segment-length results further support this interpretation. The 30-second setting is included because it is a common choice in beat/downbeat tracking systems, including Beat This [101]. It performs reasonably well for beat and downbeat tracking, but is clearly weaker for dynamics and change points. Moving from 30 seconds to 60 seconds improves dynamic F1 from 49.1 to 54.4 and change-point F1 from 19.2 to 26.1, confirming that dynamics-related targets benefit from musical context beyond the short excerpts typically used for metrical tracking. However, the trend is not monotonic. The 90-second setting achieves the best dynamic and change-point F1 scores, but its downbeat F1 drops sharply, reducing the overall average. The 120-second setting is worse than 60 seconds on all four tasks. These results show that longer audio context is helpful for dynamics, but only up to the point where the joint model can still maintain stable event localisation. We therefore use 60-second segments as the default setting, not because it maximises every individual task, but because it gives the best multi-task trade-off

across dynamics, change points, beats, and downbeats.

5.7.3 Cross-dataset evaluation

The 5-fold MazurkaBL evaluation already gives a partial test of generalisation, because the folds are piece-disjoint and the model is always evaluated on unseen mazurkas within the same corpus. However, it remains an in-domain setting: performer pool, recording conditions, repertoire family, and annotation conventions are all inherited from MazurkaBL. We therefore add a stricter cross-dataset experiment on Vienna4x22 and Batik-plays-Mozart, using them as test-only corpora after the MazurkaBL training stage.

For each method and input feature, the five checkpoints obtained from MazurkaBL cross-validation are evaluated on each external corpus without fine-tuning. For multitask systems, the transferred fold is selected by the five-class dynamic F1 score because the dynamic label is the primary output of this chapter; the change-point, beat, and downbeat scores are then reported from the same fold. For single-task systems, each task is trained independently, so the best transferred fold is selected separately for the corresponding task. This mirrors the structure of the in-domain experiments, where the single-task baselines are independent models while the multitask system is a single shared model.

Table 5.4: Cross-dataset evaluation on Vienna4x22 and Batik-plays-Mozart. Models are trained on MazurkaBL and applied to both external corpora without fine-tuning. Per-task F1 scores (%) report the best transferred MazurkaBL fold under the protocol described above. The best score is highlighted in bold, and the second-best is underlined.

Method	Feature	Vienna4x22				Batik-plays-Mozart			
		Dyn. F1	CPt. F1	Beat F1	Dbt. F1	Dyn. F1	CPt. F1	Beat F1	Dbt. F1
TCN+DBN [99]	log-Mel	–	–	58.4	22.9	–	–	23.1	12.1
Beat This [101]	log-Mel	–	–	72.6	**33.2**	–	–	29.5	12.0
Single-task Multiscale	BSSL	<u>32.6</u>	26.6	<u>74.2</u>	**33.2**	<u>53.9</u>	**20.2**	**40.0**	**16.7**
w/o. BSSL	log-Mel	31.5	<u>34.7</u>	74.0	21.5	**54.3**	14.1	38.6	13.3
Multitask Multiscale	BSSL	32.0	29.8	73.8	<u>29.3</u>	53.0	<u>14.3</u>	<u>39.2</u>	<u>15.1</u>
w/o. BSSL	log-Mel	**38.2**	**36.7**	**74.5**	27.3	50.2	<u>14.3</u>	37.8	13.9

Table 5.4 shows that cross-dataset transfer is substantially harder than the in-domain MazurkaBL evaluation, but the failure pattern is not uniform across tasks or corpora. On Vienna4x22, the best dynamic F1 is 38.2%, far below the 54.4% in-domain dynamic F1 of the proposed BSSL system. At the same time, beat F1 remains around 74%, which indicates that the models have not simply lost the ability to locate the metrical grid. The more difficult part is the transfer of five-level symbolic dynamic labels under a different corpus convention.

The label-prior table explains why Vienna4x22 is especially challenging. In the MazurkaBL training data, the loud region *f*/*ff* accounts for 33.0% of beat-level labels. In Vienna4x22, the same region accounts for only 2.6%, while *pp*, *p*, and *mf* together account for 97.4%. A model trained on MazurkaBL therefore enters Vienna4x22 with a prior in which loud labels are normal

events, even though they are almost absent in the target corpus. This is harsh under five-class macro-F1, because the metric gives each symbolic class its own contribution and does not know that neighbouring dynamic markings may be editorially close in some passages. The Vienna result should therefore be read as a combined effect of acoustic transfer, repertoire transfer, and score-annotation prior shift, not simply as a failure of the audio front-end.

Batik-plays-Mozart behaves differently. Its label distribution is concentrated in *p* and *f*, which are also the two dominant labels in MazurkaBL. As a result, the dynamic F1 remains near the in-domain range for several variants, with the single-task log-Mel model reaching 54.3%. However, the change-point, beat, and downbeat scores are lower on Batik-plays-Mozart than on Vienna4x22. The change-point behaviour is consistent with the distributional table: Batik-plays-Mozart has 2,327 change events over 23,662 beats, whereas MazurkaBL has 19,033 over 685,209 beats. The dynamic vocabulary is therefore familiar, but the rate and placement of dynamic transitions are much less Mazurka-like. More broadly, this suggests that dynamic-label transfer and metrical-structure transfer fail for different reasons: Batik is favourable for the dynamic vocabulary but less favourable for the change-point and beat/downbeat heads, whereas Vienna preserves beat-level transfer more strongly but exposes a severe dynamic-prior mismatch.

The feature comparison in the external tests should also be interpreted cautiously. BSSL remains the best in-domain choice because it is compact, perceptually grounded, and strong for the dynamics-related tasks on MazurkaBL. In the Vienna4x22 cross-dataset setting, however, the log-Mel multitask variant obtains the best scores for dynamics, change points, and beats. This does not contradict the in-domain result; rather, it shows that front-end quality alone cannot compensate for target-prior shift. A perceptually aligned feature helps the model read loudness structure, but the final symbolic label still depends on the distribution and editorial convention of the corpus being evaluated.

5.7.4 Discussion: why score dynamics is not a relabelled MIDI velocity problem

The objective in this chapter is not note-level control of a synthesiser but a score-facing summary of expressive intensity on a musically meaningful coordinate. Score dynamics is therefore not obtained here by simply discretising an underlying velocity curve. It is learned as a symbolic description of dynamic regimes together with the metrical structure needed to place that description on a score.

This distinction explains several design choices. The model uses longer context because phrase-level dynamic intent extends beyond local note attacks. The post-processing snaps change points to beats because the available annotations are editorial and metrical rather than freely continuous. The evaluation is beat-synchronous because a musically meaningful score label should be judged on the score grid, not only on raw frame precision.

The cross-dataset results make this distinction more concrete. If score dynamics were merely a continuous loudness variable relabelled into five fixed bins, an out-of-domain corpus with compatible audio quality would mainly cause smooth calibration errors. Instead, Vienna4x22

shows that the symbolic prior itself matters: a passage that is moderate in acoustic intensity may be notated as *p*, *mf*, or *f* depending on the local phrase, edition, and corpus convention. A model trained on MazurkaBL carries a learned expectation about how often each of these labels should occur, and this expectation can become wrong even when the metrical grid remains reasonably stable. The model is therefore learning an acoustic, metrical, and editorial mapping at the same time.

Note-level velocity and score dynamics serve different purposes. Note-level velocity is the more precise target when the goal is editing or rendering, whereas beat-synchronous score dynamics is the more interpretable target when the goal is symbolic annotation, educational feedback, or performance analysis.

5.7.5 Limitations

Several limitations should be kept in mind. First, the dynamic vocabulary is restricted to the five static score levels present in MazurkaBL plus a *blank* class. The model does not explicitly represent longer-span symbols such as *crescendo* and *diminuendo*; instead, it captures local transitions through change points. Second, the beat-aligned post-processing reflects the annotation style of MazurkaBL. This is musically sensible for this dataset, but it may not transfer directly to corpora with different editorial conventions.

Third, although the chapter now includes Vienna4x22 and Batik-plays-Mozart, the empirical scope is still solo piano. The two external corpora test distribution shift within classical piano, not immediate generalisation to other instruments or mixed ensembles. They are also much smaller than MazurkaBL, so their results should be interpreted as transfer diagnostics rather than as a complete benchmark of score-dynamics transcription.

A related future direction is to connect the present multi-task framework with the broader training regime used in modern beat and downbeat tracking. Systems such as Beat This [101] are designed for general beat/downbeat estimation rather than for score-dynamics transcription, and the broader beat-tracking corpora available for such systems may strengthen the metrical heads of a transformer-based or multi-scale model. However, those corpora generally do not provide compatible beat-synchronous score-dynamics labels or change-point annotations. Adding them would therefore change the problem into a partially supervised multi-task setting, where some datasets supervise only beats/downbeats while MazurkaBL-like corpora supervise the full dynamics structure. This is a promising extension, but it is separate from the fully supervised score-dynamics experiment completed in this chapter.

Finally, the feature comparison suggests that no single front-end is uniformly optimal across all tasks or corpora. BSSL is the better global choice for the present chapter because it gives the strongest in-domain balance and a much smaller model, but downbeat tracking and some external transfer cases still benefit from log-Mel input. Looking forward, the Rach3 dataset [145] provides rehearsal-style annotations across a broader repertoire and is currently still under construction. Once at usable scale, Rach3 would offer a complementary validation target and an additional source for continued training or calibration of the framework presented here.

5.8 Summary

This chapter changed the target representation from note-level MIDI velocity to score-level dynamics. The aim is no longer to control a synthesiser note by note, but to recover a beat-synchronous symbolic description that can be written back to a score. This distinction motivates the whole design: BSSL is used because dynamics are closer to perceived loudness than to raw spectral magnitude; the multi-scale encoder provides both phrase-level context and frame-level timing; and the four heads jointly estimate dynamic levels, change points, beats, and downbeats because the dynamic label is only meaningful once it is placed on a metrical coordinate.

The results should be read in that representational frame. On MazurkaBL, the compact BSSL-based multi-task model provides a strong in-domain benchmark with far fewer parameters than the log-Mel variant. The external Vienna4x22 and Batik-plays-Mozart tests then show the harder limitation: score-dynamics transfer is not only acoustic transfer. It also depends on label priors and editorial conventions, so dynamic-label transfer and metrical-structure transfer can fail in different ways. This chapter therefore answers the thesis requirement for a score-facing coordinate, while also clarifying why score dynamics is not simply MIDI velocity binned into five loudness classes.

Chapter 6

Beyond Piano: Cross-Instrument MIDI Velocity Estimation via Differentiable SoundFont Proxies

6.1 Chapter Overview and Position in the Thesis

This chapter extends the thesis's velocity-recovery work beyond piano. The two earlier velocity chapters established complementary methods within the supervised piano regime: Chapter 3 addressed Scenario S1 by completing missing MIDI velocities from symbolic structure alone, and Chapter 4 addressed Scenario S2 by refining audio-derived velocities using an aligned score. Both relied on the same supervisory anchor: MAESTRO [5], in which note-aligned velocity labels are available because the performances were captured on sensor-equipped Yamaha Disklavier instruments. Outside piano, this anchor disappears. Public datasets for guitar, violin, and other acoustic instruments [20, 123, 124] provide audio and aligned notes but no note-level velocity labels. Supervised training and label-based domain transfer are therefore both unavailable in the target domain.

The central question of this chapter is therefore: *how can note-level MIDI velocities be recovered from real performance audio for instruments where ground-truth velocity labels are not accessible?* The answer pursued here is analysis-by-synthesis: predict velocities whose rendered dynamics are consistent with the observed performance audio, and drive the prediction with gradients passed through a differentiable interface to a renderer.

This framing leads to two candidate supervision interfaces. The first is a differentiable waveform synthesiser (Diff-Synth) that renders predicted velocities into audio and matches them against the real recording in waveform or spectral space. The second is a differentiable SoundFont proxy (Diff-SFProxy) that maps predicted velocities to note-wise loudness-related acoustic parameters and matches those parameters against measurements of the real audio. Both interfaces share the same front-end estimator and the same regularisers, so the comparison isolates the effect of the supervision signal itself.

From the thesis perspective, this chapter occupies a distinct position. It keeps the note-level

MIDI velocity target and the score-informed VeloEst architecture introduced in Chapter 4, but it relaxes the assumption that ground-truth velocity labels are available for the target instrument. It complements Chapter 5, which addressed Scenario S3 with a different output (beat-synchronous score dynamics), by extending the velocity-focused work to a label-scarce setting. The chapter therefore contributes to thesis Requirement R5 by combining numerical diagnostics, perceptual loudness correlations, and renderer-robust evaluation evidence, and to Requirement R6 by treating beyond-piano transfer as a missing-supervision problem rather than ordinary acoustic domain shift.

The chapter is delimited in two respects. First, label-scarce cross-instrument velocity estimation has few direct baselines that target the same setting. Second, renderer-based supervision introduces a renderer-conditioned interpretation of velocity that does not coincide with a single physical measurement of the performer's gesture. The chapter is therefore framed as a practical solution under explicit assumptions, rather than as a claim about universal real-performance fidelity.

6.2 Task-Specific Related Work and Motivation

MIDI velocity is the per-note control value that most symbolic music systems use to represent how strongly a note is played. In synthesisers, velocity explicitly controls behaviour such as amplitude, sample selection, and filtering. In acoustic instruments, velocity has a clear physical effect on perceptual loudness but is rarely captured directly, except on sensor-equipped pianos. Reliable velocity labels are crucial for expressive rendering [135, 146], music generation [136, 147], performance analysis [148], and data curation [117].

Most publicly available MIDI files lack velocity driven by human performance. A common practice in music software is a rule-based mapping from score-level dynamic markings such as p and f (where p stands for *piano*, soft, and f stands for *forte*, loud) to fixed velocity values, or defaulting to 64 if no marking is present [7]. Such velocities carry minimal expressive information.

Velocity estimation has so far been almost exclusively a piano problem. Learning-based velocity estimation from performance audio recovers expressive labels well, but almost entirely in the piano domain [23, 94, 95]. The reason is data: piano datasets such as MAESTRO [5] provide note-aligned velocity labels sourced from Yamaha Disklavier performances. For guitar, violin, and many other instruments, public datasets [20, 123, 124] provide audio and aligned notes but no velocity labels. This precludes both supervised training and label-based domain transfer. The asymmetry between piano and other instruments is therefore not primarily a modelling gap but a labelling gap, and any method that aims to address the gap must operate without ground-truth note-level velocity supervision in the target domain.

Synthetic-data transfer for transcription. A common response to label scarcity in transcription is to train on synthesised audio, where symbolic labels are available by construction,

and then transfer the model to real performances [86, 116, 149]. Some recent works follow this route specifically to obtain synthetic velocity supervision [150, 151].

This chapter takes a different and simpler route. Instead of training on synthesised audio with synthetic labels, the front-end estimator is trained directly against real performance audio. The synthesiser is involved at training time only as a differentiable interface that connects predicted velocities to a measurable property of the real recording. This avoids the synthesis-to-performance domain gap that dominates synthetic-data transfer pipelines, at the cost of needing a differentiable supervision interface that is reliable across the instruments of interest.

Differentiable waveform synthesisers. Differentiable digital signal processing provides a flexible route for embedding synthesis inside end-to-end learning [106]. The same idea has been extended to instrument-specific waveform synthesisers, including DDSP-Piano [109] and DDSP-Guitar-Synth [111], both of which accept MIDI-oriented inputs that include per-note velocity. Such models can render predicted velocities into audio and propagate gradients from a waveform-domain loss back into a velocity estimator. The DDSP family also includes integrated systems with detailed expressive control [108].

Two limitations of waveform-level supervision are relevant here. First, a Diff-Synth only approximates the real instrument; its waveform statistics differ from real recordings in timbre, articulation, microphone response, and room acoustics. A waveform or spectral loss therefore spends much of its gradient budget on residual mismatch rather than on velocity-dependent behaviour. Second, an instrument-specific Diff-Synth requires a dedicated design: piano-oriented synthesisers expect piano-specific controls such as pedal state, while guitar-oriented synthesisers expect string-wise trajectories. Reusing one Diff-Synth across instruments is therefore impractical.

Neural proxies for black-box audio systems. Because standard SoundFont rendering is non-differentiable, gradients cannot pass through it directly, which motivates a neural proxy. Martínez Ramírez *et al.* [112] demonstrated differentiable signal processing with black-box audio effects. Barkan *et al.* [152] introduced InverSynth II, which learns a differentiable synthesiser proxy for sound matching and combines it with self-supervised inference-time finetuning. Combes *et al.* [153] studied neural proxies that map presets to perceptually informed audio embeddings for black-box software synthesisers. These works show that surrogate networks can bridge a non-differentiable renderer and gradient-based optimisation.

The proxy used in this chapter follows the same general strategy but is adapted to the velocity-estimation task. Unlike the audio-embedding proxies of prior work, the proxy here does not aim to reproduce waveforms or perceptual embeddings of complete sounds. Instead, it predicts a small set of note-wise loudness-related acoustic parameters that are extracted by the same procedure from both proxy outputs and real recordings. The two design benefits are clear. First, many SoundFonts can plug in off the shelf, whereas Diff-Synth requires a dedicated design per instrument. Second, gradients focus on velocity-relevant intensity and attack cues rather than on waveform details that are dominated by recording and timbral mismatch.

6.3 Problem Setting

Let the source domain be a labelled piano corpus

$$\mathcal{D}_{\mathrm{s}} = \big\{(a_j, E_j, V_j)\big\}_{j=1}^{M_{\mathrm{s}}}, \tag{6.1}$$

where a_j is the performance audio, $E_j = \{(p_n, t_n, \ell_n)\}_{n=1}^{N_j}$ is the aligned note-event list with pitch p_n, onset t_n, and duration ℓ_n, and $V_j = (v_{j,1}, \ldots, v_{j,N_j})$ is the corresponding ground-truth note-level MIDI velocity vector. Let the target domain be a label-scarce corpus

$$\mathcal{D}_{\mathrm{t}} = \big\{(a_j, E_j)\big\}_{j=1}^{M_{\mathrm{t}}}, \tag{6.2}$$

where aligned note events are available but per-note velocity labels are not accessible. The goal is to estimate a velocity vector $\hat{V}_j \in [0,1]^{N_j}$ for each target recording such that the rendered dynamics of $(E_j, \hat{V}_j)$ are consistent with the real audio a_j.

Why velocity in this setting is renderer-conditioned. For piano with captured MIDI, note velocity is directly observed at performance time. For many non-piano acoustic recordings, however, no canonical ground-truth MIDI velocity is attached to the performance. The reason is physical. A guitarist's pluck or a violinist's bowing involves richer continuous control than a single onset scalar can describe, including pluck force and angle, bow speed and pressure, contact point, and articulation. A one-dimensional MIDI velocity is therefore not a universal physical truth for these instruments.

This chapter adopts an operational definition that is consistent with the analysis-by-synthesis framing. For a target instrument, let R denote a chosen renderer that exposes a per-note velocity control. The desired velocity vector is the one that, when rendered through R, best reproduces the perceptual dynamics of the original performance audio under a chosen comparison space:

$$V^\star = \arg\min_{V \in [0,1]^N} d\big(\,\mathcal{F}\big(R(E,V)\big),\ \mathcal{F}(a)\,\big), \tag{6.3}$$

where $\mathcal{F}$ is a feature-extraction operator and d is a distance in that feature space. This definition is renderer-relative. It does not claim that $V^\star$ recovers the performer's physical gesture. It claims that $V^\star$ is a useful note-wise dynamics coordinate for analysis, resynthesis, and downstream symbolic processing under the chosen synthesis chain.

This view resolves a common ambiguity. The goal of this chapter is not to infer an impossible hidden sensor measurement from historical audio. The goal is to recover a compact note-wise dynamic control that is faithful to the observed performance in a task-relevant space.

What makes the cross-instrument transfer hard. Three concrete difficulties shape the design. First, the label regime changes between piano and other instruments. Source-domain supervision is strong, while target-domain supervision is weak or indirect. Second, the meaning of velocity changes across instruments: in piano, onset energy and hammer-string

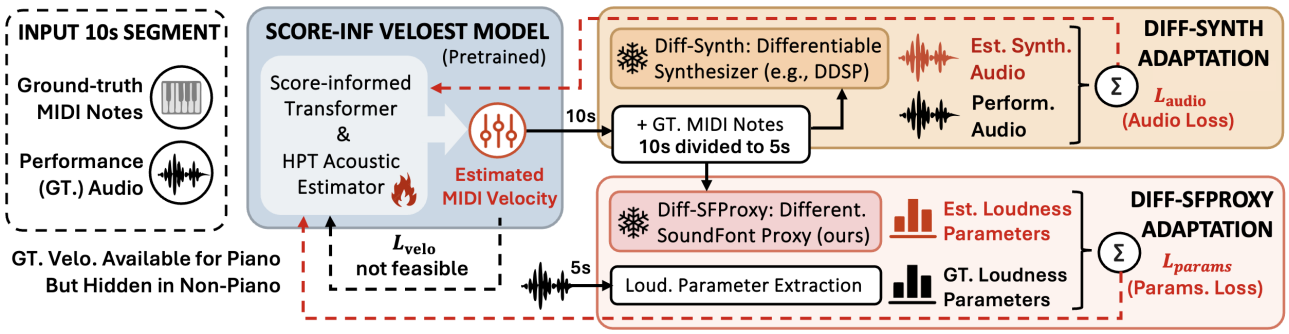


Figure 6.1: Overview of the cross-instrument velocity estimation framework. VeloEst is trained with ground-truth velocity labels on piano ($\mathcal{L}_{\text{velo}}$) and then fine-tuned for target instruments through one of two adaptation strategies. Diff-Synth provides waveform-domain supervision ($\mathcal{L}_{\text{audio}}$). Diff-SFProxy provides note-wise loudness-parameter supervision ($\mathcal{L}_{\text{params}}$).

interaction are tightly linked to velocity, whereas in guitar pluck style, string resonance, and finger-string coupling complicate the mapping. Third, waveform mismatch is severe even when score alignment is reliable, because real recordings and synthetic renders differ in timbre, room response, microphone placement, post-processing, and noise floor. A waveform loss can be dominated by these nuisance factors instead of by velocity-relevant structure.

6.4 Cross-Instrument Velocity Estimation Framework

Figure 6.1 summarises the framework. VeloEst is first trained in the piano domain using ground-truth velocity labels. The piano-pretrained checkpoint is then adapted to the target instrument through one of two differentiable interfaces. The Diff-Synth interface renders predicted velocities into waveforms and matches them against the real audio with a multi-scale spectral loss. The Diff-SFProxy interface maps predicted velocities to note-wise loudness-related acoustic parameters and matches them against parameters extracted from the real audio. During adaptation, only VeloEst parameters are updated; both back-ends remain frozen.

The two interfaces share a common training objective. They differ only in the main loss term, in the back-end being frozen, and in the output space they compare against. This shared structure is deliberate. It allows the comparison between waveform-level and parameter-level supervision to be attributed to the supervision interface rather than to incidental differences in optimisation or regularisation.

6.4.1 VeloEst: Shared Front-End Estimator

VeloEst acts as the shared front-end of the framework, estimating note-wise MIDI velocities $\hat{V}$ from the performance audio and aligned note events. We use the score-informed HPT model with onset and frame as score-guidance, which is identified as the best VeloEst configuration in Chapter 4 [154]. We keep its architecture unchanged and use the publicly available checkpoint trained on MAESTRO with ground-truth velocity supervision $\mathcal{L}_{\text{velo}}$. This fixed estimator keeps the cross-instrument experiment focused on the supervision interface rather than on introducing

a new velocity-estimation architecture.

The estimator has two stages. The acoustic module, borrowed from the High-Resolution Piano Transcription system [4], takes log-Mel spectrograms computed at 100 fps from 22050 Hz mono audio (FFT size 2048, hop 221 samples, Hann window) as input and predicts a coarse velocity roll $V^{(0)}$. A score-informed Transformer then refines $V^{(0)}$ into $\hat{V}$ using aligned note information. The detailed architecture and supervised training procedure are given in Chapter 4; the present chapter treats this estimator as a fixed black-box front-end whose checkpoint is then adapted.

6.4.2 Adaptation via Diff-Synth

In the Diff-Synth path, the back-end is a frozen, instrument-specific differentiable waveform synthesiser. Given note events E derived from the aligned score and the predicted velocities $\hat{V}$, Diff-Synth renders audio $\tilde{a}$. We adopt DDSP-Piano [109] for piano and DDSP-Guitar-Synth [111] for guitar; both are MIDI-oriented and accept per-note velocity as a native control.

The adaptation loss is the multi-scale spectral loss of [106], which is widely adopted in the differentiable synthesiser family [108, 109, 111]. It is applied between the rendered audio $\tilde{a}$ and the real performance audio a:

$$\mathcal{L}_{\text{audio}} = \sum_{\omega \in \Omega} \Big(\big\| |\text{STFT}_\omega(\tilde{a})| - |\text{STFT}_\omega(a)| \big\|_1 + \big\| \log(|\text{STFT}_\omega(\tilde{a})| + \epsilon) - \log(|\text{STFT}_\omega(a)| + \epsilon) \big\|_1 \Big), \quad (6.4)$$

where $\Omega = \{2048, 1024, 512, 256, 128, 64\}$ as in [106]. For each $\omega \in \Omega$, STFT_ω uses an ω-sample Hann window with 75% overlap and an ω-point FFT. The logarithm is applied to $|\cdot| + \epsilon$ with $\epsilon = 10^{-7}$ for numerical stability.

The advantage of this interface is that it exposes the front end to rich waveform gradients. The risk, already noted in Section 6.2, is that any timbral or recording mismatch between $\tilde{a}$ and a also enters the gradient. When the synthesiser is not a faithful match for the target recordings, the gradient can become dominated by nuisance compensation rather than by velocity correction.

6.4.3 Adaptation via Diff-SFProxy

In the Diff-SFProxy path, the back-end is a frozen differentiable proxy P of an instrument-specific SoundFont. Let F denote the note-wise loudness-related acoustic parameter extractor that reads audio conditioned on the note events E. Both F and P output the same kind of low-dimensional, note-wise dynamics descriptor; their precise construction is deferred to Section 6.5. Given predicted velocities $\hat{V}$, the two feature streams are

$$Z = F(a, E), \qquad \tilde{Z} = P(E, \hat{V}). \quad (6.5)$$

The adaptation loss is a Huber (Smooth-L_1) distance between the two extracted parameter groups,

$$\mathcal{L}_{\text{params}} = \text{Huber}_\beta\Big(\tilde{Z} - Z\Big), \tag{6.6}$$

with the default $\beta = 1.0$. Each $Z_n = (\text{PHE}_n, \text{OSF}_n) \in \mathbb{R}^2$ contains the two note-wise parameters defined in Section 6.5.1, and the loss averages over both components and all valid notes, weighting PHE and OSF equally (1:1). The Huber form tolerates outliers from onset contamination such as overlapping notes, while the parameter-space objective concentrates gradients on note intensity and attack behaviour rather than on waveform details that are weakly related to velocity.

The asymmetry between Eq. (6.4) and Eq. (6.6) is the central design point of this chapter. The Diff-Synth path optimises in waveform/spectral space, where velocity is one of many factors influencing the signal. The Diff-SFProxy path optimises in a low-dimensional note-wise dynamics space, where velocity is the dominant factor by construction. The remainder of the chapter tests whether this difference matters in practice.

6.4.4 Shared Regularisers and Total Objective

Both adaptation paths share two auxiliary regularisers on the predicted onset-velocity distribution $\hat{\mathbf{v}} \in [0,1]^N$, where N is the number of valid onsets in a batch. The first is an anti-collapse prior,

$$\mathcal{L}_{\text{anti}}(\hat{\mathbf{v}}) = (\bar{v} - \mu)^2 + \max\big(0,\ \sigma^2_{\min} - \text{Var}(\hat{\mathbf{v}})\big), \tag{6.7}$$

where $\bar{v}$ and $\text{Var}(\hat{\mathbf{v}})$ are the empirical mean and variance over onset positions, $\mu = 0.5$ is a desired operating point, and $\sigma^2_{\min} = 0.01$ is a minimum-variance threshold. The mean term resists drift towards extremes. The hinge variance term penalises collapse towards a single output value.

The second is a soft saturation penalty,

$$\mathcal{L}_{\text{satu}}(\hat{\mathbf{v}}) = \frac{1}{N}\sum_{n=1}^{N}\big[\max(0,\ \hat{v}_n - \tau)\big]^2, \tag{6.8}$$

which quadratically penalises predictions above a near-maximum threshold $\tau = 0.95$. This discourages VeloEst from pushing mass into the saturated region, where audio-derived gradients become uninformative because further increases in velocity produce diminishing changes in the rendered output.

The total adaptation objective combines the active main loss with the two regularisers,

$$\mathcal{L} = \mathcal{L}_{\text{main}} + \lambda_{\text{anti}}\,\mathcal{L}_{\text{anti}} + \lambda_{\text{satu}}\,\mathcal{L}_{\text{satu}}, \tag{6.9}$$

where $\mathcal{L}_{\text{main}} \in \{\mathcal{L}_{\text{audio}},\ \mathcal{L}_{\text{params}}\}$ selects the active interface and the regulariser weights are zero outside adaptation. For both adaptation branches the fixed weight ratio $1 : 0.2 : 0.4$ is used, found by grid search and held constant across all experiments.

6.5 Differentiable SoundFont Proxy

This section defines the loudness-related acoustic parameters that form the proxy's output space, the proxy architecture, and the proxy's offline training procedure. The proxy is trained once for each instrument-SoundFont pair and then frozen throughout the adaptation experiments in Sections 6.7 and 6.8.

6.5.1 Loudness-Related Acoustic Parameters

To characterise the velocity-dependent acoustic behaviour of a SoundFont, two note-wise parameters are extracted from rendered or real audio. The extractor F computes both parameters from the magnitude STFT $X(k,t) = |\mathrm{STFT}(k,t)|$, using the same FFT settings as Section 6.4.1. Here k indexes frequency bins and t indexes STFT frames. For each valid note $n \in \{1,\dots,N\}$, the extractor stacks the parameters into

$$Z_n = \big(\mathrm{PHE}_n,\ \mathrm{OSF}_n\big) \in \mathbb{R}^2, \tag{6.10}$$

and stacks all note-wise pairs into $Z = (Z_1,\dots,Z_N) \in \mathbb{R}^{N\times 2}$. The two parameters are designed to capture the early, velocity-sensitive portion of each note while remaining robust to alignment slack and to mismatch between SoundFont and real recordings.

Pitch-conditioned harmonic energy (PHE). The first parameter draws on harmonic-structure-based note-intensity estimation [60]. It accumulates magnitude energy around the retained harmonics of the note pitch:

$$\mathrm{PHE}_n = \log\left(1 + \sum_{h=1}^{H_n} \frac{1}{|\mathcal{B}_{n,h}|\,|\mathcal{T}_n|} \sum_{k\in\mathcal{B}_{n,h}} \sum_{t\in\mathcal{T}_n} X(k,t)^2\right), \tag{6.11}$$

where h indexes harmonics, $\mathcal{B}_{n,h}$ is the ± 1-bin frequency band around the h-th harmonic of note n, $\mathcal{T}_n$ is the set of STFT frames within 120 ms after the note onset, and $H_n \le 5$ is the number of retained harmonics below Nyquist. The narrow band handles STFT leakage while rejecting nearby pitches. The five-harmonic cap keeps PHE on lower partials that are less exposed to inharmonicity, tuning or rendering mismatch, and overlap.

Onset-window spectral flux (OSF). The second parameter is based on spectral-flux onset functions, which are used in onset detection by capturing positive spectral differences [155, 156]. It measures the broadband, half-wave-rectified spectral increase around the note onset:

$$\mathrm{OSF}_n = \log\left(1 + \frac{1}{|\mathcal{O}_n|} \sum_{t\in\mathcal{O}_n} \sum_{k} \big[X(k,t) - X(k,t-1)\big]_+\right), \tag{6.12}$$

where $\mathcal{O}_n$ is the set of STFT frames from 20 ms before to 80 ms after the note onset, and $[u]_+ = \max(u,0)$ for any scalar u. The short pre-onset side absorbs alignment slack and provides

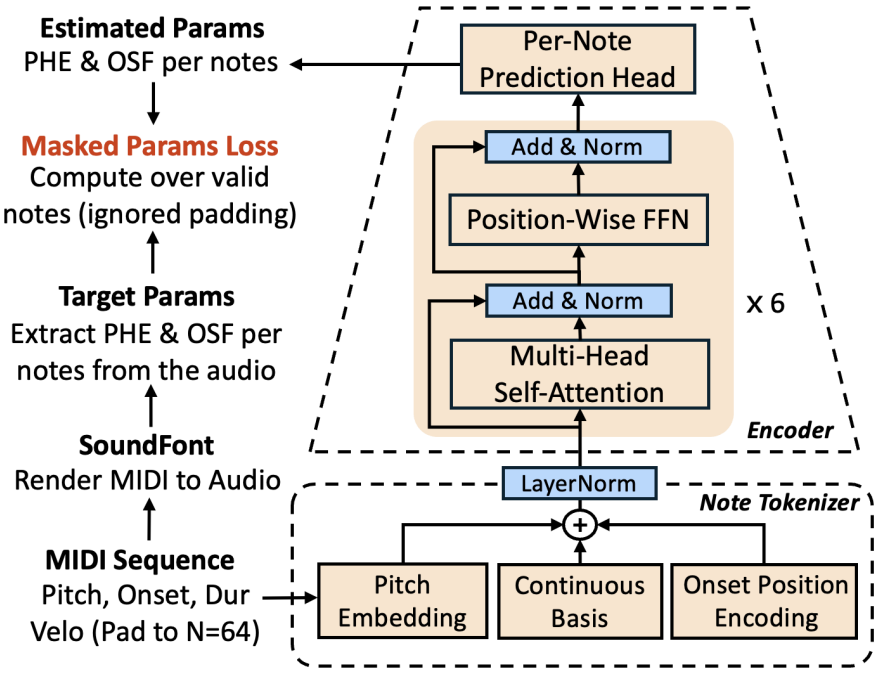


Figure 6.2: An encoder-only Transformer maps padded note sequences to per-note PHE/OSF targets extracted from SoundFont-rendered audio.

a local reference. The longer post-onset side covers the main struck or plucked-string transient without folding in sustained energy.

These two parameters are intentionally narrow. They do not aim to model full loudness perception. They focus on local note-wise cues that are strongly influenced by velocity in many synthesis chains, and they form the supervision signal that the proxy is required to predict.

6.5.2 Proxy Architecture

Figure 6.2 shows the proxy architecture. A Transformer encoder maps a padded sequence of up to $N_{\max} = 64$ note events in a 5 s segment to the corresponding $(\widetilde{\mathrm{PHE}}_n, \widetilde{\mathrm{OSF}}_n)$ pairs. The choice $N_{\max} = 64$ is motivated by a data analysis on MAESTRO showing that human performances rarely exceed 64 notes per 5 s.

Each note token combines three components. The first is a learnable pitch embedding. The second is a shared learnable basis that maps onset, duration, and velocity (each normalised to $[0, 1]$) to additive directions in token space. The third is a two-layer MLP with GELU activation on the normalised onset, acting as a learnable positional encoding. These components are summed and passed through LayerNorm to form the note token. Padded slots are then zeroed.

The encoder stacks six pre-norm Transformer layers ($d = 256$, 8 heads, FFN width $4d = 1024$, GELU, dropout 0.1), with a key-padding mask excluding padded slots from attention. A shared note-wise head ($d \to d \to 2$, with GELU) emits each $(\widetilde{\mathrm{PHE}}_n, \widetilde{\mathrm{OSF}}_n)$ pair. The encoder-only design fits this set-to-set regression task naturally, because the proxy has no autoregressive generation requirement: each note's targets depend on the full event context but are produced in parallel. All three segment-length variants of the proxy (2/5/10 s, with $N_{\max}$ set to 32/64/128 accordingly) share the same architecture and yield approximately 4.9 M parameters.

6.5.3 MIDI Segment Sampling

The proxy is trained entirely on procedurally generated MIDI segments. Each segment is a list of note events $\{(p_n, t_n, \ell_n, v_n)\}_{n=1}^{N}$ in which pitch, onset, duration, and velocity $v_n \in [0, 1]$ are all written by the sampler rather than measured from any recording. The velocities therefore serve both as the proxy's input control variable during training and as the recoverable target in the velocity-recovery diagnostic of Section 6.7; this is the only sense in which a per-note velocity "ground truth" exists in this chapter.

Note events come from a three-component mixture, weighted 0.5/0.3/0.2.

Coverage component (50%). Ordinary polyphonic textures: pitches uniform over the MIDI piano range $[21, 108]$, polyphony capped at 14, durations uniform in $[0.08, 0.80]$ s, and inter-onset intervals uniform in $[0.03, 0.35]$ s. Per-note velocities are stratified between uniform, near-boundary, and extreme values.

Boundary component (30%). A low-polyphony pass (cap of 2, chord probability 0.12) with velocities biased towards the SoundFont's velocity-layer boundaries. SoundFonts internally store a small number of discrete sample layers, one per velocity range, and a small velocity change near a layer boundary produces a *sharp* acoustic transition rather than a smooth gain change. The proxy must learn these transitions, and this component concentrates training mass on them.

Stress component (20%). Deliberately adversarial: polyphony up to 16, chord size up to 8, inter-onset intervals as short as 0.01 s, and velocities biased towards the extremes of $[0, 1]$.

Within any chord, in all three components, per-note velocities are sampled independently, so the proxy is trained to produce per-note rather than chord-uniform velocity responses.

6.5.4 Teacher Rendering and Training Objective

Each sampled segment (E, V) is rendered to audio with the SoundFont and then reduced to per-note PHE/OSF as the teacher target.

Teacher rendering. Audio is rendered using the chosen SoundFont (`.sfz`) via the Python package `sfizz` at 22050 Hz, in segments of 2, 5, or 10 s with reverb disabled and short fade-outs to suppress synthesiser state leakage. The extractor F from Section 6.5.1, conditioned on the original note events, then produces the per-note target $Z = (\mathrm{PHE}_n, \mathrm{OSF}_n)_{n=1}^{N}$. Rendered segments and their targets are cached once and reused across epochs.

Training objective. Given input (E, V), the proxy predicts $\widetilde{Z} = (\widetilde{\mathrm{PHE}}_n, \widetilde{\mathrm{OSF}}_n)_{n=1}^{N}$. We train it by minimising the Huber loss $\mathcal{L}_{\text{params}}$ of Eq. (6.6), with a strict mask excluding padded slots from the gradient computation.

One loss form, two stages. $\mathcal{L}_{\text{params}}$ appears at two distinct stages of the framework, and it is helpful to keep them apart. In *proxy training* (this section), the proxy parameters are updated; the loss matches proxy prediction $P(E, V)$ against the teacher target $F(R(E, V))$ extracted from SoundFont-rendered audio. In *adaptation* (Section 6.4.3), the proxy is frozen and VeloEst parameters are updated; the loss matches $P(E, \hat{V})$ against $F(a, E)$ extracted from *real* audio. Same functional form, different operands, different parameters being updated.

6.6 Experimental Setup

The experiments are organised around two questions. First, can the frozen Diff-SFProxy provide useful gradients for velocity recovery when the true velocity is known in synthetic data? This is a precondition for the next experiment and is reported in Section 6.7. Second, when the front-end VeloEst model is adapted using real target-domain audio, does Diff-SFProxy provide a better supervision interface than waveform-domain Diff-Synth? This is the main empirical question of the chapter, reported in Section 6.8. The two experiments use different evaluation metrics: the diagnostic uses synthetic per-note velocity MAE, defined inline in Section 6.7.2, whereas the adaptation experiment uses perceptual loudness metrics, defined in Section 6.8.2. The present section defines only the datasets and the SoundFont assets shared by both experiments.

6.6.1 Datasets

The experiments use piano as the source domain, where velocity labels are available, and guitar as the main label-scarce target domain.

For piano, training and in-domain evaluation use the MAESTRO V3 dataset [5], which contains 198.7 h of Yamaha Disklavier performances with note-aligned MIDI velocities. The default train/valid/test split is followed throughout. Cross-dataset evaluation uses the SMD dataset [118], which contains 4.7 h of additional Yamaha Disklavier performances with the same label format.

For guitar, adaptation uses the GAPS dataset [124], which contains 14 h of nylon-string classical guitar with aligned notes but no velocity labels, again using its default train/valid/test split. Cross-dataset evaluation uses the François Leduc (FL) dataset [123], which contains 4.0 h of solo jazz guitar recordings on a mix of nylon-string, steel-string, electric, and acoustic guitars. The two guitar corpora differ substantially in playing style, recording chain, and instrument variety, which makes FL a useful test of out-of-distribution behaviour for guitar adaptation.

This dataset configuration has a deliberate asymmetry. On piano, ground-truth velocities are available on both the adaptation and the cross-dataset evaluation set, which permits direct error metrics. On guitar, no ground-truth velocities are available on either set, which forces a different evaluation protocol based on perceptual loudness comparison (Section 6.8.2).

6.6.2 SoundFont Selection

The SoundFont is chosen deliberately for each target instrument because a mismatched timbre would produce a teacher signal that the target recordings cannot match, regardless of how well the proxy is trained. Candidates were auditioned from the public FreePats sound banks, and the closest match was selected for each instrument family. Salamander Grand Piano [104] is used for piano, since it approximates the Yamaha grand piano timbre of MAESTRO and SMD. Spanish Classical Guitar [157] is used for guitar, since it is a close match for GAPS and for the nylon-string portion of FL.

For the metric diagnostic in Section 6.9.2, an alternative piano SoundFont is also used: YDP Grand Piano [158], an earlier and less realistic Yamaha grand piano sample. This second SoundFont is not used to train any proxy. It is used only to test how sensitive the chosen evaluation metric is to a change of rendering asset.

6.7 Diff-SFProxy Velocity-Recovery Diagnostic

Before deploying Diff-SFProxy in cross-instrument adaptation, we test whether its gradients can be trusted. The question is direct: *do the proxy's gradients point velocity in the right direction?* The answer is obtained through a controlled inversion on synthetic audio, where the true velocities are known by construction. The test segments are drawn from the same procedural sampler as the proxy's training data (Section 6.5.3), so the velocities are written into each segment by the sampler rather than measured from any recording. This holds for any instrument-SoundFont pair, including guitar, which is why a per-note velocity MAE is reported for both piano and guitar proxies in Table 6.1. The setting also isolates the gradient pathway from any real-audio domain gap. A pass on this diagnostic is a precondition for the cross-instrument adaptation experiments in Section 6.8.

6.7.1 Training Protocol

Each proxy is trained on 20k MIDI segments, with an additional 2k for validation. The optimiser is AdamW with a learning rate of 10^{-4}, weight decay 0.01, batch size 64, and a fixed random seed of 19. Diff-SFProxy is trained for 200 epochs; the validation loss typically plateaus between epochs 150 and 200. The checkpoint with the lowest validation loss is selected for each run. To accommodate the three input segment lengths 2/5/10 s, $N_{\max}$ is set to 32/64/128, respectively. All three proxies use the same Transformer architecture, resulting in approximately 4.9 M parameters each.

6.7.2 Diagnostic Protocol

Synthetic MIDI segments are rendered with the chosen SoundFont, and their PHE/OSF parameters are extracted as the target. All note velocities are then reset to 64, the proxy is frozen, and gradient descent is run on the candidate velocities until the proxy's predicted

PHE/OSF match the target. The optimiser is Adam with 120 steps and a learning rate of 0.03. The headline metric is the per-note velocity MAE, on the MIDI scale $[0, 127]$, between recovered and true velocities. A substantial reduction from the initial error indicates that the proxy's gradients yield update directions towards the true velocities, which is exactly the property needed for downstream adaptation.

Two test groups are used, each containing 2k MIDI segments. The **in-domain test** draws from the same sampler used for training. The **stress test** uses denser polyphony, larger chords, and tighter inter-onset intervals to probe behaviour beyond the training distribution.

6.7.3 Results

Table 6.1 reports the diagnostic for piano and guitar proxies at 2/5/10 s segment lengths. Three patterns are visible. First, in-domain recovery cuts the initial error to about a third for all six proxies, both for piano (gain around 67–68%) and for guitar (gain around 72–74%). Second, under the stress test the recovery improves from 2 s to 5 s, most clearly on piano (gain rises from 49.5% to 54.0%), suggesting that a longer context helps the proxy disentangle dense overlapping notes. Third, from 5 s to 10 s the gain disappears: all corresponding cells fall within half a MIDI unit of each other. The 10 s setting therefore adds training cost without further benefit and is discarded in the main study. The 5 s proxy is used as the default in the cross-instrument adaptation, while the 2 s proxy is kept as an ablation in the next section.

Table 6.1: Velocity-recovery diagnostic for Diff-SFProxy. Init and Recovery report per-note velocity MAE in MIDI units $[0, 127]$ before and after optimisation; Gain = (Init − Recovery)/Init × 100%. Within each instrument, the best gain is highlighted in bold. The 5 s configuration is the default used in the cross-instrument adaptation.

	In-domain			**Stress**		
Diff-SFProxy	Init	Recovery	Gain ↑	Init	Recovery	Gain ↑
Piano, 2 s	37.2	12.3	66.9%	38.6	19.5	49.5%
Piano, 5 s	37.9	12.2	**67.8%**	38.5	17.7	**54.0%**
Piano, 10 s	38.2	12.5	67.3%	38.5	17.8	53.7%
Guitar, 2 s	33.6	9.3	72.3%	34.2	11.3	67.0%
Guitar, 5 s	34.6	9.1	**73.7%**	33.8	10.7	**68.3%**
Guitar, 10 s	34.6	9.5	72.5%	33.8	10.8	68.0%

This diagnostic does not validate Diff-SFProxy on real audio. It only validates that the gradients computed through the proxy point in the correct direction in a synthetic, label-controlled regime. Real-audio behaviour is the subject of the next section.

6.8 Cross-Instrument VeloEst Adaptation

The adaptation experiment serves two distinct purposes. On guitar, where velocity labels are unavailable, it provides a label-free route to MIDI velocity estimation and allows the practical

question of beyond-piano deployment to be answered directly. On piano, where ground-truth velocities are available on MAESTRO and SMD, it serves as a controlled diagnostic. We adapt on MAESTRO so that predictions can be checked against true labels, verifying whether the non-velocity supervision pushes velocity towards the right value. The piano result is therefore not a claim about improving over a supervised piano baseline; it is a sanity check that the supervision signal is informative.

The label scarcity referred to here is on real recordings. GAPS and FL datasets provide aligned note events, but the per-note velocities used by the performers were never captured by an instrument-side sensor and are therefore not accessible as supervision. This is the opposite of the synthetic guitar segments used in Section 6.7, where the velocity of every note is written into the MIDI segment by the sampler itself before any rendering takes place, and is therefore directly available for per-note MAE comparison. Real-recording label scarcity is a property of how the dataset was captured, not of the instrument itself.

6.8.1 Training Protocol

Three settings are compared on the same VeloEst front-end: the piano-pretrained baseline applied without further training, Diff-Synth adaptation, and Diff-SFProxy adaptation. The Diff-Synth back-ends [109, 111] were originally implemented for 3 s input segments. Each is retrained at 2 s and 5 s with its original hyperparameters, then frozen, so that crop length is comparable across the two adaptation interfaces.

During adaptation, VeloEst processes 10 s segments and outputs a frame-wise velocity roll. The frozen back-end processes shorter crops, using 5 s by default and 2 s for ablation. Back-end losses are averaged over valid notes in each crop and back-propagated through the onset-frame velocity reads to the full velocity roll, which keeps the gradient scale comparable across crop lengths. Adaptation is run separately on the MAESTRO and GAPS training sets for 120k iterations using Adam with learning rate 10^{-4}, batch size 8, and step decay 0.9 every 10k iterations. SMD and FL datasets are used only for evaluation. All runs use seed 19, and the final checkpoint is selected by validation r_{BSSL} on MAESTRO for piano and on GAPS for guitar.

6.8.2 Evaluation Metrics

For piano datasets with ground-truth velocities, we follow [23] and report per-note $\mathrm{MAE}_{\mathrm{Velo}}$ on the MIDI scale $[0, 127]$, computed against the true note velocities. For datasets without velocity labels, $\mathrm{MAE}_{\mathrm{Velo}}$ cannot be computed. We instead re-render the aligned notes through the selected SoundFont using the predicted velocities and compare the resynthesis with the original recording in perceptual loudness space.

The comparison features are Bark-scale specific loudness (BSSL) and Bark-scale total loudness (BSTL), as defined within the Zwicker psychoacoustic framework [29]. BSSL is a frame-wise critical-band representation, and BSTL is its band-summed contour, tracking perceived loudness over time. Both features capture the main perceptual effect targeted by velocity estimation,

namely the temporal shaping of loudness, while reducing sensitivity to timbral and recording-condition mismatch between real audio and SoundFont resynthesis. Two natural alternatives are deliberately avoided. Raw RMS energy ignores frequency-dependent perceptual sensitivity. Note-wise loudness measures, including the PHE and OSF used inside the proxy, are restricted to onset-locked windows and miss the continuous dynamics shape carried by velocity expression. They also coincide with the proxy training signal, which would make the evaluation circular.

Pearson correlation r between real and rendered loudness contours is the primary metric. It is invariant to global scale and offset, so it discounts loudness differences introduced by gain, microphone response, or renderer calibration. Pearson correlation computed on BSSL and BSTL is denoted r_{BSSL} and r_{BSTL}, respectively. Cosine similarity (cs) and MAE on these features are retained as diagnostic alternatives: cosine similarity is scale-invariant but not offset-invariant, while MAE is sensitive to both. Their behaviour is examined in Section 6.9.

6.8.3 Results

Tables 6.2 and 6.3 report evaluation across the four test sets, separated by instrument family. The row labelled *VeloEst: pretrained on MAESTRO* is the supervised checkpoint applied without further training, which serves as the zero-shot transfer baseline for SMD, GAPS, and FL datasets.

Three observations stand out. First, on piano (Table 6.2) the supervised checkpoint remains the strongest reference. Both adaptation strategies underperform on piano because they optimise indirect performance-matching objectives rather than the available velocity labels. This is expected and consistent with the role assigned to the piano experiment: it is a sanity check, not a competition.

Second, on guitar (Table 6.3) Diff-SFProxy achieves the best r_{BSSL} and r_{BSTL} on both GAPS and FL datasets, outperforming flat velocity, zero-shot transfer, and Diff-Synth across all back-end crop lengths. Diff-Synth, by contrast, does not reliably improve guitar performance

Table 6.2: Evaluation on piano datasets, where ground-truth velocities are available. Diff-Synth and Diff-SFProxy back-ends process 5 s segments by default. Best score is highlighted in bold, and the second-best is underlined. The bottom two rows are ablations on Diff-SFProxy.

Method	MAESTRO test set			SMD dataset		
	$\text{MAE}_{\text{Velo}} \downarrow$	$r_{\text{BSSL}} \uparrow$	$r_{\text{BSTL}} \uparrow$	$\text{MAE}_{\text{Velo}} \downarrow$	$r_{\text{BSSL}} \uparrow$	$r_{\text{BSTL}} \uparrow$
Flat Velocity: set at 64	14.8	0.757	0.622	15.4	0.701	0.582
VeloEst: pretrained on MAESTRO	**3.9**	**0.878**	**0.892**	**8.0**	**0.829**	**0.816**
VeloEst + Diff-Synth	19.2	0.776	0.752	19.7	0.789	0.742
- uses 2s backend instead of 5s	25.0	0.702	0.553	25.9	0.662	0.520
VeloEst + Diff-SFProxy	<u>10.5</u>	<u>0.869</u>	<u>0.881</u>	<u>11.3</u>	<u>0.821</u>	<u>0.806</u>
- uses 2s backend instead of 5s	10.9	0.858	0.870	11.8	0.820	0.804
- w/o anti-collapse and satu. loss	12.9	0.789	0.698	14.4	0.735	0.653
- w/o VeloEst pretrained weights	20.5	0.770	0.757	21.2	0.723	0.703

Table 6.3: Evaluation on guitar datasets, where ground-truth velocities are not accessible. MAE_{Velo} is therefore omitted. Diff-Synth and Diff-SFProxy back-ends process 5 s segments by default. Best score is highlighted in bold, and the second-best is underlined.

Method	GAPS test set		FL dataset	
	$r_{\text{BSSL}} \uparrow$	$r_{\text{BSTL}} \uparrow$	$r_{\text{BSSL}} \uparrow$	$r_{\text{BSTL}} \uparrow$
Flat Velocity: set at 64	0.737	0.701	0.714	0.587
VeloEst: pretrained on MAESTRO	0.769	0.798	0.740	0.699
VeloEst + Diff-Synth	0.669	0.624	0.646	0.547
- uses 2s backend instead of 5s	0.760	0.770	0.729	0.636
VeloEst + Diff-SFProxy	**0.794**	**0.860**	**0.777**	**0.788**
- uses 2s backend instead of 5s	<u>0.787</u>	<u>0.836</u>	<u>0.771</u>	<u>0.756</u>
- w/o anti-collapse and saturation loss	0.740	0.705	0.723	0.614
- w/o VeloEst pretrained weights	0.702	0.686	0.683	0.600

and can degrade it. On FL dataset, the 5 s Diff-Synth back-end falls below flat velocity in r_{BSSL} (0.646 vs. 0.714). This supports the concern raised in Section 6.4.2: waveform-level objectives are more exposed to timbral and room-acoustic mismatch than parameter-level supervision.

Third, the asymmetry between the two adaptation interfaces widens with crop length. On FL dataset, Diff-Synth's r_{BSSL} falls by 0.083 when moving from 2 s to 5 s, while Diff-SFProxy's r_{BSSL} varies within 0.01. Longer crops give waveform-level objectives more of the underlying timbral mismatch to fit, while parameter-level supervision sees only per-note targets and is therefore much less sensitive to crop length. This mirrors a long-standing observation in inverse-problem design: a supervision signal that only depends on the latent variable of interest is more stable than a high-dimensional signal that mixes that variable with many nuisance factors.

It is also worth noting that the zero-shot piano checkpoint already exceeds flat velocity on both guitar sets, suggesting a partial cross-instrument transfer of velocity sensitivity. Diff-SFProxy adds a consistent further gain on top of this baseline, which is the main practical message of the chapter.

6.8.4 Ablations

The bottom rows of Tables 6.2 and 6.3 isolate two design choices. Removing $\mathcal{L}_{\text{anti}} + \mathcal{L}_{\text{satu}}$ gives a plausible MAE_{Velo} on MAESTRO (12.9 vs. 10.5), but r_{BSSL} falls near the flat-velocity baseline (0.789 vs. 0.757). The outputs collapse towards a near-constant value, consistent with the median-seeking behaviour of MAE: central predictions can keep absolute error tolerable while losing the dynamic contour. The two regularisers are therefore not optional. They are required to prevent the model from exploiting the median solution that the loss landscape silently rewards.

Removing the piano-pretrained checkpoint leaves the Diff-SFProxy loss without a supervised velocity anchor. Performance drifts across all datasets and underperforms flat velocity on three of the four guitar metrics. Piano pretraining therefore acts as a crucial prior for cross-

instrument adaptation. This is consistent with a more general pattern in transfer learning: a strong supervised initialisation often determines whether a downstream self-supervised objective converges to a useful solution or to a degenerate one.

Figure 6.3 makes both failure modes visible on a guitar excerpt: without the regularisers (panel c), the predicted velocity contour collapses towards a near-constant value across notes; without MAESTRO pretraining (panel d), the contour drifts away from the reference rather than tracking it. The proposed configuration in panel (b) is the only one of the three that recovers a velocity sequence whose dynamic shape resembles the ground truth in panel (a).

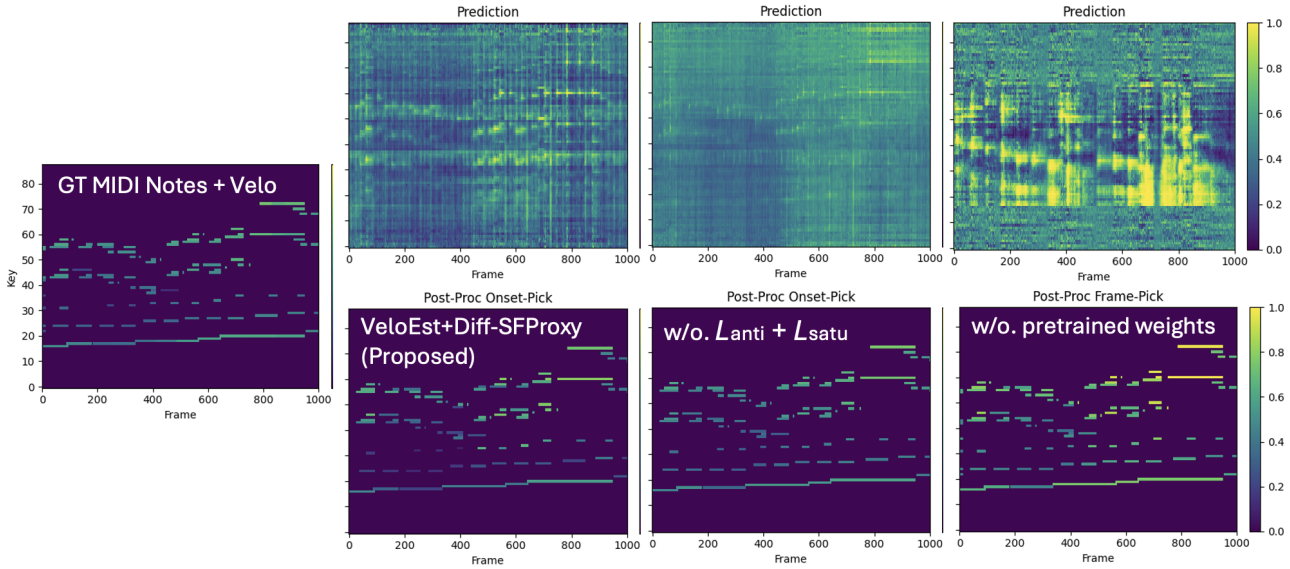


Figure 6.3: Qualitative ablation on a guitar excerpt. From left to right: (a) the reference MIDI notes with ground-truth velocities; (b) the proposed VeloEst + Diff-SFProxy adaptation, whose predicted velocities track the dynamic contour of the reference; (c) the same model trained without the anti-collapse and saturation regularisers ($\mathcal{L}_{\text{anti}} + \mathcal{L}_{\text{satu}}$), where the predicted velocities collapse towards a near-constant median; (d) Diff-SFProxy adaptation without the MAESTRO-pretrained VeloEst initialisation, trained directly on guitar, where the predicted contour drifts away from the reference. The two failure modes correspond to the bottom two rows of Tables 6.2 and 6.3.

6.9 Discussion

6.9.1 What the Recovered Velocity Actually Represents

Velocity is not the only MIDI control that shapes rendered audio loudness. Pedal, reverb, and continuous controllers all contribute in parallel [18]. Diff-Synth back-ends such as DDSP-Piano expose several of these controls. Consequently, when adaptation updates only velocity, the velocity gradient can become entangled with residuals caused by fixed or unobserved controls. Diff-SFProxy avoids this contamination by treating velocity as the only loudness-shaping degree of freedom, but the choice carries its own cost. The recovered velocity absorbs effects that, in the real performance, came from other parallel controls.

The recovered velocity is therefore not a canonical property of the acoustic performance. It is defined relative to the chosen SoundFont as the velocity that best reproduces the perceptual

dynamics of the original audio. This renderer-conditioned velocity is the practical deployment target for label-scarce expressive analysis or resynthesis, in line with the operational definition adopted in Section 6.3. Two consequences follow. First, the same real recording can give rise to different recovered velocities under different SoundFonts, so any reported velocity is implicitly tagged by the renderer used. Second, the evaluation metric must be robust to the SoundFont choice and yet sensitive enough to distinguish expressive from flat dynamics. The next subsection tests whether the metric used in this chapter satisfies these two requirements.

6.9.2 Why Pearson r on BSSL/BSTL

Table 6.4 tests three candidate metrics, Pearson correlation r, cosine similarity cs, and MAE on BSSL and BSTL, against the two requirements implied above: sensitivity to expressive dynamics and robustness to renderer-specific loudness calibration. Real audio is compared with GT-note resynthesis under either ground-truth or flat velocity, and using either the Salamander default piano or the alternative YDP Grand Piano.

Table 6.4: MAESTRO metric diagnostic. Real audio is compared with GT-note resynthesis using GT or flat velocity under the Salamander (default) or YDP Grand Piano. Summary rows show GT-Flat separation and SoundFont drift. Best score is highlighted in bold, and the second-best is underlined.

Method	MAESTRO test set					
	BSSL			BSTL		
	r	cs	MAE	r	cs	MAE
GT velocity	0.870	0.939	0.702	0.883	0.975	3.706
- uses YDP Piano	0.836	0.926	0.510	0.872	0.973	1.983
$\Delta\lvert\text{Sal.} - \text{YDP}\rvert \downarrow$	<u>0.034</u>	**0.013**	0.193	<u>0.011</u>	**0.002**	1.723
Flat velocity = 64	0.752	0.887	0.786	0.620	0.928	4.065
- uses YDP Piano	0.733	0.883	0.637	0.617	0.930	2.750
$\Delta\lvert\text{Sal.} - \text{YDP}\rvert \downarrow$	<u>0.019</u>	**0.004**	0.149	<u>0.003</u>	**0.002**	1.314
Δ (GT − Flat) ↑	**+0.118**	<u>+0.052</u>	-0.083	**+0.263**	<u>+0.047</u>	-0.359

To test dynamics sensitivity, $\Delta(\text{GT}-\text{Flat}\,64)$ asks whether a metric can distinguish expressive resynthesis from a flat one. Among the similarity metrics, Pearson r produces the larger separation on both BSSL (+0.118) and BSTL (+0.263). Cosine similarity also orders GT above flat, but it severely compresses the margin (+0.052 on BSSL and +0.047 on BSTL), proving *insensitive* to fine dynamics differences.

To evaluate renderer robustness, $\Delta|\text{Sal.} - \text{YDP}|$ asks whether a metric is stable when the rendering SoundFont is swapped while the underlying note velocities are kept the same. Here, MAE proves *over-sensitive* to absolute gain: it drifts by 0.193 on BSSL and 1.723 on BSTL, substantially more than the corresponding drift on r (0.034 and 0.011). On real recordings, this drift would conflate dynamics differences with renderer choice and make the evaluation unreliable.

Pearson r on BSSL/BSTL therefore provides the right trade-off for this chapter. Cosine similarity and MAE fail on sensitivity and robustness, respectively, and are kept only as diagnostic alternatives. This is also the reason the main results table reports r_{BSSL} and r_{BSTL} as the headline metrics for label-free guitar evaluation.

6.9.3 When Diff-Synth Should Still Be Preferred

Diff-SFProxy is not a universal replacement for Diff-Synth. Three conditions favour the waveform interface. First, the matched differentiable synthesiser must exist for the target instrument and expose a faithful per-note velocity control. Second, its waveform statistics must be acoustically close to the target recordings. Third, the supervision must be allowed to reach beyond velocity and update auxiliary controls such as pedal or articulation. When all three conditions hold, waveform supervision can in principle deliver richer gradients than parameter supervision, and the present chapter does not contest this in general terms.

The claim of this chapter is narrower. In many cross-instrument settings, none of these three conditions holds robustly. A matched Diff-Synth may not exist for the target instrument, may be expensive to retrain at the desired crop length, or may differ from the real recordings in timbre and recording chain. A practical SoundFont, by contrast, is widely available, plug-and-play, and instrument-agnostic at the proxy-design level. In that regime, a note-wise proxy that keeps only the subset of synthesised behaviour most related to velocity can be the more useful engineering choice. This is the regime in which the present chapter is positioned.

6.9.4 Limitations

Several boundaries should be kept in mind. First, the recovered velocity is renderer-conditioned (Section 6.9.1). The SoundFont is selected by ear from a small candidate pool. A mismatched SoundFont would produce a teacher signal that the target recordings cannot match, so renderer choice is part of the operational definition rather than an incidental detail.

Second, the experiments focus on piano and guitar. Both instruments are predominantly percussive in their attack profile, which favours the OSF parameter. Sustained-bow instruments such as violin, or breath-driven instruments such as flute, would require acoustic parameters that capture sustain and continuous-modulation cues beyond the PHE/OSF pair used here. The claim is therefore tied to the tested attack-driven setting.

Third, the framework treats velocity as a single one-dimensional control. As noted in Section 6.3, this is an inherent simplification for many acoustic instruments, where multiple continuous controls shape note dynamics. The recovered velocity is therefore best read as a coarse note-level dynamics coordinate rather than as a complete description of expressive intent.

Fourth, the framework relies on aligned note events. Onset and duration errors propagate into the PHE/OSF windows and into the Diff-Synth rendering, and would degrade adaptation accordingly. The chapter assumes a score-informed setting in which alignment is already good enough for note-wise analysis, keeping the experiment focused on the supervision interface rather

than on alignment recovery.

6.10 Summary

This chapter returned to note-level MIDI velocity, but removed the condition that made supervised piano work possible: ground-truth velocity labels in the target domain. The proposed framework treats beyond-piano velocity estimation as a missing-supervision problem. A piano-pretrained VeloEst supplies the prior, while adaptation is driven through a renderer interface. The comparison between Diff-Synth and Diff-SFProxy shows that the useful question is not simply whether the renderer is differentiable, but whether its gradient is concentrated on the control variable being estimated.

Diff-SFProxy is preferred in the tested guitar setting because it supervises velocity through note-wise loudness-related parameters rather than through waveform reconstruction. This reduces the influence of timbre, room, microphone, and renderer mismatch, and makes the recovered velocity a renderer-conditioned dynamics coordinate rather than a claim about the player's physical gesture. The ablations are important here: without piano pretraining and anti-collapse regularisation, the model can drift toward median or near-constant solutions. The chapter therefore closes the velocity branch of the thesis under the weakest supervision assumption considered here: aligned note events remain available, but the velocity target itself must be constructed through a perceptually grounded proxy.

Chapter 7

Conclusion

This thesis studied dynamics recovery as a problem of missing expressive labels under structural preservation. The starting point was not the general observation that musical performance is expressive, but the narrower problem that many useful music representations already contain reliable structure while lacking a dynamics layer. A MIDI file may contain the correct notes but flat velocities. An AMT output may recover note events but not the intensity attached to them. A score-level transcription may contain metre and pitch content but no dynamic markings. A beyond-piano dataset may provide audio and aligned notes but no trustworthy velocity labels.

The thesis question introduced in Chapter 1 was:

> **How can musically interpretable dynamics labels be recovered from incomplete, weakly expressive, or weakly supervised music data without rewriting note content or note timing?**

The answer developed in this thesis is conditional rather than universal. Dynamics is not a single target shared across all settings. It may be a note-level MIDI control, a beat-synchronous score marking, a perceived-loudness contour used for evaluation, or a renderer-conditioned coordinate constructed under missing supervision. A valid recovery method must therefore preserve the trusted musical structure, choose the output coordinate appropriate to the target representation, and evaluate the result in the space where that representation has musical meaning.

This thesis builds that answer through four controlled recovery settings. Chapter 3 studies symbolic-only MIDI velocity completion. Chapter 4 studies score-informed correction of audio-derived MIDI velocity. Chapter 5 studies audio-only recovery of beat-synchronous score dynamics and metrical structure. Chapter 6 extends velocity recovery beyond the supervised piano regime through renderer-conditioned proxy supervision. These chapters form a scenario map rather than one end-to-end pipeline: each preserves a different structural coordinate and asks what kind of dynamics label can be recovered under that condition.

7.1 Summary of Contributions

The thesis makes one formulation-level contribution and four method-level contributions. Together, they are best read as instantiations of the same principle: the recoverable dynamics layer depends on what structure is trusted and what supervision is available. The thesis does not propose a single model for all forms of expression. It defines controlled recovery settings and studies the representation, model design, and evaluation method appropriate to each.

7.1.1 A unified formulation of dynamics-oriented label recovery

The first contribution is the thesis-level formulation itself. Chapter 2 established that the gap between loudness, MIDI velocity, and score dynamics is not a minor conversion problem. A fixed mapping from velocity to *p*, *mf*, or *f* is attractive because it is simple, but it collapses several distinct stages: performer action, instrument response, acoustic signal, auditory perception, and score-level interpretation. The thesis therefore treats each dynamics representation as a valid but partial coordinate on expression.

This formulation leads to a clear division of tasks. If the target is note-level MIDI velocity, the system should work on note onsets and preserve note identity. If the target is score dynamics, the system should work on a musical coordinate such as beats or phrases, not arbitrary audio frames alone. If the supervision is indirect, the recovered label must be interpreted relative to the proxy that supplies the learning signal. These distinctions are simple, but they prevent a large conceptual error: treating dynamics as a scalar loudness value that can be moved freely between audio, MIDI, and notation.

Table 7.1: Scenario-level summary of the thesis.

Chapter	Available structure	Recovered dynamics layer	Main thesis role
Chapter 3	Fixed symbolic notes and timing	Note-level MIDI velocity	Shows that dynamics can be completed from symbolic context alone.
Chapter 4	Performance audio and aligned score events	Corrected note-level MIDI velocity	Shows that score structure can act as a late correction prior for AMT velocity.
Chapter 5	Performance audio only	Beat-synchronous score dynamics, change points, beats, and downbeats	Shows that score-facing dynamics can be recovered jointly with the metrical coordinate needed to write them.
Chapter 6	Performance audio and aligned note events without target velocity labels	Renderer-conditioned note-level MIDI velocity	Shows that beyond-piano recovery is primarily a missing-supervision problem.

7.1.2 Symbolic MIDI velocity completion

The first method contribution establishes the most controlled setting: the symbolic note sequence is fixed, and only the missing MIDI velocity layer is restored. Chapter 3 reformulates this task as image colourisation on sparse piano-roll matrices. This is not only an architectural analogy. Polyphonic symbolic music contains local pitch–time neighbourhoods and vertical chordal patterns that are weakened when the task is treated as a plain sequence. A U-Net with window attention and a sparsity-aware loss can exploit this structure while leaving the underlying note content unchanged.

The chapter also demonstrates that a decoupled expressive layer has perceptual value. The model changes only velocity while preserving notes and timing, yet the listening test shows a clear improvement over flat and sequence-oriented baselines. This supports the central thesis stance: expressive recovery does not always require full expressive generation. In many practical settings, a controlled post-processing layer is the more appropriate object.

7.1.3 Score-informed velocity correction

The second method contribution introduces performance audio while keeping the note structure fixed through an aligned score. Chapter 4 proposes a modular correction strategy: keep the AMT backbone unchanged, then use score information as a late residual correction signal for note-level velocity. This design is deliberately different from early-fusion score-informed transcription. It treats score information as a structural prior rather than as a replacement for acoustic evidence.

The empirical result is that the late score-informed module adds only a small parameter overhead on top of an existing AMT backbone, transfers across HPT, HPPNet, and DynEst with consistent improvements in MAESTRO-to-SMD cross-dataset behaviour, and remains stable under moderate audio–score timing shifts. The deeper thesis result is about modularity. If expressive AMT is to remain useful as AMT front ends evolve, dynamics recovery should not depend on a single fixed transcription architecture. A late correction layer preserves this modularity while still using the musical information contained in the score.

7.1.4 Audio-only score-dynamics recovery

The third method contribution changes the target itself from note-level MIDI velocity to score-facing dynamic markings. Chapter 5 therefore requires a different temporal coordinate. Score dynamics is not naturally a note-onset variable; it describes dynamic regimes, changes, and phrase-level intensity states. The chapter formulates the task as joint estimation of dynamic levels, dynamic change points, beats, and downbeats because the dynamic label is only useful once it is placed on a writable metrical grid.

This chapter provides the score-dynamics counterpart to the velocity-focused chapters. It uses Bark-scale specific loudness as a perceptually grounded input representation, paired with a multi-scale encoder and an MMoE decoder that resolve the conflict between long-range dynamic

context and precise metrical localisation. The result shows that score-facing dynamics can be estimated directly from audio when the model also estimates the coordinate on which those labels are meaningful. The point is not that score dynamics is a discretised version of MIDI velocity. It is a different symbolic target, and its modelling must respect that difference.

7.1.5 Beyond-piano velocity recovery through proxy supervision

The fourth method contribution removes the strongest assumption used by supervised piano work: target-domain velocity labels. Chapter 6 treats beyond-piano velocity estimation as a missing-supervision problem rather than ordinary domain shift. Piano has an unusually strong supervisory anchor because datasets such as MAESTRO provide note-aligned velocity labels through sensor-equipped instruments. For guitar and many other instruments, aligned notes and audio may be available, but true note-level velocity labels are not.

The chapter addresses this setting through analysis-by-synthesis. A piano-pretrained VeloEst front end is adapted using differentiable supervision from either a waveform synthesiser or a SoundFont proxy. The main contribution is Diff-SFProxy: a compact differentiable proxy that maps note events and predicted velocities to note-wise loudness-related parameters. This parameter-level interface is more robust than waveform-level supervision under the tested recording mismatch because it supervises the subset of rendered behaviour most closely related to velocity. The recovered velocity is renderer-conditioned, but that conditioning is explicit and operationally useful. It gives a practical route to note-level dynamics recovery when the target domain lacks direct velocity labels.

7.2 Cross-Chapter Findings

Across the four studies, the main findings are not only architectural. They concern when a dynamics label is well defined, what evidence can support it, and how it should be evaluated.

7.2.1 Structure is a condition of recovery, not a convenience

All four studies depend on preserving some trusted musical structure. In Chapter 3, this is the symbolic note sequence. In Chapter 4, it is the aligned score. In Chapter 5, it is the beat and downbeat grid that makes score dynamics writable. In Chapter 6, it is the aligned note event list used for renderer-conditioned adaptation. The thesis therefore treats dynamics recovery as a constrained restoration problem. Without a stable coordinate system, the task collapses into unconstrained expressive generation.

This stricter formulation is also the source of the practical value. A recovered layer can be inspected, edited, attached to an existing score, or used to enrich a corpus without changing the underlying musical content. The preservation constraint is therefore not merely conservative. It is what makes the output musically accountable.

7.2.2 Velocity and score dynamics are not interchangeable labels

MIDI velocity is a note-level control parameter. Score dynamics is a sparse, contextual marking system. Perceived loudness is an acoustic or perceptual observation that can support either task but is not identical to either output. This distinction explains why the thesis uses different temporal units, losses, and metrics across chapters instead of forcing all dynamics representations into one scalar target.

This is also why Chapter 6 defines recovered velocity relative to a renderer. Outside piano, a MIDI velocity value is not a universal physical measurement, but it can still be a useful note-wise dynamics coordinate for analysis and resynthesis. The thesis therefore avoids the false choice between physical exactness and musical usefulness. A dynamics representation can be operationally useful while still being representation-dependent.

7.2.3 Evaluation must match the claimed representation

Point-wise velocity error is useful when ground-truth velocity labels exist, but it can miss perceptual flatness. Listening tests and dispersion-based diagnostics reveal whether a restored velocity layer produces audible expressiveness. For score dynamics, beat-synchronous F1 is more appropriate because the output is a symbolic annotation on a metrical grid. For label-scarce beyond-piano recovery, perceptual loudness correlation is more defensible than velocity MAE because there is no ground-truth velocity label in the target domain.

This produces a general evaluation principle: the metric should be tied to the representation being recovered. A low velocity error is not the same as a convincing score dynamic marking. A good waveform reconstruction is not the same as accurate velocity recovery. A strong note-transcription system is not automatically a strong dynamics-transcription system. Treating these distinctions explicitly is part of the contribution of the thesis.

7.2.4 Weaker supervision changes the claim, not necessarily the usefulness

The thesis moves from supervised symbolic completion to indirect renderer-conditioned adaptation. As supervision weakens, the recovered label should be interpreted more carefully. A supervised piano velocity estimate can be compared against Disklavier labels. A score-dynamics estimate can be compared against an editorial annotation convention. A beyond-piano renderer-conditioned velocity cannot claim to be the performer's physical gesture, but it can still be a useful control coordinate for resynthesis, curation, and expressive analysis.

The result is not that weaker supervision makes the task impossible. Rather, weaker supervision changes what can be claimed. Each claim is valid inside its representational boundary. The thesis contribution is to make those boundaries explicit instead of hiding them behind a single generic term such as dynamics.

7.3 Limitations and Scope

The thesis has several boundaries. These boundaries are not artefacts of unfinished implementation; they follow from the problem definition and from the representational choices made throughout the dissertation.

First, the thesis focuses on dynamics rather than on the full space of musical expression. Timing, articulation, pedalling, timbre, and phrasing interact with dynamics in real performance. The systems developed here isolate dynamics because it is perceptually salient, directly useful for data curation, and recoverable under structural preservation. This isolation is methodologically useful, but it also means that the recovered layer is one dimension of expression rather than a complete performance model.

Second, the strongest direct supervision in the thesis remains piano-centric. This reflects the state of available velocity-labelled datasets rather than a belief that piano dynamics is the universal template for all instruments. Chapter 6 addresses this bias by moving into a label-scarce guitar setting, but the broader point remains: once the target instrument lacks direct performance-control labels, the interpretation of recovered velocity must be stated operationally.

Third, score dynamics is inherently contextual. A dynamic marking is not a fixed loudness threshold. It depends on edition, local musical function, register, texture, and phrase context. Chapter 5 treats score dynamics as a beat-synchronous symbolic target because that is the most useful interface for notation and analysis. The resulting labels should therefore be read as score-facing annotations under a specific annotation convention, not as universal statements about physical loudness.

Fourth, several chapters assume that structural coordinates are available or recoverable. Score-informed velocity correction assumes aligned score events. The beyond-piano proxy framework assumes aligned note events. Audio-to-score dynamics estimates beats and downbeats, but still relies on the idea that beat-synchronous output is the right symbolic interface. These assumptions are not weaknesses of the argument. They are the conditions under which label recovery, rather than unconstrained generation, becomes meaningful.

Finally, the thesis uses multiple forms of evidence rather than a single unified benchmark. This is necessary because the targets differ across chapters. It also means that the contributions should be read as a coherent scenario map, not as four entries in one leaderboard. The unifying object is not one architecture or one dataset. It is the recovery of musically interpretable dynamics under different structural and supervisory conditions.

7.4 Future Research Directions

The results of the thesis suggest several directions for future research. These directions extend the thesis argument rather than repair it.

7.4.1 Probabilistic bridges between dynamics representations

A natural direction is to model the relations between velocity, loudness, and score dynamics probabilistically. The thesis has deliberately avoided fixed lookup tables because they collapse representation-specific information. A stronger future model could instead treat the three layers as coupled but uncertain variables. A passage might support several plausible score-dynamic annotations, several possible velocity profiles, and several loudness realisations depending on instrument and performance style.

Such a model would allow consistency constraints without enforcing deterministic conversion. A predicted *ff* region should normally be compatible with strong local loudness or velocity evidence, but the relation should be contextual rather than fixed. Conversely, a sequence of high velocities may indicate a dynamic peak, but only relative to register, texture, and phrase structure. Probabilistic modelling is a better fit for this many-to-many relation than a scalar threshold.

7.4.2 Uncertainty-aware score annotation

Score dynamics is a human-readable target, but it is not always a single-label truth. Different editions and annotators can choose different markings for similar musical evidence. Future systems should therefore represent uncertainty explicitly. Instead of forcing one label at every beat, an annotation system could output ranked alternatives, confidence distributions, or region-level ambiguity flags.

This would be useful in pedagogy and editing. A high-confidence dynamic label could be written directly into an automatically transcribed score. A low-confidence region could be presented to a musician for review. In this way, dynamics recovery would become a human-in-the-loop annotation tool rather than a black-box replacement for editorial judgement.

7.4.3 Instrument-general dynamics coordinates

The beyond-piano problem remains one of the most important directions for expressive AMT. The central difficulty is not only that guitar, violin, voice, and wind instruments sound different from piano. It is that their expressive control spaces are different. Piano velocity is a useful note-onset coordinate. Bowed strings and wind instruments also depend on continuous control dimensions such as bow pressure, bow speed, breath, vibrato, and articulation.

Future work can generalise the label-recovery stance by defining instrument-appropriate dynamics coordinates rather than forcing all instruments into piano-style velocity. For some instruments, the recovered layer may remain a MIDI-like note intensity value. For others, it may require onset intensity plus sustain shape, articulation class, or continuous modulation descriptors. The principle from this thesis still applies: the target must be chosen according to the instrument, the available evidence, and the intended use of the recovered label.

7.4.4 Corpus-scale expressive data curation

A practical direction is corpus-scale deployment. Many music datasets already contain partial structure: MIDI files with flat velocity, audio recordings with aligned scores, or symbolic scores without expressive annotations. The methods in this thesis suggest that these corpora can be enriched without recollecting them from scratch. Symbolic velocity completion can improve static MIDI archives. Score-informed correction can refine existing audio–score datasets. Audio-to-score dynamics can add human-readable markings to performance recordings. Proxy supervision can extend the approach to label-scarce instruments.

The long-term value of such systems is not only better transcription. It is the construction of richer musical datasets. Once dynamics labels are recovered consistently, they can support expressive rendering, performance analysis, pedagogical feedback, and generative modelling. The thesis therefore positions dynamics recovery as a data infrastructure problem as much as a modelling problem.

7.4.5 Towards expressive AMT beyond note detection

The broader direction is an AMT paradigm in which note detection is only the first layer. A musically useful transcription should not stop at pitch and onset. It should also recover the expressive information that makes the transcription interpretable as performance and usable as symbolic music data. Dynamics is the layer studied in this thesis, but the same logic can extend to articulation, pedal, phrasing, and timbre.

The key lesson is that each expressive layer needs its own target definition and coordinate system. Adding more outputs to a network is not enough. The system must know whether it is predicting a physical control, a perceptual descriptor, or a score-facing annotation. It must also evaluate each output in the space where that output has musical meaning. This is the path from automatic note transcription towards expressive transcription.

7.5 Closing Remarks

This thesis shifts expressive AMT from the question of whether expression can be predicted to the more precise question of what kind of dynamics label can be recovered under a given structural and supervisory condition. Its central claim is that dynamics recovery becomes tractable when the musical structure is preserved, the target representation is explicit, and the evaluation metric matches the recovered layer.

The resulting view is deliberately constrained. It does not treat expression as a single quantity, and it does not attempt to rewrite a performance wholesale. Instead, it treats dynamics as an interpretable layer that can be restored, refined, or constructed from the evidence available. This provides a practical route for improving weakly expressive MIDI, enriching AMT outputs, adding score-level dynamic markings, and extending expressive data curation beyond the piano domain.

References

[1] D. W. Robinson and R. S. Dadson, "A re-determination of the equal-loudness relations for pure tones," *British Journal of Applied Physics*, vol. 7, no. 5, pp. 166–181, 1956.

[2] International Organization for Standardization, "Acoustics — normal equal-loudness-level contours," International Standard ISO 226:2003, International Organization for Standardization, Geneva, Switzerland, 2003, 2nd edition.

[3] Rod Nave, "Equal loudness curves," HyperPhysics, Georgia State University, n.d., [Online]. Available: http://hyperphysics.phy-astr.gsu.edu/hbase/Sound/eqloud.html.

[4] Qiuqiang Kong, Bochen Li, Xuchen Song, Yuan Wan, and Yuxuan Wang, "High-resolution piano transcription with pedals by regressing onset and offset times," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3707–3717, 2021.

[5] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck, "Enabling factorized piano music modeling and generation with the maestro dataset," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[6] Emmanouil Benetos, Simon Dixon, Zhiyao Duan, and Sebastian Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2019.

[7] Roger B. Dannenberg, "The interpretation of midi velocity," in *Proceedings of the International Computer Music Conference (ICMC)*, 2006, pp. 193–196.

[8] Jason Palamara and W. Scott Deal, "A dynamic representation solution for machine learning-aided performance technology," *Frontiers in Artificial Intelligence*, vol. 3, pp. 29, 2020.

[9] Hyon Kim, Pedro Ramoneda, Marius Miron, and Xavier Serra, "An overview of automatic piano performance assessment within the music education context," in *Proceedings of the International Conference on Computer Supported Education (CSEDU)*, 2022, pp. 465–474.

[10] Huan Zhang, Vincent K.M. Cheung, Hayato Nishioka, Simon Dixon, and Shinichi Furuya, "Llaqo: Towards a query-based coach in expressive music performance assessment," in *Pro-*

ceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2025, pp. 1–5.

[11] Eun Ji Park, "Music dynamics visualization for music practice and education," *Multimedia Tools and Applications*, vol. 84, pp. 36145–36161, 2025.

[12] Bochen Li, Xinzhao Liu, Karthik Dinesh, Zhiyao Duan, and Gaurav Sharma, "Creating a multitrack classical music performance dataset for multimodal music analysis: Challenges, insights, and applications," *IEEE Transactions on Multimedia*, vol. 21, no. 2, pp. 522–535, 2019.

[13] Francesco Foscarin, Andrew McLeod, Philippe Rigaux, Florent Jacquemard, and Masahiko Sakai, "ASAP: a dataset of aligned scores and performances for piano transcription," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 534–541.

[14] Alia Morsi and Xavier Serra, "Bottlenecks and solutions for audio to score alignment research," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 272–279.

[15] Carlos E Cancino-Chacón, Maarten Grachten, Werner Goebl, and Gerhard Widmer, "Computational models of expressive music performance: A comprehensive and critical review," *Frontiers in Digital Humanities*, vol. 5, pp. 25, 2018.

[16] Werner Goebl, *The Role of Timing and Intensity in the Production and Perception of Melody in Expressive Piano Performance*, Ph.D. thesis, Karl-Franzens-Universität Graz, Graz, Austria, 2003.

[17] Katerina Kosta, Oscar F. Bandtlow, and Elaine Chew, "Practical implications of dynamic markings in the score: Is piano always piano?," in *AES 53rd International Conference: Semantic Audio*, 2014, pp. 1–3.

[18] Jesung Ryu, Seungyeon Rhyu, Hong-Gyu Yoon, Eunchong Kim, Ju Young Yang, and Taehyun Kim, "Mid-fild: Midi dataset for fine-level dynamics," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024, pp. 222–230.

[19] Keon Ju Maverick Lee, Jeff Ens, Sara Adkins, Pedro Sarmento, Mathieu Barthet, and Philippe Pasquier, "The gigamidi dataset with features for expressive music performance detection," *Transactions of the International Society for Music Information Retrieval*, vol. 8, no. 1, pp. 1–19, 2025.

[20] Nazif Can Tamer, Yigitcan Özer, Meinard Müller, and Xavier Serra, "High-resolution violin transcription using weak labels," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 223–230.

[21] Jackson Loth, Pedro Sarmento, Saurjya Sarkar, Zixun Guo, Mathieu Barthet, and Mark B. Sandler, "Goat: A large dataset of paired guitar audio recordings and tablatures," *CoRR*, p. abs/2509.22655, 2025.

[22] Yifan Xie, Zixun Guo, and Mathieu Barthet, "Fine-grained midi expression transcription from wind and string instrument audio via sim2real transfer learning," in *Proceedings of the International Symposium on Computer Music Multidisciplinary Research (CMMR)*, 2025, pp. 570–581.

[23] H. Kim and X. Serra, "A method for midi velocity estimation for piano performance by a U-net with attention and FiLM," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 304–310.

[24] Jyoti Narang, Nazif Can Tamer, Viviana De La Vega, and Xavier Serra, "Automatic estimation of singing voice musical dynamics," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 256–263.

[25] Zhanhong He, David Cooper, Defeng Huang, and Roberto Togneri, "Filling midi velocity using u-net image colorizer," in *Proceedings of the International Symposium on Computer Music Multidisciplinary Research (CMMR)*, 2025, pp. 949–960.

[26] Zhanhong He, Hanyu Meng, Defeng Huang, and Roberto Togneri, "Joint estimation of piano dynamics and metrical structure with a multi-task multi-scale network," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 14607–14611.

[27] Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, and Ed H. Chi, "Modeling task relationships in multi-task learning with multi-gate mixture-of-experts," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2018, pp. 1930–1939.

[28] Hugo Fastl and Eberhard Zwicker, *Psychoacoustics: Facts and Models*, Springer Series in Information Sciences, 22. Springer, Berlin, Heidelberg, 3 edition, 2007.

[29] Eberhard Zwicker and Hugo Fastl, *Psychoacoustics: Facts and Models*, vol. 22 of *Springer Series in Information Sciences*, Springer, Berlin, 2nd, updated edition, 1999.

[30] Katerina Kosta, Rafael Ramirez, Oscar F. Bandtlow, and Elaine Chew, "Mapping between dynamic markings and performed loudness: A machine learning approach," *Journal of Mathematics and Music*, vol. 10, no. 2, pp. 149–172, 2016.

[31] Sageev Oore, Ian Simon, Sander Dieleman, Douglas Eck, and Karen Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, pp. 955–967, 2020.

[32] Carlos Eduardo Cancino-Chacón, *Computational Modeling of Expressive Music Performance with Linear and Non-linear Basis Function Models*, Ph.D. thesis, Johannes Kepler University Linz, 2018.

[33] Jyoti Narang, Marius Miron, Ajay Srinivasamurthy, and Xavier Serra, "Analysis of musical dynamics in vocal performances using loudness measures," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2022, pp. 33–39.

[34] Harvey Fletcher and Wilden A Munson, "Loudness, its definition, measurement and calculation," *Journal of the Acoustical Society of America*, vol. 5, no. 2, pp. 82–108, 1933.

[35] John Backus, *The Acoustical Foundations of Music*, W. W. Norton, New York, 2 edition, 1977.

[36] Eberhard Zwicker, "Unterteilung des hörbaren frequenzbereichs in frequenzgruppen," *Acustica*, vol. 10, pp. 185, 1960.

[37] Elias Pampalk, Andreas Rauber, and Dieter Merkl, "Content-based organization and visualization of music archives," in *Proc. ACM Int. Conf. Multimedia*. 2002, pp. 570–579, ACM.

[38] Elias Pampalk, Simon Dixon, and Gerhard Widmer, "Exploring music collections by browsing different views," *Computer Music Journal*, vol. 28, no. 2, pp. 49–62, 2004.

[39] Deutsches Institut für Normung, "DIN 45631/A1:2010-03 calculation of loudness level and loudness from the sound spectrum – zwicker method – amendment 1: Calculation of the loudness of time-variant sound," Standard, Deutsches Institut für Normung, March 2010.

[40] International Organization for Standardization, "ISO 532-1:2017 acoustics – methods for calculating loudness – part 1: Zwicker method," Standard, International Organization for Standardization, June 2017.

[41] Ernst Terhardt, "Calculating virtual pitch," *Hearing Research*, vol. 1, no. 2, pp. 155–182, 1979.

[42] Manfred R. Schroeder, Bishnu S. Atal, and John L. Hall, "Optimizing digital speech coders by exploiting masking properties of the human ear," *Journal of the Acoustical Society of America*, vol. 66, no. 6, pp. 1647–1652, 1979.

[43] Renato Panda, Ricardo Malheiro, and Rui Pedro Paiva, "Audio features for music emotion recognition: A survey," *IEEE Transactions on Affective Computing*, vol. 14, no. 1, pp. 68–88, 2023.

[44] Marco Fabiani and Anders Friberg, "Influence of pitch, loudness, and timbre on the perception of instrument dynamics," *The Journal of the Acoustical Society of America*, vol. 130, no. 4, pp. EL193–EL199, 2011.

[45] Alice Vivien Barchet, Johanna M. Rimmele, and Claire Pelofi, "Tensemusic: An automatic prediction model for musical tension," *PLOS ONE*, vol. 19, no. 1, pp. e0296385, 2024.

[46] Zhouyu Fu, Guojun Lu, Kai Ming Ting, and Dengsheng Zhang, "A survey of audio-based music classification and annotation," *IEEE Transactions on Multimedia*, vol. 13, no. 2, pp. 303–319, 2011.

[47] Juhan Nam, Keunwoo Choi, Jongpil Lee, Szu-Yu Chou, and Yi-Hsuan Yang, "Deep learning for audio-based music classification and tagging: Teaching computers to distinguish rock from bach," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 41–51, 2019.

[48] Ian McLoughlin, Lam Pham, Yan Song, Xiaoxiao Miao, Huy Phan, Pengfei Cai, Qing Gu, Jiang Nan, Haoyu Song, and Donny Soh, "Spectrogram features for audio and speech analysis," *Applied Sciences*, vol. 16, no. 2, pp. 572, 2026.

[49] Malcolm Slaney, "Auditory toolbox version 2: A matlab toolbox for auditory modeling work," Tech. Rep. 1998-010, Interval Research Corporation, 1998.

[50] Steven B. Davis and Paul Mermelstein, "Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 28, no. 4, pp. 357–366, 1980.

[51] Beth Logan, "Mel frequency cepstral coefficients for music modeling," in *Proceedings of the International Symposium on Music Information Retrieval (ISMIR)*, 2000.

[52] Abel Herrera and Fernando Del Rio, "Frequency bark cepstral coefficients extraction for speech analysis by synthesis," *The Journal of the Acoustical Society of America*, vol. 128, no. 4, pp. 2290–2290, 2010.

[53] Rujia Chen, Akbar Ghobakhlou, and Ajit Narayanan, "Interpreting cnn models for musical instrument recognition using multi-spectrogram heatmap analysis: A preliminary study," *Frontiers in Artificial Intelligence*, vol. 7, pp. 1499913, 2024.

[54] Haruka Kondo, Sotaro Kondoh, and Shinya Fujii, "Perceived vibrato and the singing power ratio explain overall evaluations in opera singing," *Frontiers in Psychology*, vol. 16, pp. 1568982, 2025.

[55] Judith C. Brown, "Calculation of a constant q spectral transform," *Journal of the Acoustical Society of America*, vol. 89, no. 1, pp. 425–434, 1991.

[56] Kin Wai Cheuk, Kat Agres, and Dorien Herremans, "The impact of audio input representations on neural network based music transcription," in *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*. 2020, pp. 1–6, IEEE.

[57] Maarten Grachten and Gerhard Widmer, "Linear basis models for prediction and analysis of musical expression," *Journal of New Music Research*, vol. 41, no. 4, pp. 311–322, 2012.

[58] Gabriel Jones and Anders Friberg, "Probing the underlying principles of dynamics in piano performances using a modelling approach," *Frontiers in Psychology*, vol. 14, pp. 1269715, 2023.

[59] Werner Goebl, Roberto Bresin, and Ari Galembo, "Once again: The perception of piano touch and tone. can touch audibly change piano sound independently of intensity?," in *Proceedings of the International Symposium on Musical Acoustics (ISMA)*, 2004, pp. 332–335.

[60] Sebastian Ewert and Meinard Müller, "Estimating note intensities in music recordings," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2011, pp. 385–388.

[61] H. Kim, M. Miron, and X. Serra, "Score-informed midi velocity estimation for piano performance by film conditioning," in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2023, pp. 139–147.

[62] Apple, "Use step input recording in logic pro for mac," Accessed: 2026-03-29.

[63] MuseScore Developers, "dynamic.cpp," MuseScore GitHub repository, Accessed: 2026-03-29.

[64] Avid, *Sibelius 7 Sounds*, Avid, 2012, Accessed: 2026-03-29.

[65] Apple, "Add dynamic marks, slurs, and crescendi to a score in logic pro for mac," Accessed: 2026-03-29.

[66] Axel Berndt and Tilo Hähnel, "Modelling musical dynamics," in *Proceedings of the 5th Audio Mostly Conference: A Conference on Interaction with Sound*, 2010, pp. 1–8.

[67] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore, and Douglas Eck, "Onsets and frames: Dual-objective piano transcription," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 50–57.

[68] Curtis Hawthorne, Ian Simon, Rigel Swavely, Ethan Manilow, and Jesse Engel, "Sequence-to-sequence piano transcription with transformers," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2021, pp. 246–253.

[69] Keisuke Toyama, Taketo Akama, Yukara Ikemiya, Yuhta Takida, Wei-Hsiang Liao, and Yuki Mitsufuji, "Automatic piano transcription with hierarchical frequency-time transformer," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 215–222.

[70] Yujia Yan, Frank Cwitkowitz, and Zhiyao Duan, "Skipping the frame-level: Event-based piano transcription with neural semi-crfs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, vol. 34, pp. 20583–20595.

[71] Yujia Yan and Zhiyao Duan, "Scoring time intervals using non-hierarchical transformer for automatic piano transcription," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 973–980.

[72] Yu-Te Wu, Berlin Chen, and Li Su, "Polyphonic music transcription with semantic segmentation," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 166–170.

[73] Rainer Kelz, Sebastian Böck, and Gerhard Widmer, "Deep polyphonic ADSR piano note transcription," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 246–250.

[74] Weixing Wei, Peilin Li, Yi Yu, and Wei Li, "HPPNet: Modeling the harmonic structure and pitch invariance in piano transcription," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 709–716.

[75] Kin Wai Cheuk, Dorien Herremans, and Li Su, "ReconVAT: A semi-supervised automatic music transcription framework for low-resource real-world data," in *Proceedings of the ACM International Conference on Multimedia*, 2021, pp. 3918–3926.

[76] Josh Gardner, Ian Simon, Ethan Manilow, Curtis Hawthorne, and Jesse Engel, "MT3: Multi-task multitrack music transcription," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[77] Wei-Tsung Lu, Ju-Chiang Wang, and Yun-Ning Hung, "Multitrack music transcription with a time-frequency perceiver," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[78] Taehyeon Kim, Man-Je Kim, and Chang Wook Ahn, "PF2N: Periodicity–frequency fusion network for multi-instrument music transcription," *Mathematics*, vol. 13, no. 11, pp. 1708, 2025.

[79] Dasaem Jeong, Taegyun Kwon, Yoojin Kim, Kyogu Lee, and Juhan Nam, "Virtuosonet: A hierarchical rnn-based system for modeling expressive piano performance," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2019, pp. 908–915.

[80] Ilya Borovik and Vladimir Viro, "Scoreperformer: Expressive piano performance rendering with fine-grained control," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 588–596.

[81] Chien-Sheng Kuo, Woei-Kae Chen, Chien-Hung Liu, and Shingchern D You, "Velocity prediction for midi notes with deep learning," in *2021 IEEE International Conference on Consumer Electronics-Taiwan (ICCE-TW)*, 2021, pp. 1–2.

[82] Taein Kim and Yunho Kim, "Piano velocity prediction using a seq2seq model with attention mechanism," in *2023 Autumn Meeting of the Acoustical Society of Japan*. 2023, pp. 1467–1470, Acoustical Society of Japan.

[83] H. Kim and X. Serra, "DiffVel: Note-level midi velocity estimation for piano performance by a double conditioned diffusion model," in *Proceedings of the International Symposium on Computer Music Multidisciplinary Research (CMMR)*, 2023, pp. 197–208.

[84] Z. Shen, B. Kong, and X. Dong, "Transformer with linear-window attention for feature matching," *IEEE Access*, vol. 11, pp. 121202–121211, 2023.

[85] Ke Chen, Xingjian Du, Bilei Zhu, Zejun Ma, Taylor Berg-Kirkpatrick, and Shlomo Dubnov, "Hts-at: A hierarchical token-semantic audio transformer for sound classification and detection," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 646–650.

[86] Ben Maman and Amit H Bermano, "Unaligned supervision for automatic music transcription in the wild," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2022, pp. 14918–14934.

[87] Drew Edwards, Simon Dixon, Emmanouil Benetos, Akira Maezawa, and Yuta Kusaka, "A data-driven analysis of robust automatic piano transcription," *IEEE Signal Processing Letters*, vol. 31, pp. 681–685, 2024.

[88] Sebastian Strahl and Meinard Müller, "Semi-supervised piano transcription using pseudo-labeling techniques," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 173–181.

[89] Qi Wang, Mingkuan Liu, and Maoshen Jia, "Multitask frequency-time feature-wise linear modulation for piano transcription with pedal," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 33, pp. 1049–1062, 2025.

[90] Emmanouil Benetos, Simon Dixon, Dimitrios Giannoulis, Holger Kirchhoff, and Anssi Klapuri, "Automatic music transcription: Challenges and future directions," *Journal of Intelligent Information Systems*, vol. 41, no. 3, pp. 407–434, 2013.

[91] W. M. Szeto, K. H. Wong, and C. H. Wong, "Finding intensities and temporal characteristics in piano music," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2005, pp. 214–219.

[92] Sam van Herwaarden, Maarten Grachten, and W. Bas de Haas, "Predicting expressive dynamics in piano performances using neural networks," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2014, pp. 47–52.

[93] Dasaem Jeong and Juhan Nam, "Note intensity estimation of piano recordings by score-informed nmf," *Journal of the Audio Engineering Society*, vol. 65, no. 4/5, pp. 263–273, June 2017.

[94] Dasaem Jeong, Taegyun Kwon, and Juhan Nam, "A timbre-based approach to estimate key velocity from polyphonic piano recordings," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 120–127.

[95] Federico Simonetta, Stavros Ntalampiras, and Federico Avanzini, "Acoustics-specific piano velocity estimation," in *Proceedings of the IEEE International Workshop on Multimedia Signal Processing (MMSP)*, 2022.

[96] Katerina Kosta, Oscar F. Bandtlow, and Elaine Chew, "MazurkaBL: score-aligned loudness, beat, and expressive markings data for 2000 chopin mazurka recordings," in *Proc. Int. Conf. Tech. for Music Notation and Rep.*, 2018, pp. 85–94.

[97] Katerina Kosta, *Computational Modelling and Quantitative Analysis of Dynamics in Performed Music*, Ph.D. thesis, Centre for Digital Music, Queen Mary University of London, 2017.

[98] Sebastian Böck, Florian Krebs, and Gerhard Widmer, "Joint beat and downbeat tracking with recurrent neural networks," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2016, pp. 255–261.

[99] Sebastian Böck and Matthew E.P. Davies, "Deconstruct, analyse, reconstruct: How to improve tempo, beat, and downbeat estimation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2020, pp. 574–582.

[100] Jingwei Zhao, Gus Xia, and Ye Wang, "Beat transformer: Demixed beat and downbeat tracking with dilated self-attention," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 169–177.

[101] Francesco Foscarin, Jan Schlüter, and Gerhard Widmer, "Beat this! accurate beat tracking without dbn postprocessing," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 962–969.

[102] Dichucheng Li, Mingjin Che, Wenwu Meng, Yulun Wu, Yi Yu, Fan Xia, and Wei Li, "Frame-level multi-label playing technique detection using multi-scale network and self-attention mechanism," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[103] FluidSynth Project, "Fluidsynth," 2025, Version 2.5.2; real-time software synthesizer based on the SoundFont 2 specifications; accessed 2026-04-13.

[104] Alexander Holm, "Acoustic grand piano: Salamander Grand Piano," FreePats Project sound bank, June 2020, Version V3+2020-06-02; Creative Commons Attribution 3.0; accessed 2026-04-18.

[105] Modartt, "Pianoteq 8," 2022.

[106] Jesse Engel, Lamtharn Hantrakul, Chenjie Gu, and Adam Roberts, "DDSP: differentiable digital signal processing," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020.

[107] Ben Hayes, Jordie Shier, György Fazekas, Andrew McPherson, and Charalampos Saitis, "A review of differentiable digital signal processing for music and speech synthesis," *Frontiers in Signal Processing*, vol. 3, pp. 1284100, 2024.

[108] Yusong Wu, Ethan Manilow, Yi Deng, Rigel Swavely, Kyle Kastner, Tim Cooijmans, Aaron Courville, Cheng-Zhi Anna Huang, and Jesse Engel, "MIDI-DDSP: Detailed control of musical performance via hierarchical modeling," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[109] Lenny Renault, Rémi Mignot, and Axel Roebel, "DDSP-Piano: A neural sound synthesizer informed by instrument knowledge," *Journal of the Audio Engineering Society*, vol. 71, no. 9, pp. 552–565, 2023.

[110] Andrew Wiggins and Youngmoo Kim, "A differentiable acoustic guitar model for string-specific polyphonic synthesis," in *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2023, pp. 1–5.

[111] Nicolas Jonason, Xin Wang, Erica Cooper, Lauri Juvela, Bob L. T. Sturm, and Junichi Yamagishi, "DDSP-Based neural waveform synthesis of polyphonic guitar performance from string-wise midi input," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2024, pp. 208–215.

[112] Marco A. Martínez Ramírez, Oliver Wang, Paris Smaragdis, and Nicholas J. Bryan, "Differentiable signal processing with black-box audio effects," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021.

[113] Siyuan Shan, Lamtharn Hantrakul, Jitong Chen, Matt Avent, and David Trevelyan, "Differentiable wavetable synthesis," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 4598–4602.

[114] Riccardo Simionato and Stefano Fasciani, "Neural sample-based piano synthesis," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2025, pp. 441–448.

[115] Qiming Yang, Ryan Leistikow, and Yi Zang, "Flowsynth: Instrument generation through distributional flow matching and test-time search," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 15102–15106.

[116] Yongyi Zang, Yi Zhong, Frank Cwitkowitz, and Zhiyao Duan, "SynthTab: Leveraging synthesized data for guitar tablature transcription," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1286–1290.

[117] Huan Zhang, Jingjing Tang, Syed Rifat Mahmud Rafee, Simon Dixon, and György Fazekas, "ATEPP: A dataset of automatically transcribed expressive piano performance," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 446–453.

[118] M. Müller, V. Konz, W. Bogler, and V. Arifi-Müller, "Saarland music data (smd)," in *Late-Breaking and Demo Session of the 12th Int. Conf. on Music Information Retrieval*, 2011.

[119] Valentin Emiya, Nancy Bertin, Bertrand David, and Roland Badeau, "Maps: A piano database for multipitch estimation and automatic transcription of music," Tech. Rep. 2010D017, INRIA, 2010.

[120] Werner Goebl, "The vienna 4x22 piano corpus," https://doi.org/10.21939/4X22, 1999, [Online]. Available: https://github.com/CPJKU/vienna4x22.

[121] Patricia Hu and Gerhard Widmer, "The batik-plays-mozart corpus: Linking performance to score to musicological annotations," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 582–589.

[122] Qingyang Xi, Rachel M. Bittner, Johan Pauwels, Xuzhou Ye, and Juan Pablo Bello, "Guitarset: A dataset for guitar transcription," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 453–460.

[123] Xavier Riley, Drew Edwards, and Simon Dixon, "High resolution guitar transcription via domain adaptation," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 1051–1055.

[124] Xavier Riley, Zixun Guo, Andrew C. Edwards, and Simon Dixon, "GAPS: a large and diverse classical guitar dataset and benchmark transcription model," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 611–617.

[125] John Thickstun, Zaid Harchaoui, and Sham M. Kakade, "Learning features of music from scratch," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.

[126] Na Wang, Guo-Dong Chen, and Ying Tian, "Image colorization algorithm based on deep learning," *Symmetry*, vol. 14, no. 11, 2022.

[127] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, vol. 30.

[128] Dasaem Jeong, Taegyun Kwon, and Juhan Nam, "Note-intensity estimation of piano recordings using coarsely aligned midi score," *Journal of the Audio Engineering Society*, vol. 68, no. 1/2, pp. 34–47, January 2020.

[129] Huan Zhang, Emmanouil Karystinaios, Simon Dixon, Gerhard Widmer, and Carlos Eduardo Cancino-Chacón, "Symbolic music representations for classification tasks: A systematic evaluation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023.

[130] International Telecommunication Union, "Method for the subjective assessment of intermediate quality level of audio systems," Recommendation ITU-R BS.1534-3, International Telecommunication Union, 2015.

[131] Qualtrics, "Qualtrics corexm," 2024.

[132] Hsiao-Tzu Hung, Joann Ching, Seungheon Doh, Nabin Kim, Juhan Nam, and Yi-Hsuan Yang, "Emopia: A multi-modal pop piano dataset for emotion recognition and emotion-based music generation," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2021, pp. 318–325.

[133] Qiuqiang Kong, Bochen Li, Jitong Chen, and Yuxuan Wang, "Giantmidi-piano: A large-scale midi dataset for classical piano music," *Transactions of the International Society for Music Information Retrieval*, May 2022.

[134] Drew Edwards, Simon Dixon, and Emmanouil Benetos, "Pijama: Piano jazz with automatic midi annotations," *Transactions of the International Society for Music Information Retrieval*, vol. 6, no. 1, pp. 89–102, 2023.

[135] Huan Zhang, Shreyan Chowdhury, Carlos Eduardo Cancino-Chacón, Jinhua Liang, Simon Dixon, and Gerhard Widmer, "Dexter: Learning and controlling performance expression with diffusion models," *Applied Sciences*, vol. 14, no. 15, pp. 6543, 2024.

[136] Ilya Borovik, Dmitrii Gavrilev, and Vladimir Viro, "SyMuPe: Affective and controllable symbolic music performance," in *Proceedings of the 33rd ACM International Conference on Multimedia*, Dublin, Ireland, 2025, MM '25, pp. 10699–10708, Association for Computing Machinery.

[137] Lukáš Samuel Marták, Patricia Hu, and Gerhard Widmer, "Sound and music biases in deep music transcription models: a systematic analysis," *EURASIP J. Audio Speech Music. Process.*, vol. 2026, pp. 5, 2026.

[138] Johannes Zeitler, Ben Maman, and Meinard Müller, "Robust and accurate audio synchronization using raw features from transcription models," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2024, pp. 120–127.

[139] Sungkyun Chang, Simon Dixon, and Emmanouil Benetos, "Rumaa: Repeat-aware unified music audio analysis for score-performance alignment, transcription, and mistake detection," in *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2025, pp. 1–5.

[140] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang, "Conformer: Convolution-augmented transformer for speech recognition," in *Proceedings of the Annual Conference of the International Speech Communication Association (INTERSPEECH)*, 2020, pp. 5036–5040.

[141] Colin Raffel, Brian McFee, Eric J. Humphrey, Justin Salamon, Oriol Nieto, Dawen Liang, and Daniel P. W. Ellis, "mir_eval: A transparent implementation of common MIR metrics," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2014, pp. 367–372.

[142] Roberto San Millán-Castillo, Eduardo Latorre-Iglesias, Martin Glesser, Salomé Wanty, Daniel Jiménez-Caminero, and José María Álvarez-Jimeno, "MOSQITO: an open-source and free toolbox for sound quality metrics in the industry and education," in *INTER-NOISE and NOISE-CON Congress and Conference Proceedings*, 2021, pp. 1164–1175.

[143] Elias Pampalk, "A matlab toolbox to compute music similarity from audio," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2004, pp. 254–257.

[144] Tzu-Ching Hung, Jingjing Tang, Kit Armstrong, Yi-Cheng Lin, and Yi-Wen Liu, "Eme33: A dataset of classical piano performances guided by expressive markings with application in music rendering," in *Proc. of IEEE Int. Conf. on Big Data*, 2024, pp. 3174–3180.

[145] Carlos Eduardo Cancino-Chacón and Ivan Pilkov, "The rach3 dataset: Towards data-driven analysis of piano performance rehearsal," 2024, pp. 28–41.

[146] S. Rhyu, S. Kim, and K. Lee, "Sketching the expression: Flexible rendering of expressive piano performance with self-supervised learning," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2022, pp. 178–185.

[147] Jingjing Tang, Erica Cooper, Xin Wang, Junichi Yamagishi, and György Fazekas, "Towards an integrated approach for expressive piano performance synthesis from music scores," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[148] Alia Morsi, Suhit Chiruthapudi, Silvan Peter, Ivan Pilkov, Laura Bishop, Akira Maezawa, Xavier Serra, and Carlos Eduardo Cancino-Chacón, "Enabling empirical analysis of piano performance rehearsal with the rach3 midi dataset," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2025, pp. 484–491.

[149] Yuta Kusaka and Akira Maezawa, "Learn from virtual guitar: A comparative analysis of automatic guitar transcription using synthetic and real audio," in *Proceedings of the IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2025, pp. 1–5.

[150] Gakusei Sato and Taketo Akama, "Annotation-free automatic music transcription with scalable synthetic data and adversarial domain confusion," in *Proceedings of the 2024 IEEE International Conference on Multimedia and Expo (ICME)*, 2024, pp. 1–6.

[151] Ting-Kang Wang, Yueh-Po Peng, Li Su, and Vincent K. M. Cheung, "Vioptt: Violin technique-aware transcription from synthetic data augmentation," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 15947–15951.

[152] Oren Barkan, Shlomi Shvartzman, Noy Uzrad, Moshe Laufer, Almog Elharar, and Noam Koenigstein, "InverSynth II: Sound matching via self-supervised synthesizer-proxy and inference-time finetuning," in *Proceedings of the International Society for Music Information Retrieval Conference (ISMIR)*, 2023, pp. 642–648.

[153] Paolo Combes, Stefan Weinzierl, and Klaus Obermayer, "Neural proxies for sound synthesizers: Learning perceptually informed preset representations," *Journal of the Audio Engineering Society*, vol. 73, no. 9, pp. 561–577, 2025.

[154] Zhanhong He, Roberto Togneri, and Defeng Huang, "Score-informed transformer for refining MIDI velocity in automatic piano transcription," in *Proceedings of the Sound and Music Computing Conference (SMC)*, 2026, Accepted, to appear.

[155] Simon Dixon, "Onset detection revisited," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2006, pp. 133–137.

[156] Sebastian Böck and Gerhard Widmer, "Maximum filter vibrato suppression for onset detection," in *Proceedings of the International Conference on Digital Audio Effects (DAFx)*, 2013, pp. 55–61.

[157] FreePats Project, "Nylon-string acoustic guitar: Spanish Classical Guitar," FreePats Project sound bank, June 2019, Version 2019-06-18; Creative Commons CC0 1.0 public domain dedication; accessed 2026-04-18.

[158] FreePats Project, "Acoustic grand piano: YDP Grand Piano," FreePats Project sound bank, Aug. 2016, Version 2016-08-04; Creative Commons Attribution 3.0; accessed 2026-04-18.